

Getting started

Thank you for choosing Freenove products!

First, read the document [About_Battery.pdf](#) in the unzipped folder.

If you did not download the zip file, please download it and unzip it via link below.

https://github.com/Freenove/Freenove_4WD_Car_Kit/archive/master.zip

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

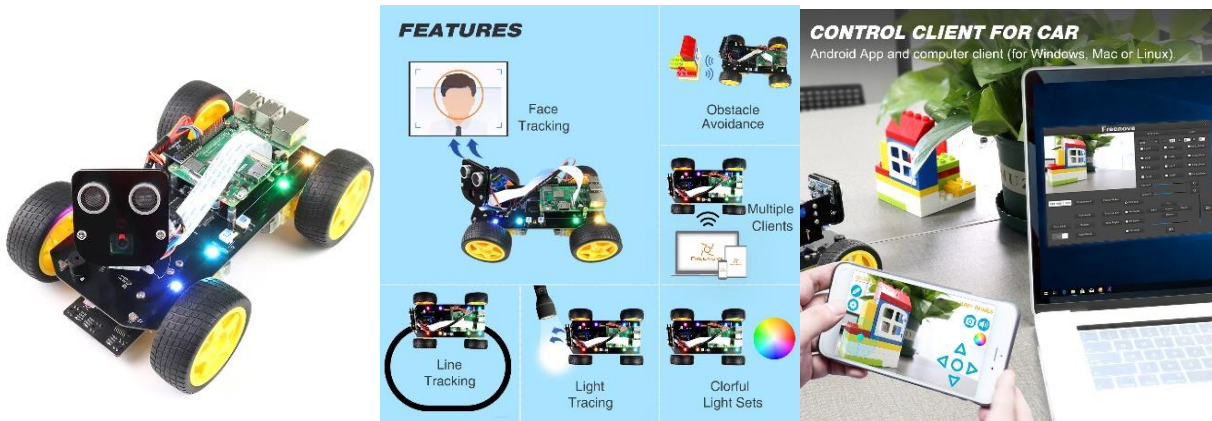
- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down! When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

Car and Robot for Raspberry Pi

We also have cars and robot kit for Raspberry Pi. If you are interesting in them, please visit our website for details.

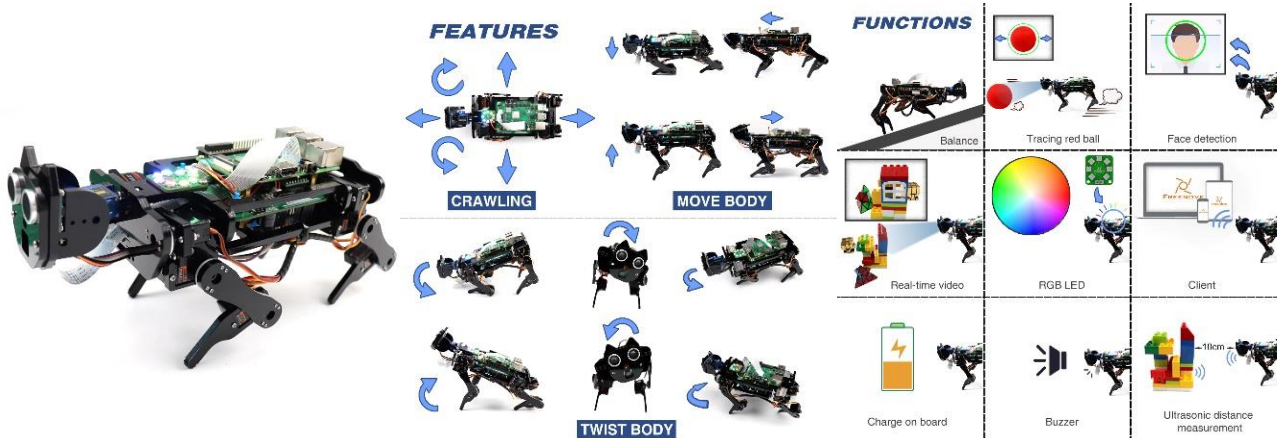
<http://www.freenove.com/store.html>

FNK0043 Freenove 4WD Smart Car Kit for Raspberry Pi



<https://www.youtube.com/watch?v=4Zv0GZUQjZc>

FNK0050 Freenove Robot Dog Kit for Raspberry Pi



https://www.youtube.com/watch?v=7BmIZ8_R9d4&t=35s

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely, but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Arduino® is a trademark of Arduino LLC (<https://www.arduino.cc/>).

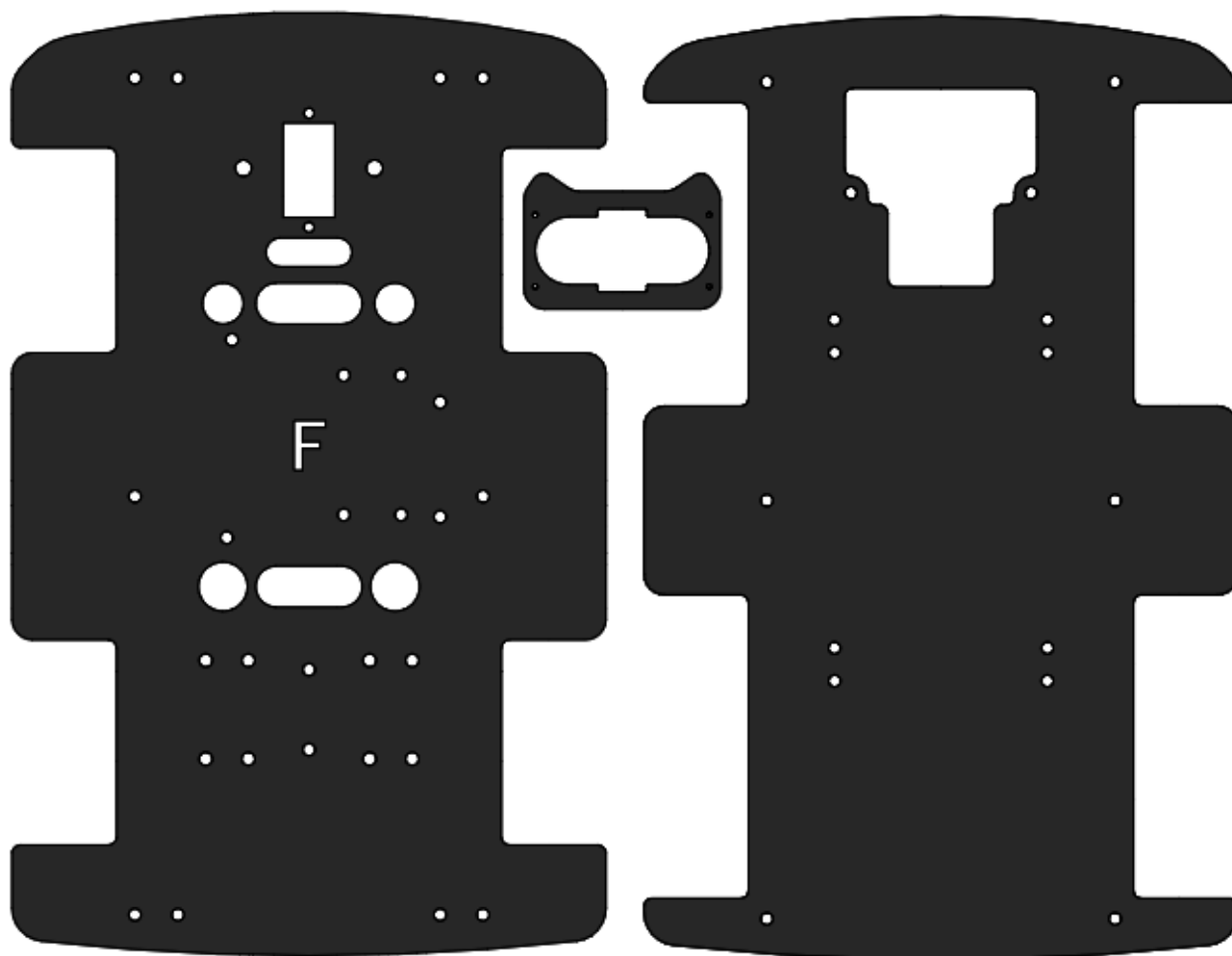
Contents

Getting started	1
Get Support and Offer Input.....	1
Safety and Precautions.....	1
Car and Robot for Raspberry Pi	2
About Freenove	3
Contents	4
List	5
Preface	11
Control Board and Software	12
Chapter 0 Software, Assembly and Play.....	15
0.1 Download and Install IDE.....	15
0.2 Upload Servo Code (Necessary)	18
0.3 Assembly.....	21
0.4 How to Play.....	32
Chapter 1 Control Basic Components.....	43
1.1 Motor	43
1.2 Buzzer and Battery level	52
1.3 RGB.....	58
1.4 Integrate functions	63
Chapter 2 Obstacle avoidance.....	71
2.1 Servo	71
2.2 Ultrasonic ranging module	73
2.3 Automatic Obstacle Avoidance Car.....	77
Chapter 3 Line tracking	90
3.1 Line tracking sensor	90
3.2 Line tracking car	93
Chapter 4 IR control.....	97
4.1 Infrared remote and receiver	97
4.2 IR Remote Car	100
4.3 Multifunctional IR Remote Car	103
Chapter 5 RF Remote Control.....	111
5.1 Remote Control	111
5.2 Receive RF Remote Control Data	114
5.3 RF Remote Car	119
5.4 Multifunctional RF24 Remote Car	123
Chapter 6 Bluetooth control.....	137
6.1 Set Bluetooth and Receive Data	137
6.2 Bluetooth Remote Car.....	147
6.3 Multifunctional Bluetooth Remote Car.....	154
What is next?	165

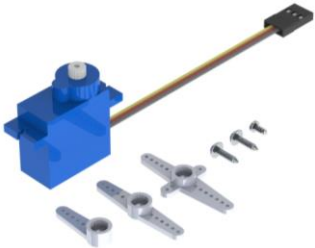
List

Acrylic Parts

The surface of the acrylic parts is covered with a layer of protective film. You need to remove it first. Some holes in the acrylic parts may have residues. You also need to clean them before using.

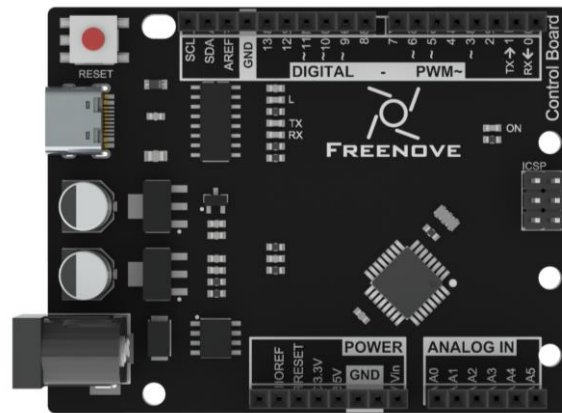


Mechanical Parts

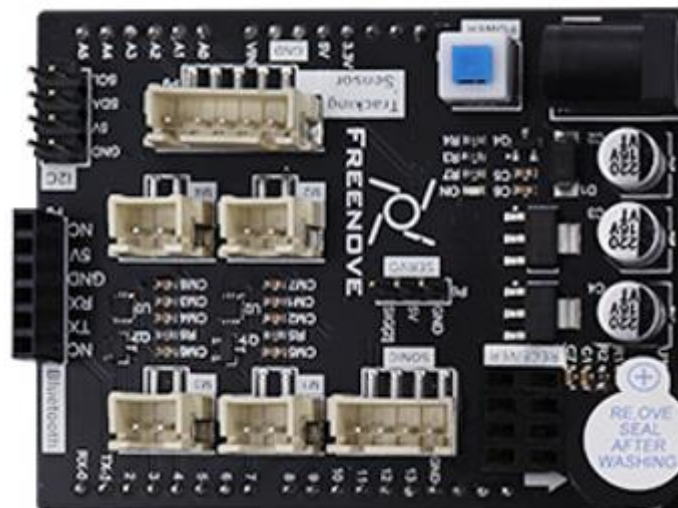
 M3*40 Copper Standoff x6 Freenove	 M3*10 Copper Standoff x5 Freenove	 M3*10 Countersunk Head Screw x5 Freenove
 M2*10 Screw x3 Freenove	 M3*8 Screw x12 Freenove	 M3*6 Screw x22 Freenove
 M2 Nut x3 Freenove	 M3 Nut x16 Freenove	 M1.4*4 Self-tapping Screw x5 Freenove
Servo x1 	Motor x4  Caution: Do not remove the cable tie from the motor; otherwise, the motor cable may become detached.	
Driver wheel x 4 	Motor bracket package x 4 Screw M3*8 , Nut M3, Screw M3*30 	

Electronic Parts

Freenove control board (Your kit will randomly contain **one control board**)



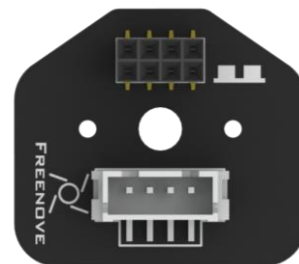
Freemove 4WD extension board

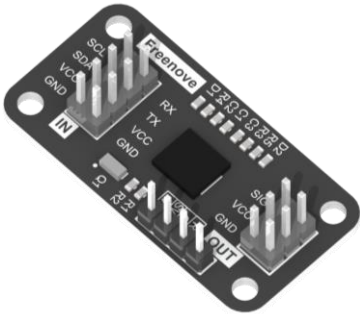
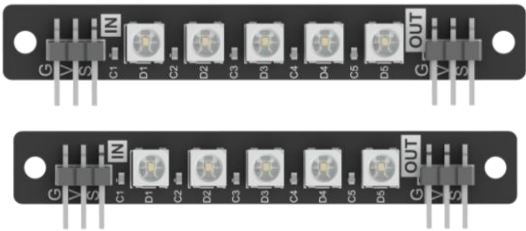
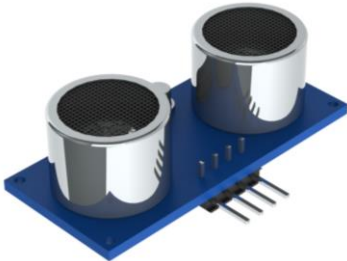
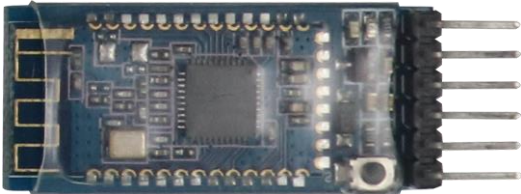
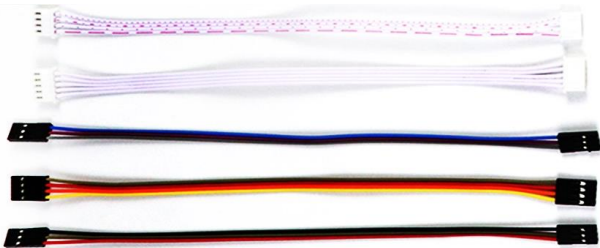
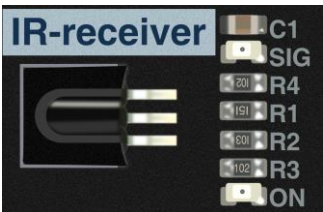
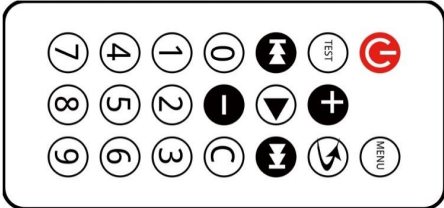


Line-tracking infrared sensor



Ultrasonic module connector



WS2812B_LED_controller  A small black PCB module with a central black IC. It has multiple pin headers labeled with functions like SCL, SDA, VCC, GND, RX, TX, VCC, GND, and OUT. The brand name 'Freenove' is printed on the board.	WS2812B_LED  Two identical black PCB modules, each featuring a row of eight WS2812B LEDs. They have pin headers labeled G, V, S, G1, D1, D2, D3, D4, D5, D6, D7, D8, G, V, S, and OUT.
Ultrasonic module  A blue PCB module with two cylindrical ultrasonic sensors mounted on it. It has a 4-pin header at the bottom.	Bluetooth  A blue PCB module with a central microcontroller chip and various surface components. It has a 4-pin header on the left and a 5-pin header on the right.
Wires  Four different types of jumper wires: a white flat ribbon cable, a white standard jumper wire, a blue standard jumper wire, and a rainbow-colored standard jumper wire.	USB Cable  Two black USB cables: one with a standard USB-A connector and another with a USB-C connector.
IRReceiver  A black PCB module with an IR receiver diode and several resistors. It has a 3-pin header. Labels include 'IR-receiver', 'C1', 'SIG', 'R4', 'R1', 'R2', 'R3', and 'ON'.	Tape  A roll of black adhesive tape with a green patterned backing.
IR remote  A white rectangular IR remote control with a numeric keypad (0-9), directional buttons, and function buttons like TEST, MENU, and a red power button.	Battery holder  A black plastic battery holder with a 9V battery compartment and a DC power jack connector.

RF remote kit package (**only contained in the Version with RF remote control**)



Tools

Cross Screwdriver x1



Socket x1 (**only one is included in the kit**)



Needed but NOT Included

18650 3.7V **rechargeable** lithium battery x2

It is easier to find proper battery on [eBay](#) than Amazon.

Please refer to **About_Battery.pdf** in the unzipped folder.



Preface

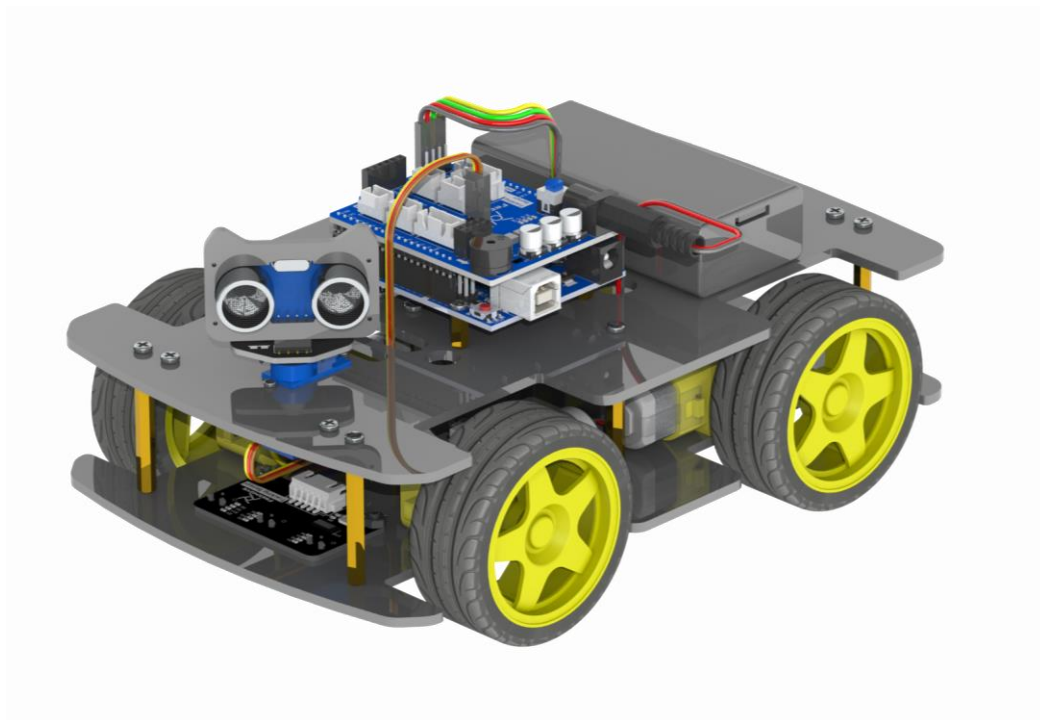
This 4WD Smart car is based on Arduino.

This tutorial will start from controlling electronic components such as LED, motor, servo, to building multifunctional car.

Please follow the tutorial step by step patiently.

If you have any concerns, please feel free to contact us. Our support team will provide quick and free customer service.

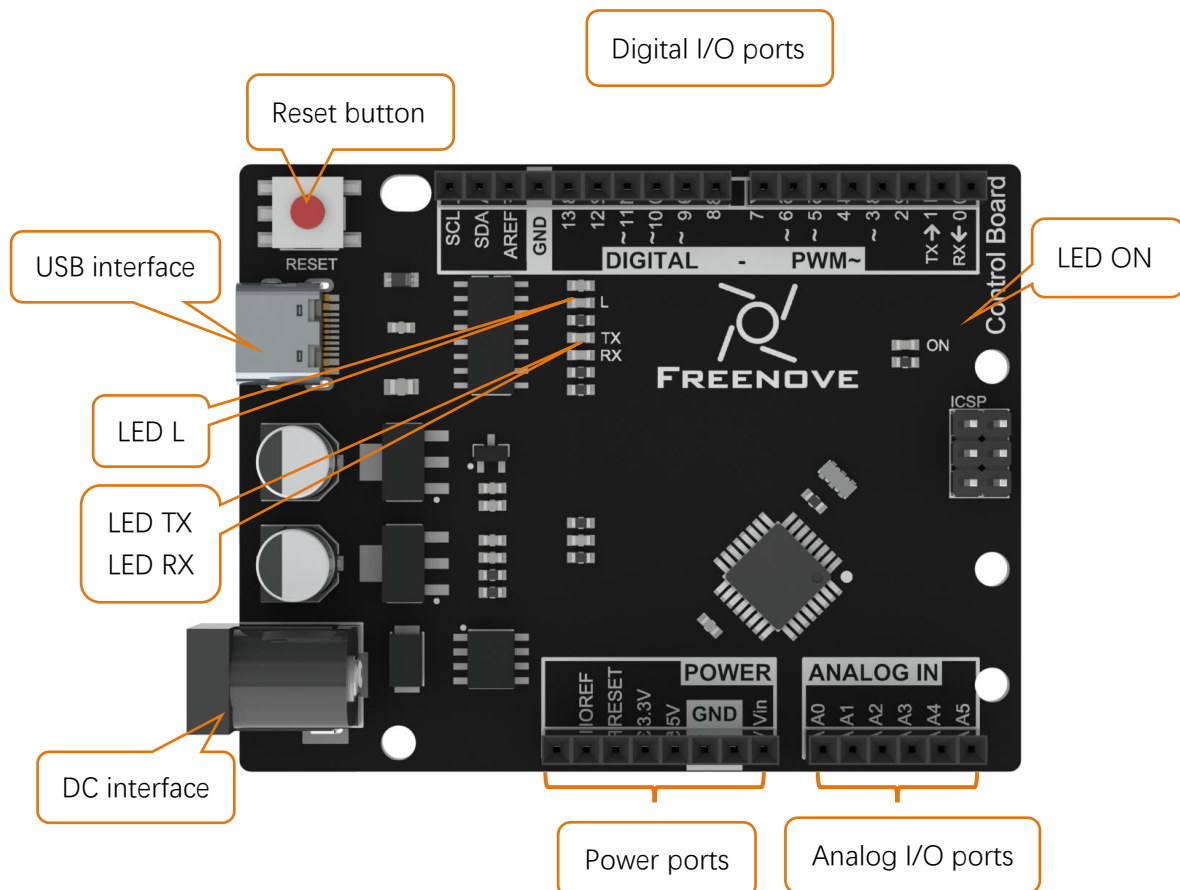
support@freenove.com



Control Board and Software

Control Board

Diagram of Freenove Control board is shown below:

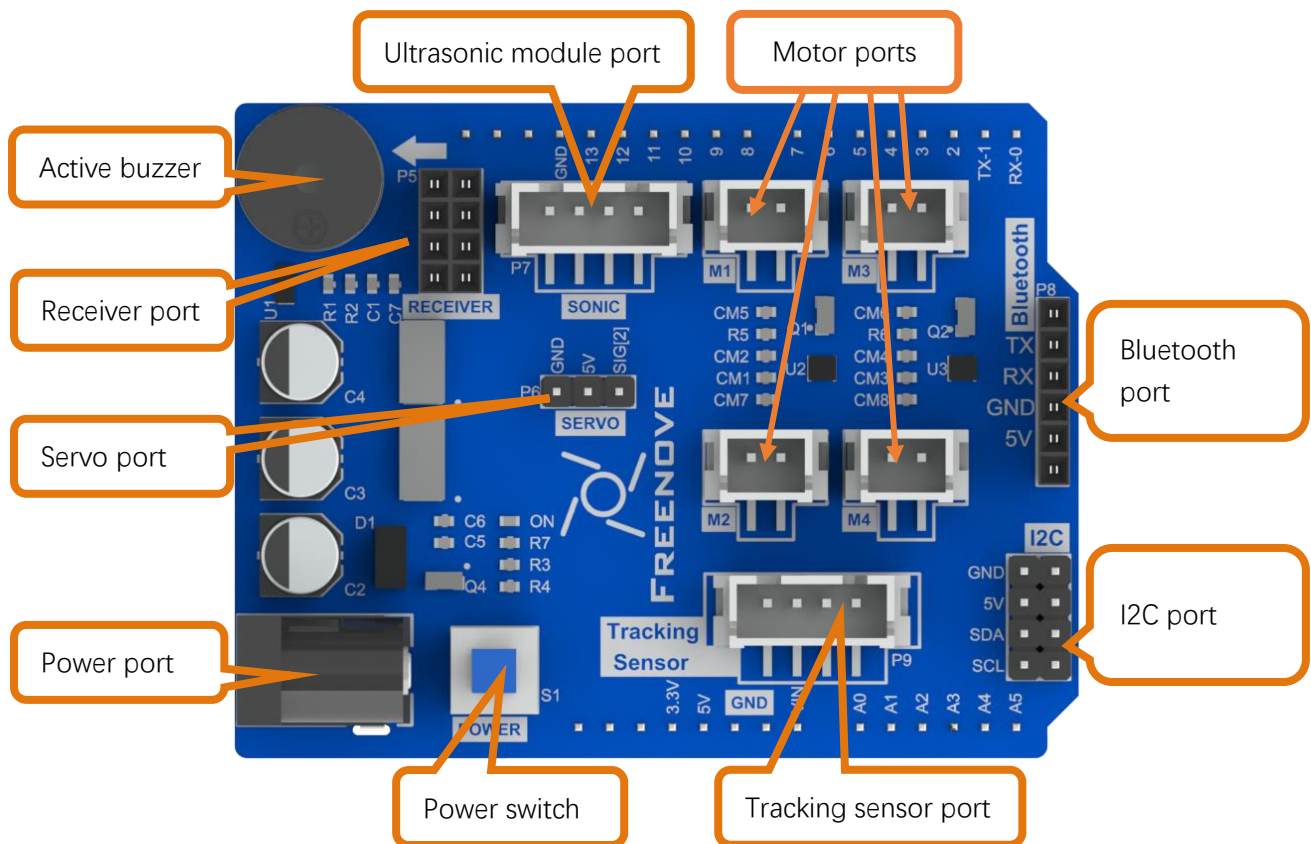


- Digital I/O ports is used to connect to other components or modules, to receive an input signal, or to send a control signal. Usually, we name it by adding a "D" in front of the number, such as D13.
- Only digital I/O ports with "~" can output PWM, Pin 3, 5, 6, 9, 10, 11.
- USB interface is used to provide power, upload code or communicate with PC.
- LED L is connected to digital I/O port 13 (D13).
- LED TX, RX is used to indicate the state of the serial communication.
- DC interface is connected with DC power to provide power for the board.
- Power ports can provide power for electronic components and modules.
- Analog I/O ports can be used to measure analog signals.
- LED ON is used to indicate the power state.

If you want to learn more about how to use Freenove control board to create interesting projects, please visit <http://www.freenove.com/store.html> for Arduino kits designed for beginners.

Extension Board

Freenove 4WD Extension board is below:



Receiver port can connect to RF module or IR module.

Relationship between extension board and the control board is as below.

Pins of Arduino	Ports of extension	Pin multiplexing	Description
0	UART-Bluetooth		Bluetooth
1			
2	Servo		
3	Direction-Right		Direction of right motor
4	Direction-Left		Direction of left motor
5	Motor-Right		Speed of right motor -PWM
6	Motor-Left		Speed of left motor -PWM
7	Trig		Ultrasonic module
8	Echo		
9	SPI-NRF24L01	IR-remote	CE/IR-remote
10			CS
11	SPI-NRF24L01		MOSI
12			MISO
13			SCK
A0	Battery voltage	Buzzer	The ADC uses an internal 5V reference and the acquisition voltage is the battery voltage *1/4. Output to the Buzzer through the comparator. When higher than the 4.5V, output HIGH.
A1	Line-tracking sensor		Left
A2			Middle
A3			Right
A4	I2C		SDA
A5			SCL

Chapter 0 Software, Assembly and Play

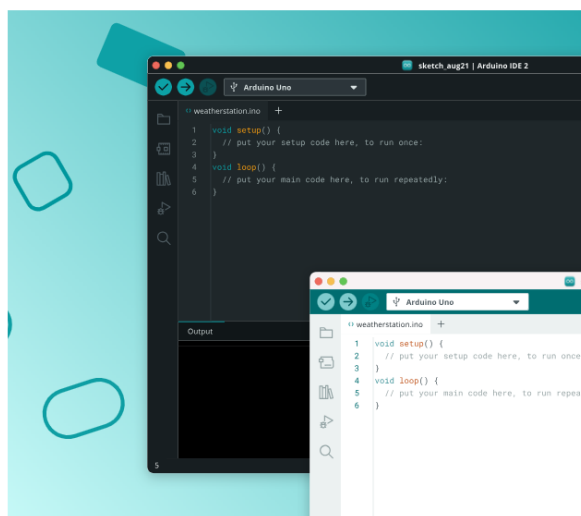
If you have any concerns, please feel free to contact us via support@freenove.com

0.1 Download and Install IDE.

Arduino Software (IDE) is used to write and upload the code for Arduino Board.

First, install Arduino Software (IDE): visit <https://www.arduino.cc/en/software/>

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6

[Release notes](#)

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD

Windows Win 10 or newer (64-bit)

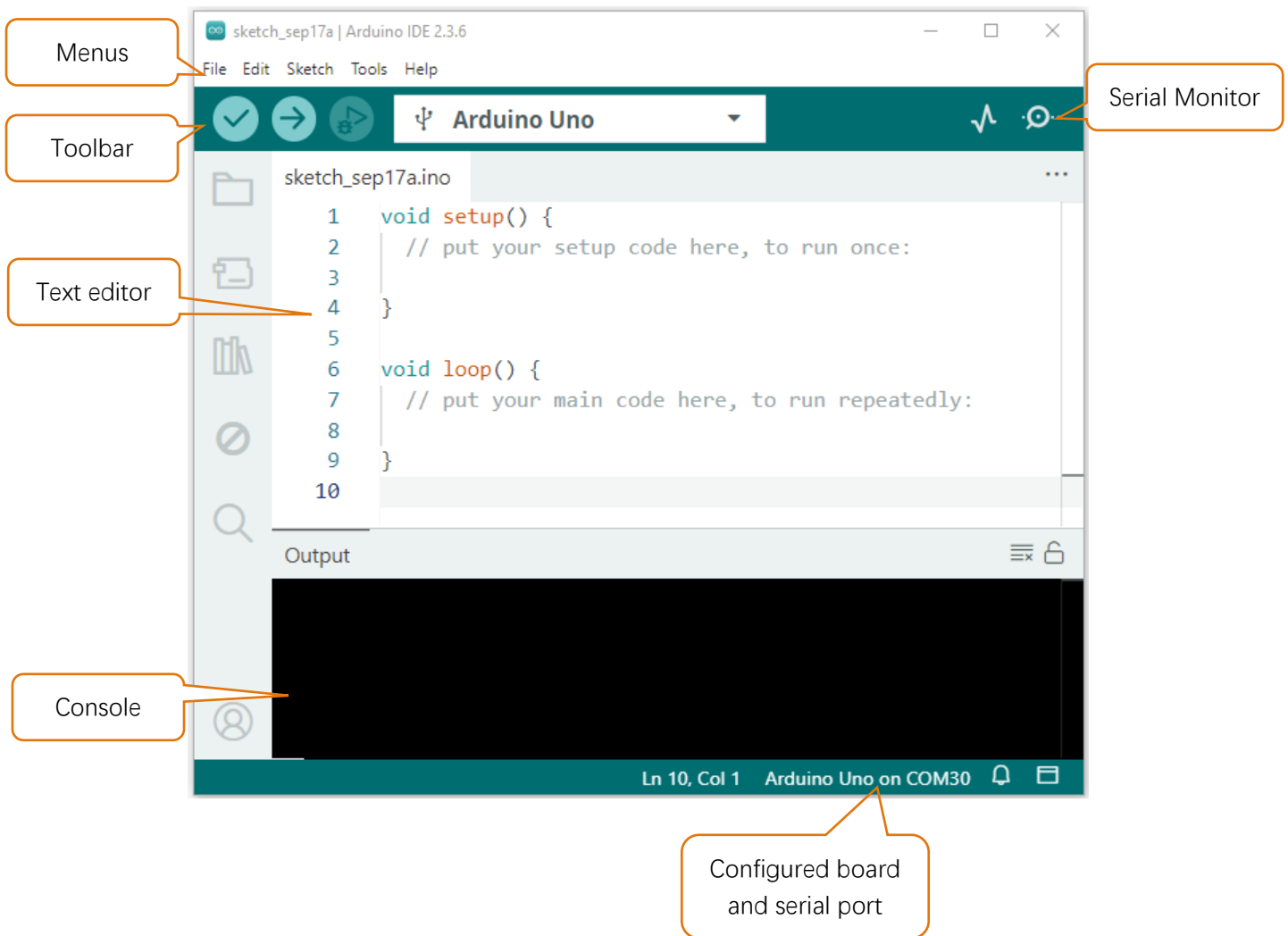
- Windows MSI installer
- Windows ZIP file
- Linux AppImage (64-bit X86-64)
- Linux ZIP file (64-bit X86-64)
- macOS Intel 10.15 Catalina or newer (64-bit)
- macOS Apple Silicon 11 Big Sur or newer (64-bit)

Legacy IDE (1.8.19)
Download a legacy version of the Arduino IDE.

After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation. After installation is completed, an shortcut will be generated in the desktop.



Run it. The interface of the software is as follows:



Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

	Verify Checks your code for errors compiling it.
	Upload Compiles your code and uploads it to the configured board.
	Debug Troubleshoot code errors and monitor program running status.
	Serial Plotter Real-time plotting of serial port data charts.
	Serial Monitor Used for debugging and communication between devices and computers.

0.2 Upload Servo Code (Necessary)

Upload code in **Freenove_4WD_Car_Kit\Sketches\00.0_Servo_90** to Freenove control board of the car.

This code will be used in assembly step.

Do not skip this step. Please remove Bluetooth Module Module when you upload code.

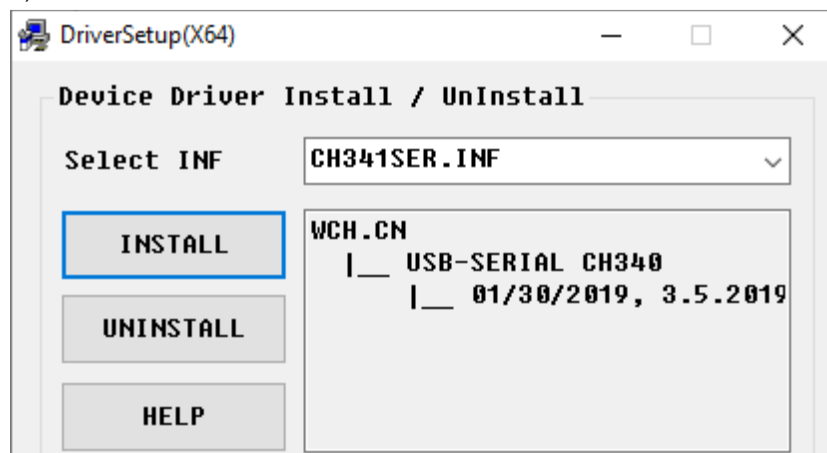
1 install Ch340 driver

1) Download the driver:

<http://www.wch-ic.com/downloads/file/65.html>

2) Connect the control board to your computer.

3) Double Click the exe file. Then click INSTALL.



For mac and linux, please download the driver from here:

<http://www.wch-ic.com/search?q=CH340&t=downloads>

Demo SDK.			
CH341SER_LINUX....	CH340/CH341 USB to serial port LINUX driver	1.5	2018-03-18
CH341SER_MAC.ZI...	CH340/CH341 USB to serial port MAC OS driver	1.5	2018-07-05

2 upload the code

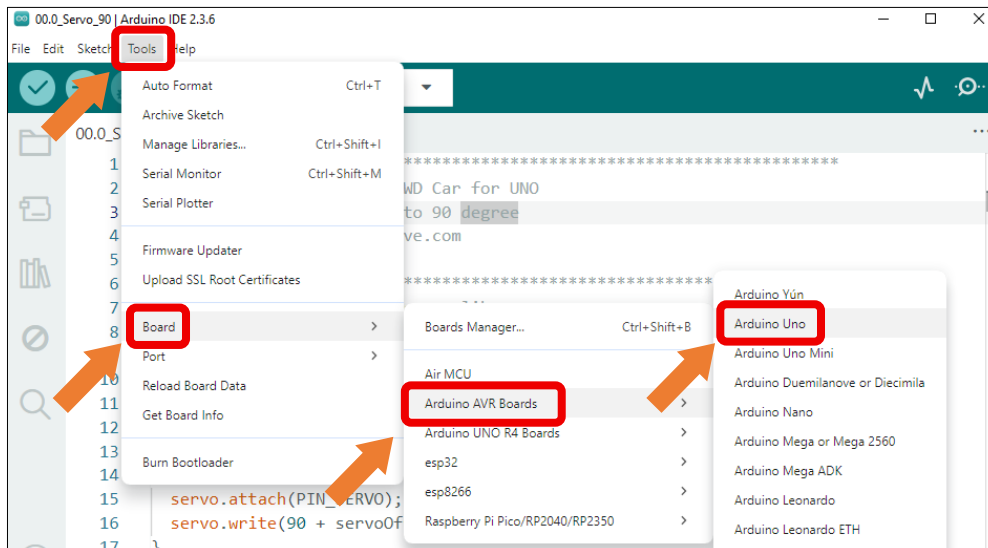
Open the example sketch "00.0_Servo_90" with Arduino IDE.



If the port doesn't work, please install the driver for the board.

Freenove_4WD_Car_Kit/Drivers

Select board "Arduino/Genuino Uno".



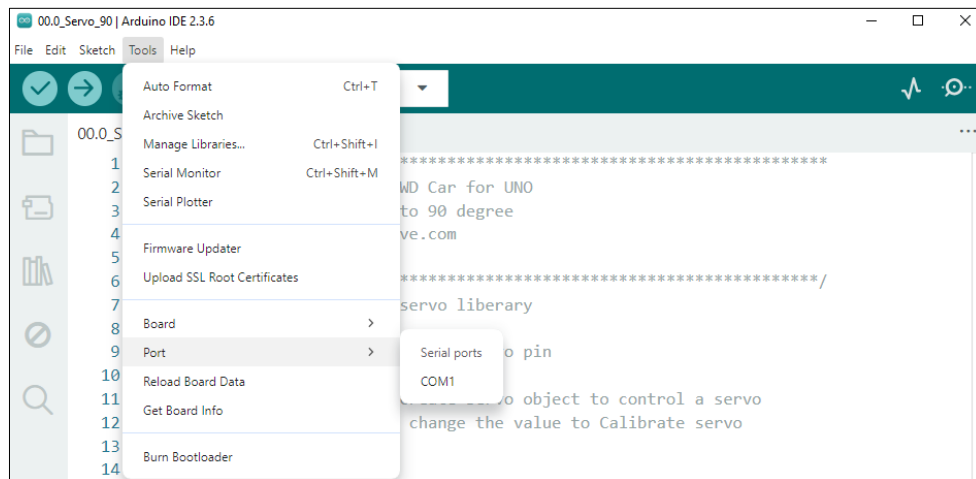
Check Port before connecting the control board to computer.

Note: Your port may be different from the following figure.

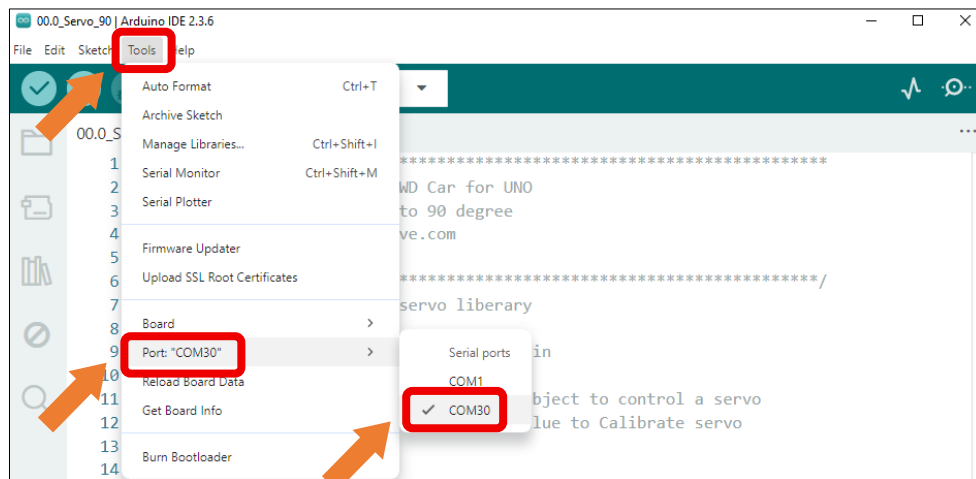
On **Windows**: It may be COM4, COM5 (Arduino Uno) or something like that.

On **Mac**: It may be /dev/cu.usbserial-710, /dev/cu.usbmodem7101 (Arduino Uno) or something like that.

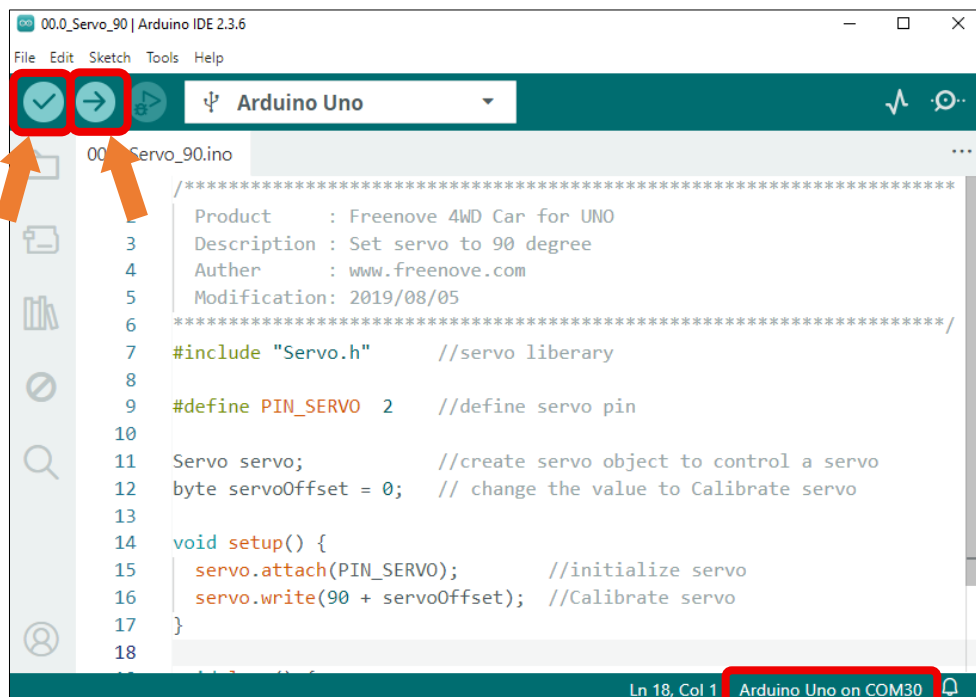
On **Linux**: It may be /dev/ttyUSB0, /dev/ttyACM0 or something like that.



Connect the control board to computer via USB cable. And then check Port again. You will find a COM added. That is the port of control board. Here it is COM19. In your computer, it would be a different number.



Click "Verify" button and "Upload" button.

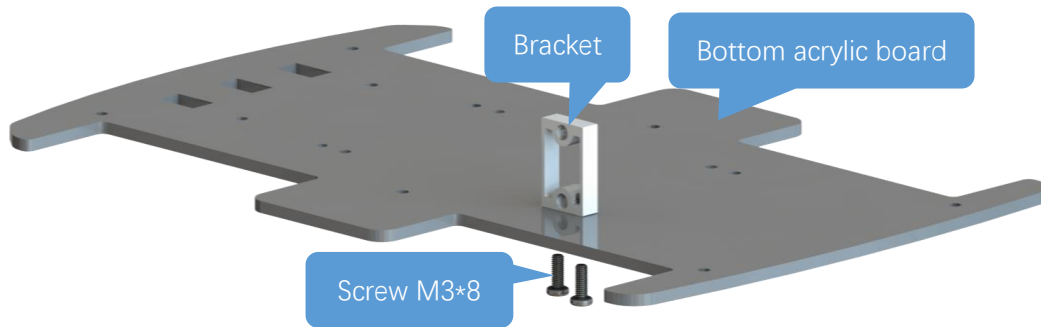


0.3 Assembly

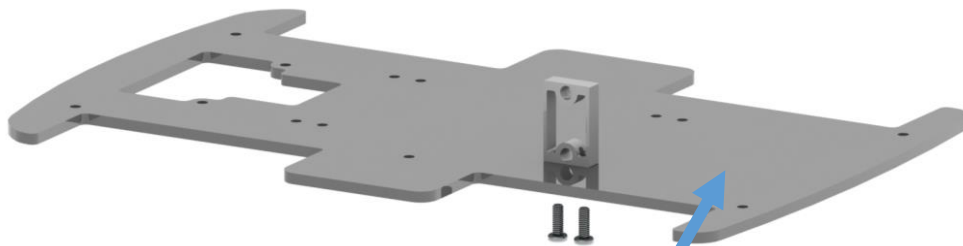
If you have any concerns, please feel free to contact us via support@freenove.com

In this chapter, we will learn how to assemble this car. This will spend some time. Please be patient.

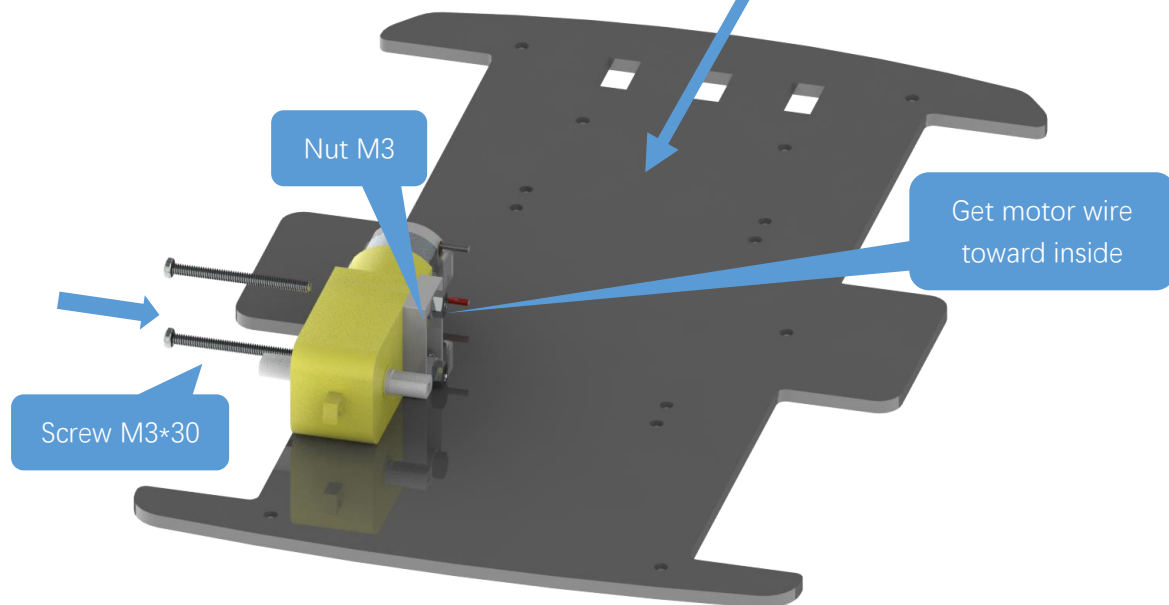
Install Motor Bracket on bottom acrylic board, with screw M3*8 in the same bag.



In your kit, the bottom acrylic board may look like below:

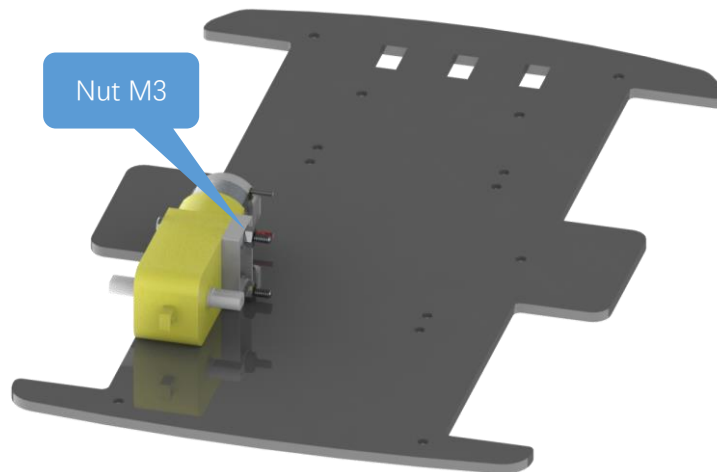


Install Motor Bracket on bottom acrylic board, with screw M3*30 and Nut M3 in the same bag.

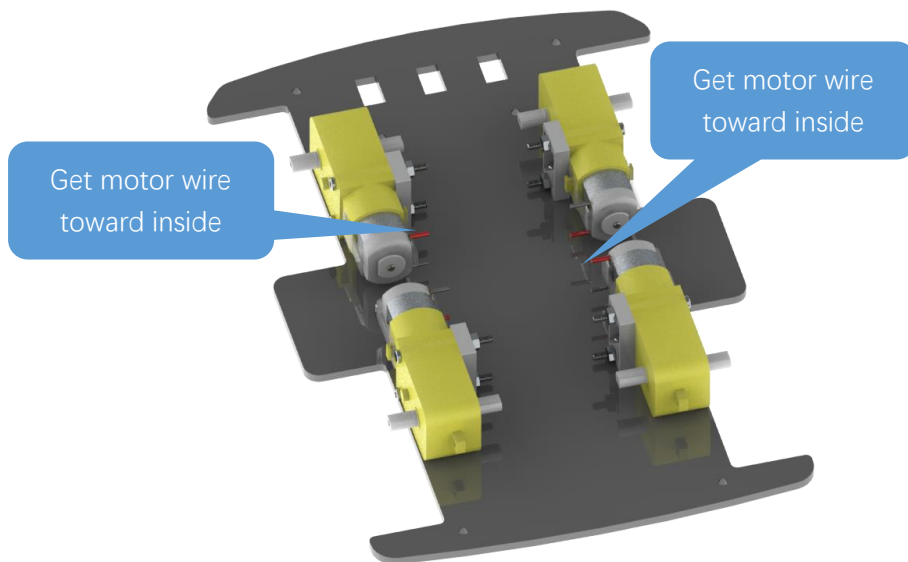


Caution: Do not remove the cable tie from the motor; otherwise, the motor cable may become detached.

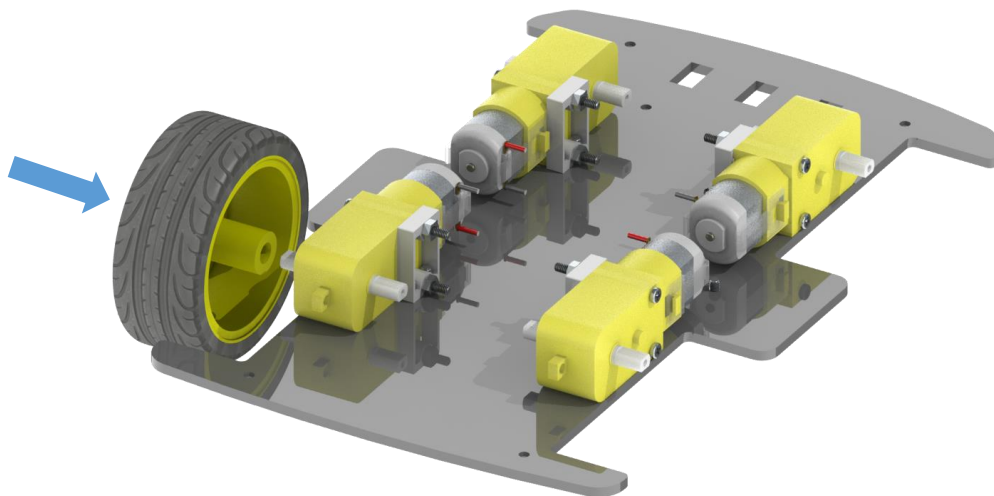
And then one motor will be installed successfully.



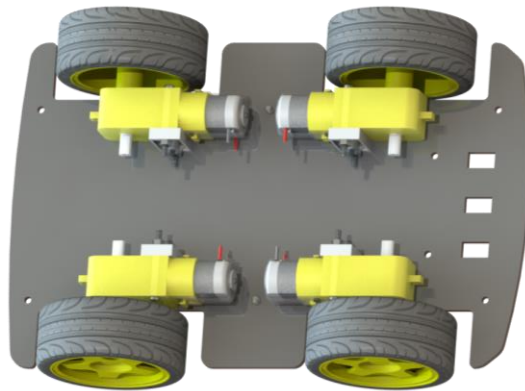
Install the rest 3 sets of motors with the same method.



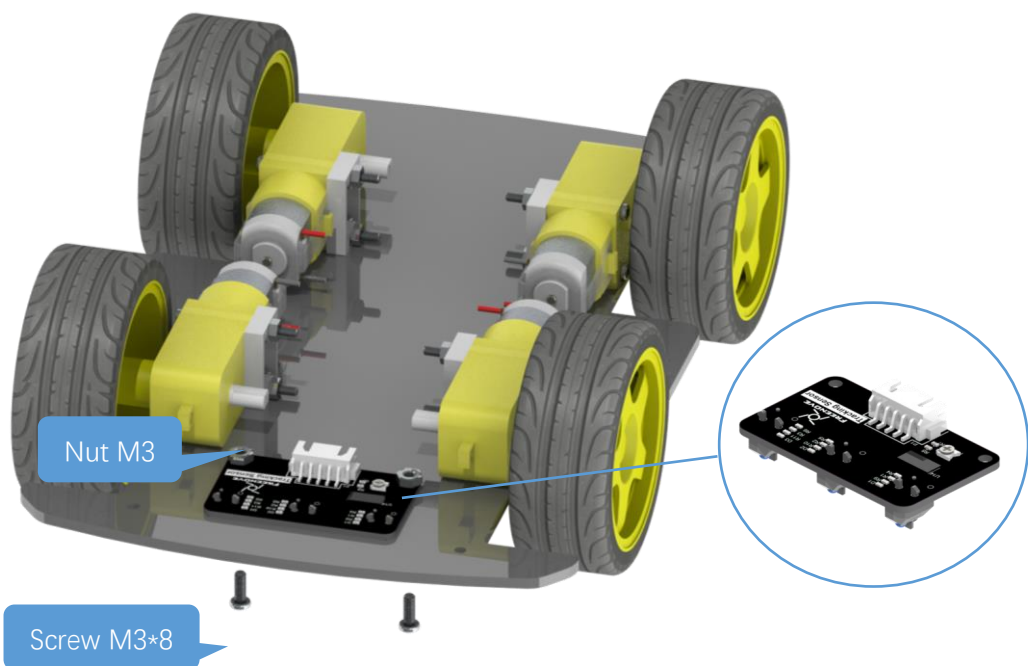
Install wheel. Note, the hole is not round.



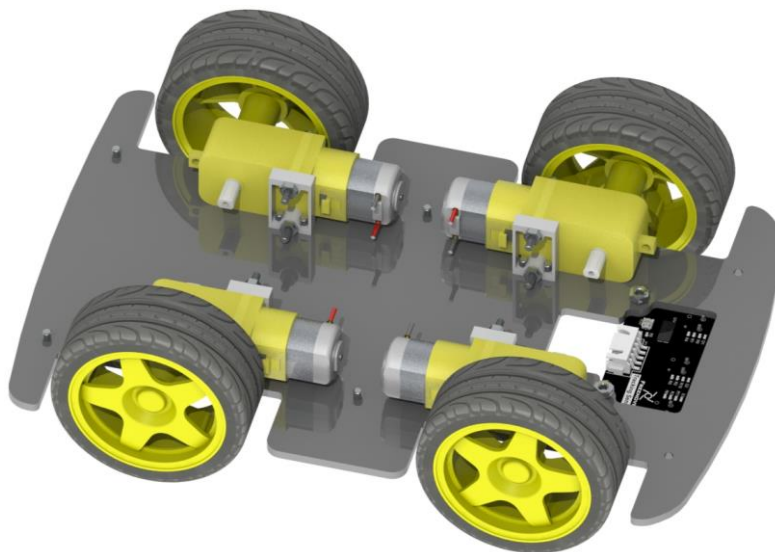
Install the rest 3 wheels.



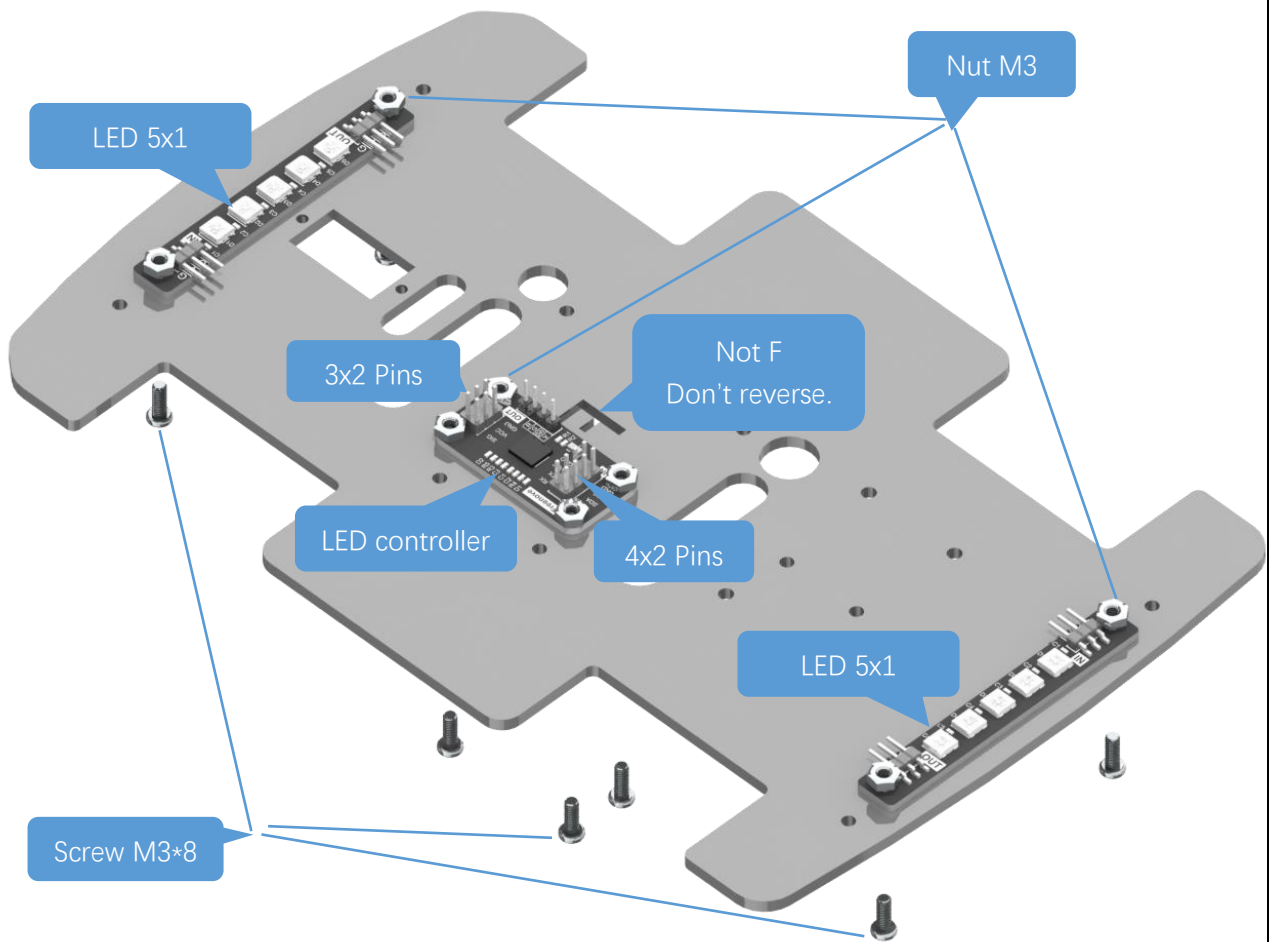
Install line tracking module with Screw M3*8 and Nut M3.



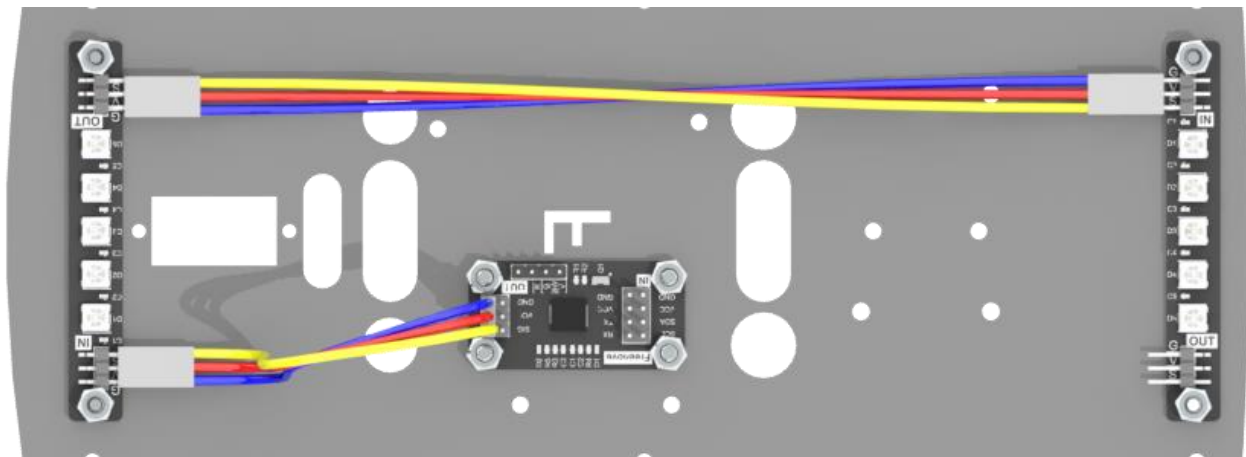
If you have other kind of bottom acrylic board, please install the tracking module like below.



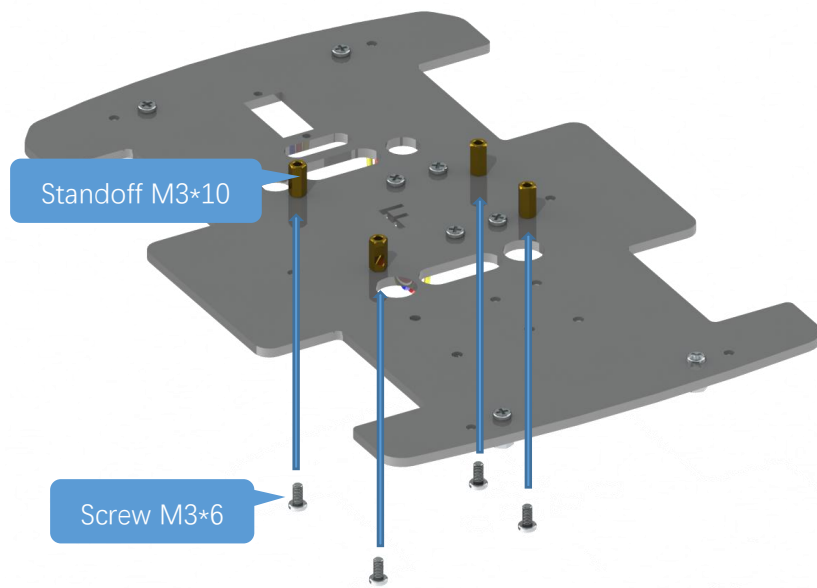
Install LED 5x1 and LED controller to top acrylic board.



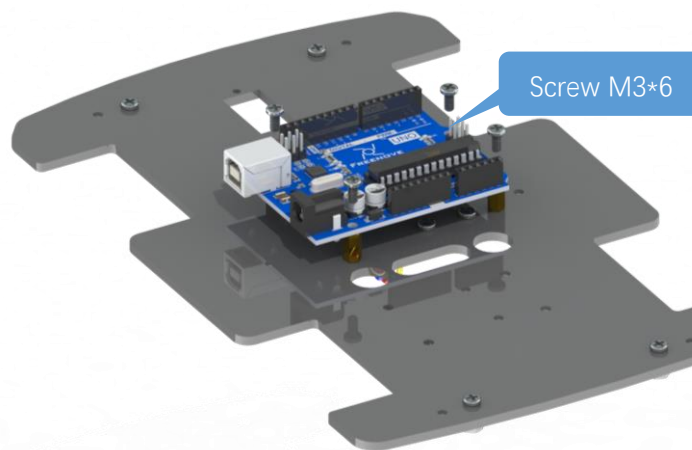
Connect wiring of LED controller and LED 5X1. The wire need to be twisted 180°.



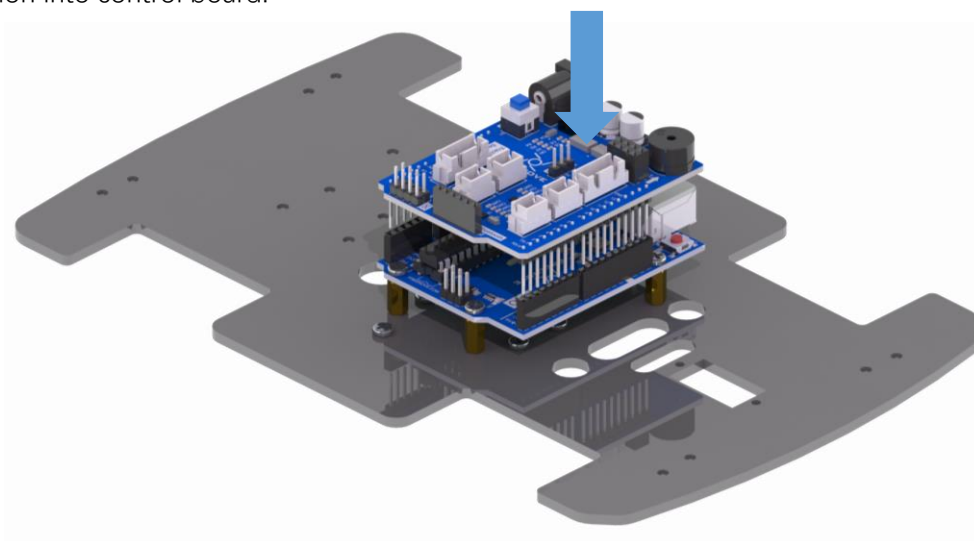
Install standoff on top acrylic board.



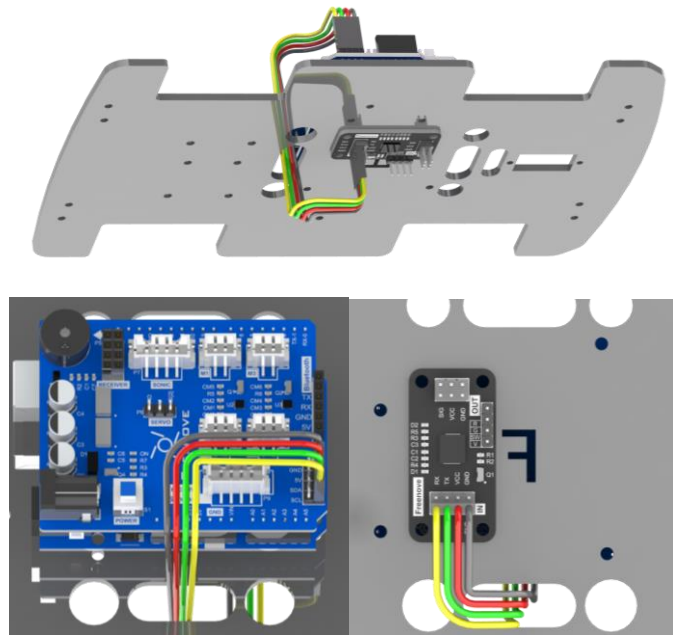
Install Freenove control board.



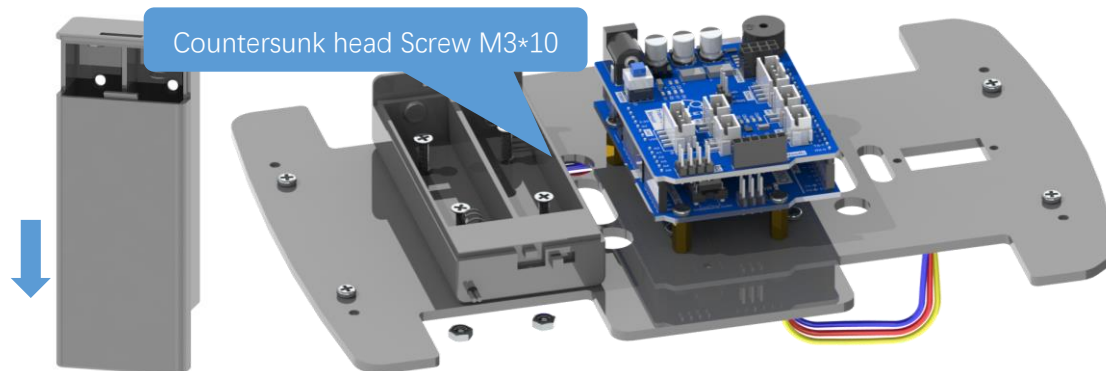
Plug extension into control board.



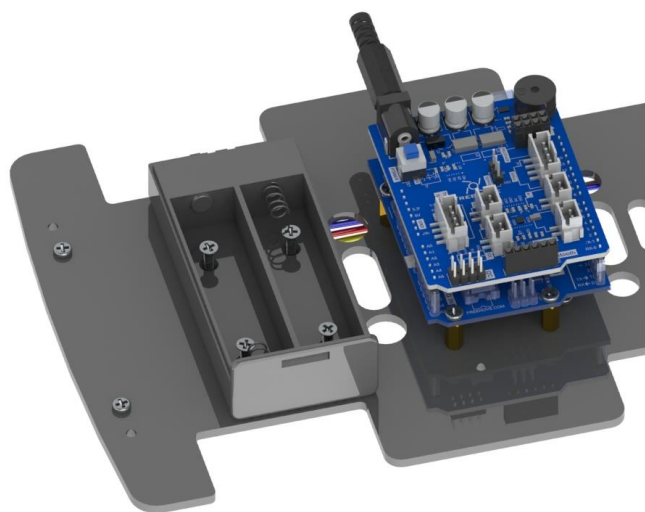
Connect LED controller with extension board with jumper wire F-F 4P. Wires of LED are hidden. GND-GND, 5V-VCC, SCA-SCL, SDA-SDA.



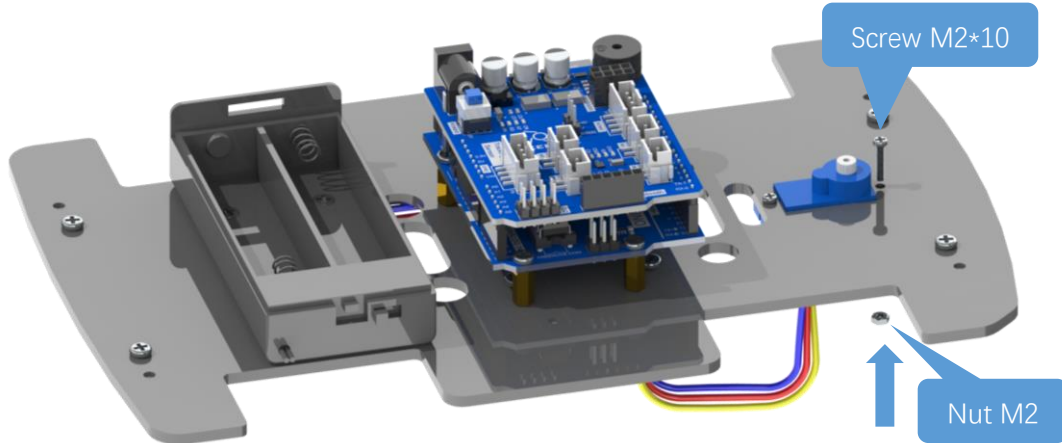
Install battery holder.



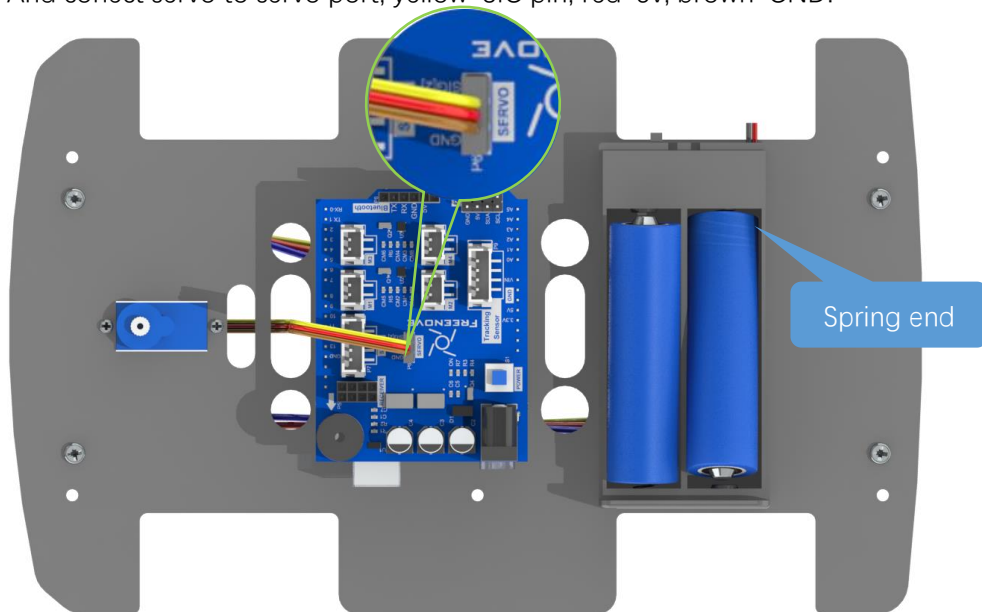
Some battery holders have short cable. Please rotate the battery holder for 180° to install.



Install servo.

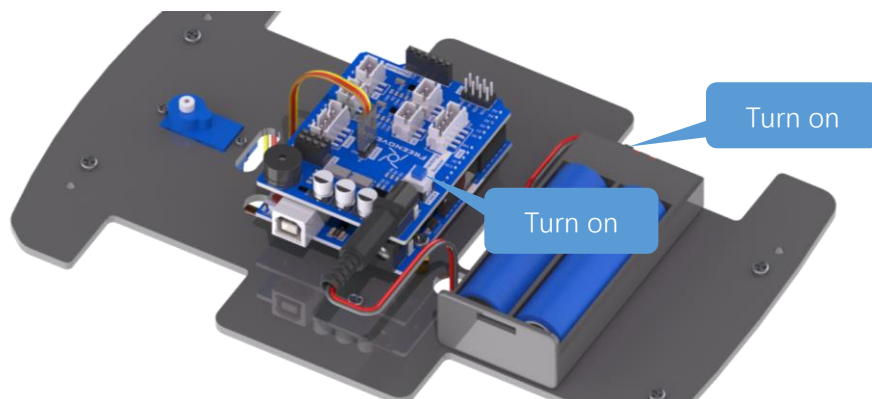


Install battery. And connect servo to servo port, yellow-SIG pin, red-5v, brown-GND.



18650 3.7V **rechargeable** lithium battery x2 It is easier to find proper battery on **eBay** than Amazon.

Connect power supply to extension board (the top board, not control board board).

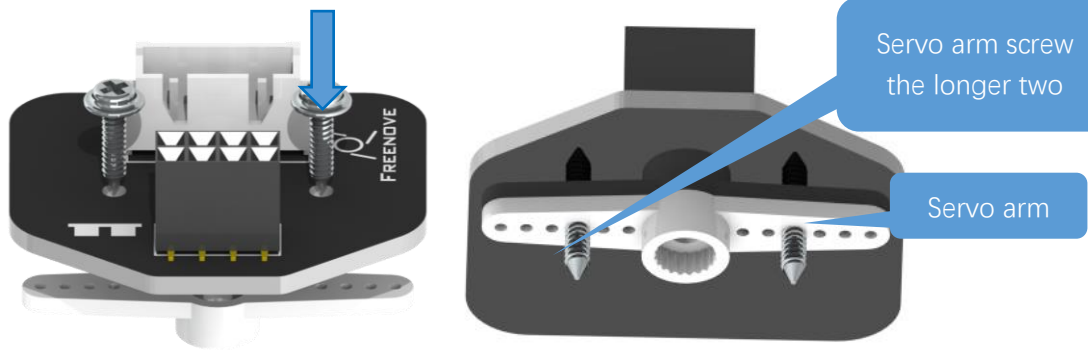


If you have uploaded code before, just turn on the switch of extension board and battery holder.

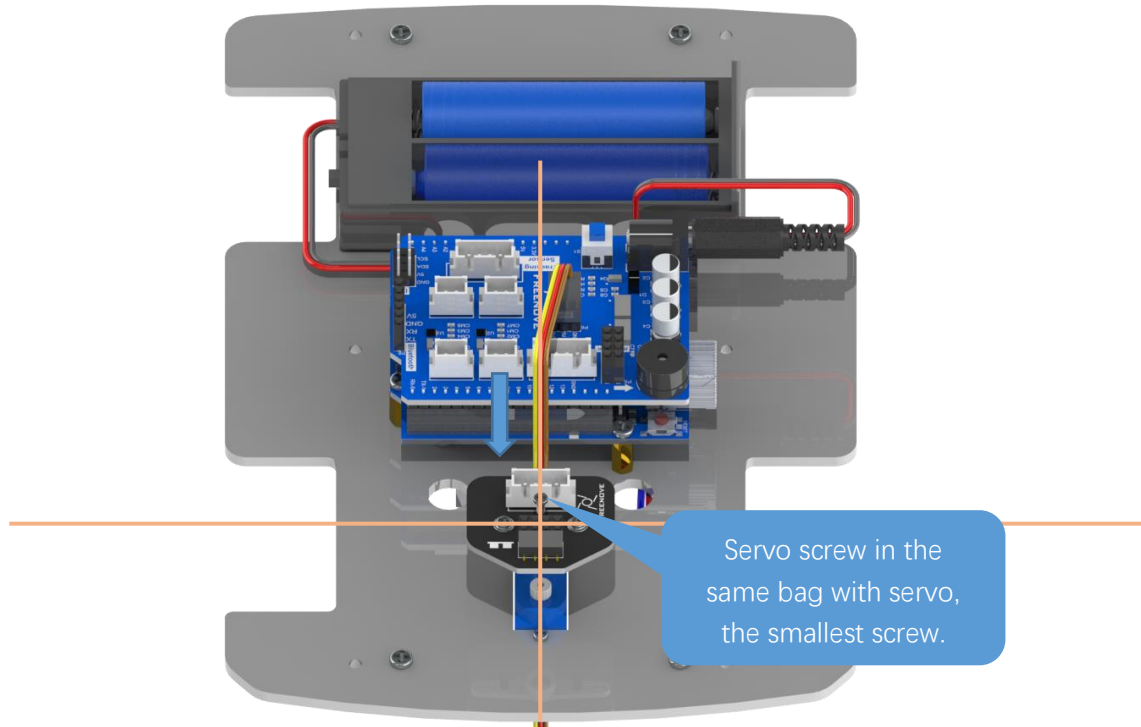
If not, you need [upload the code](#).

Some battery holders have short cable. Please rotate the battery holder for 180° to install.

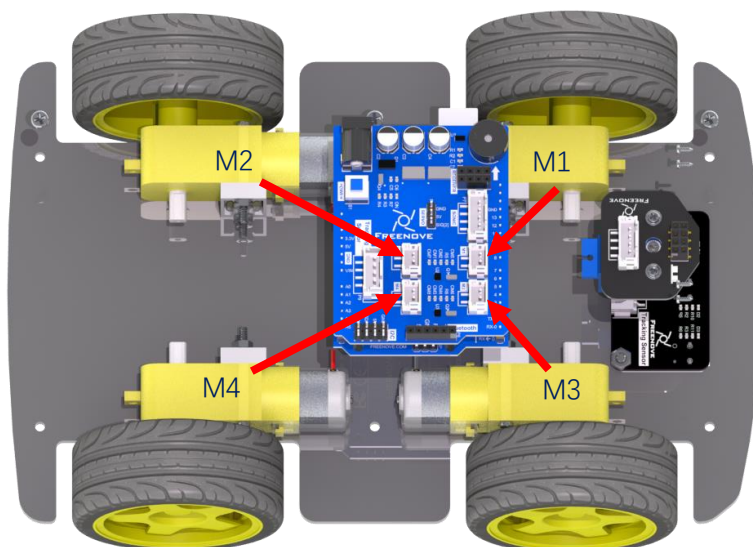
Install servo arm with sonic module connector. The screw and arm are in the same bag with servo.



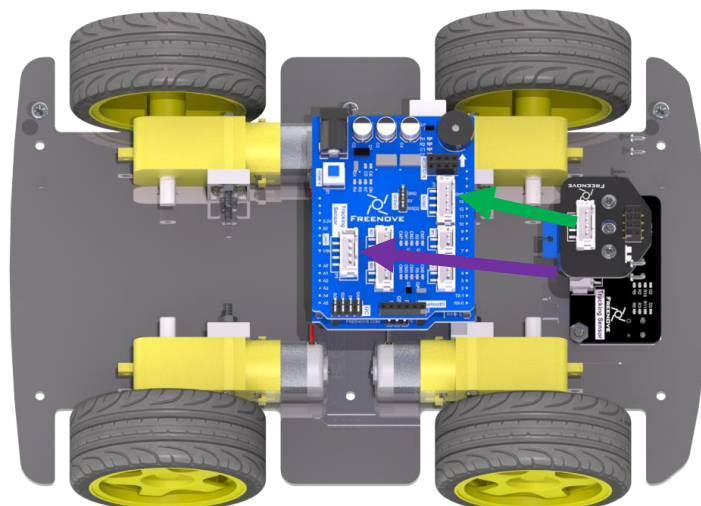
Connect servo arm to servo. Make sure servo arm is installed at 90 degrees.



Connect the motor wire to the interface of the board. As below.



Connect the ultrasonic and tracking modules to the board. As below.

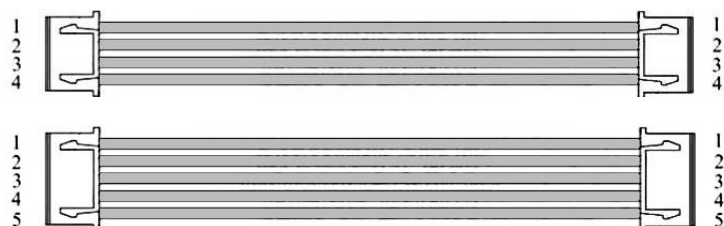


Note: The cables of the ultrasonic module and the tracking module are as shown below.

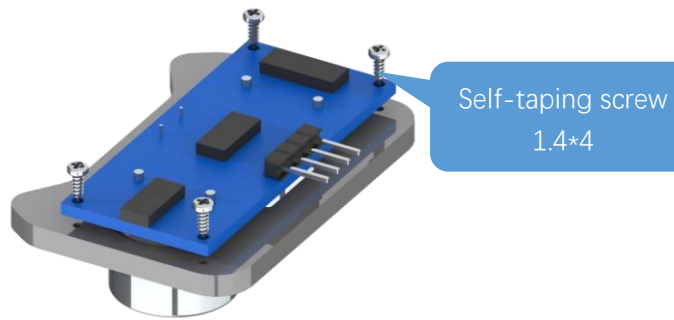


The following shows the cable with wrong sequence. If you receive such cable, please contact us:

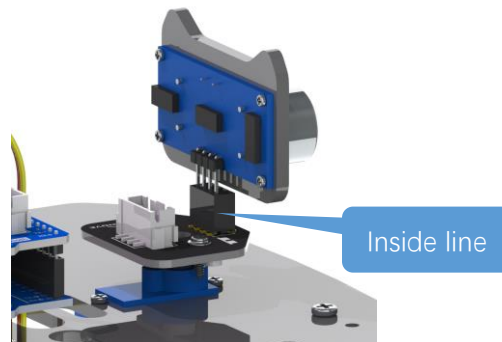
✉ support@freenove.com



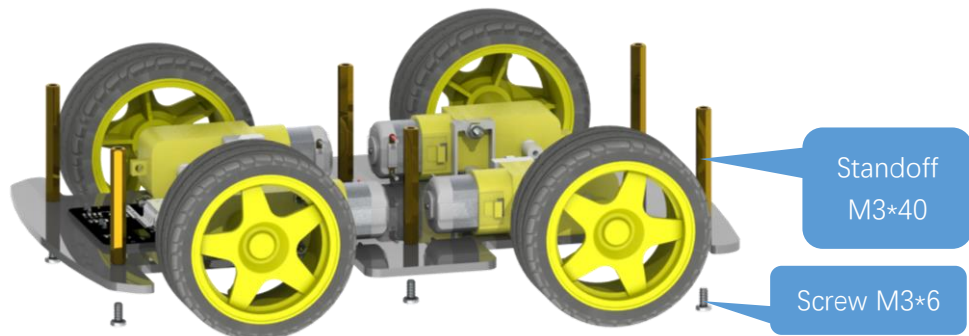
Install ultrasonic module to acrylic board.



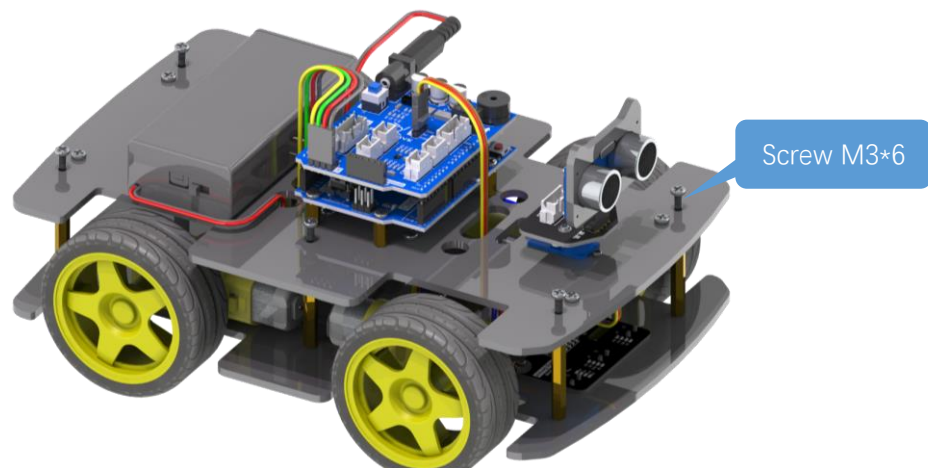
Plug ultrasonic module.



Install standoff M3*40 on bottom acrylic board.

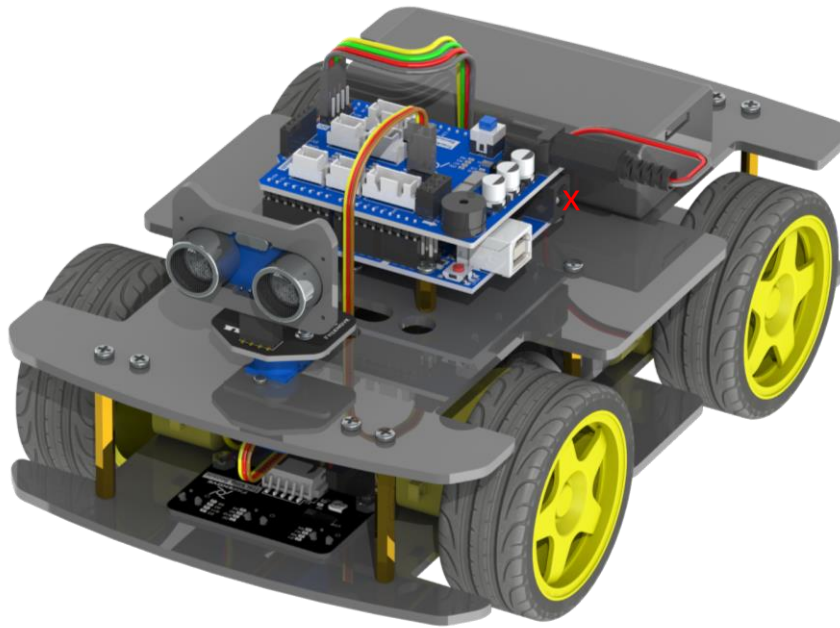


Install top acrylic board to standoff.



Now the whole assembly is completed.

Connect the power to upper port.

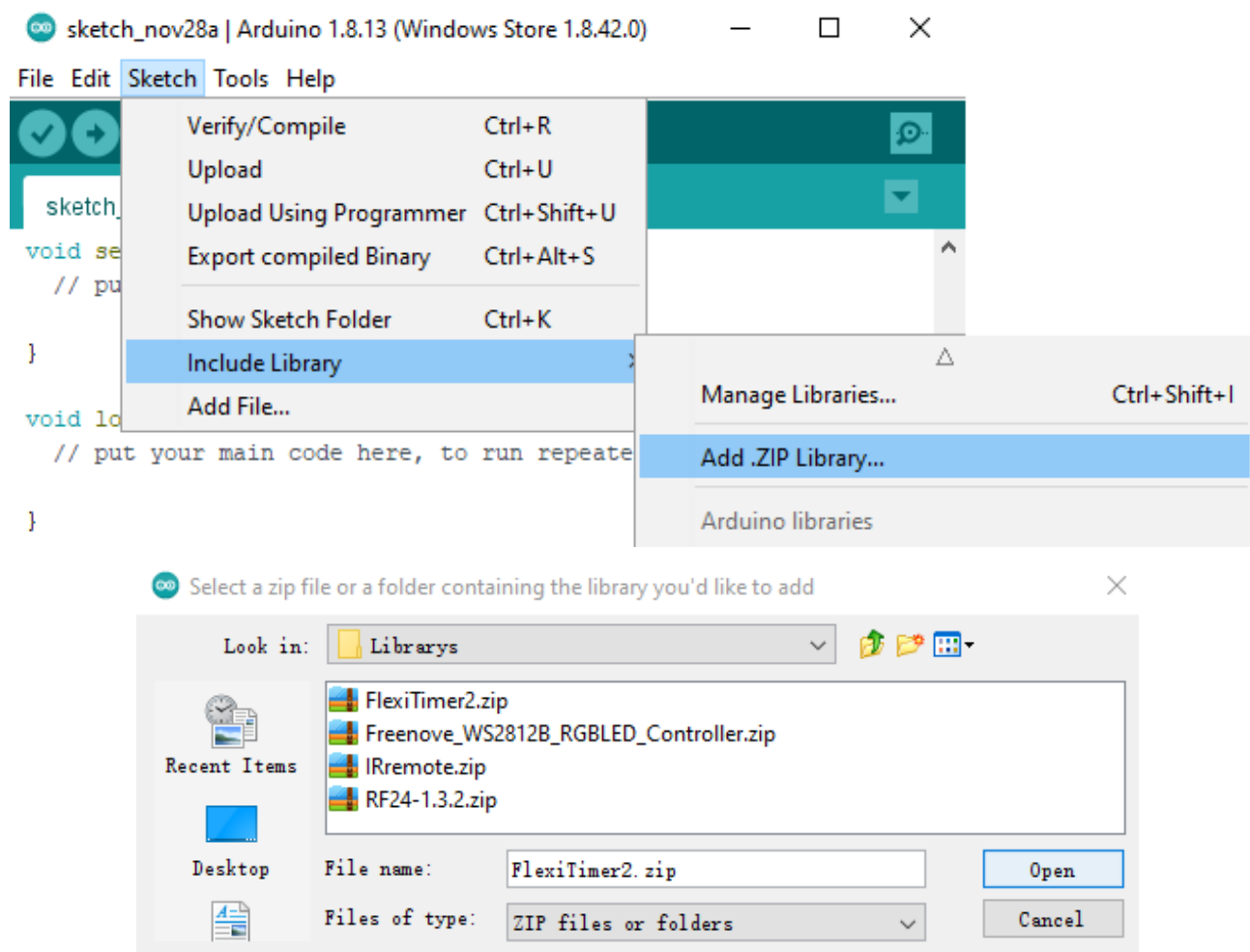


0.4 How to Play

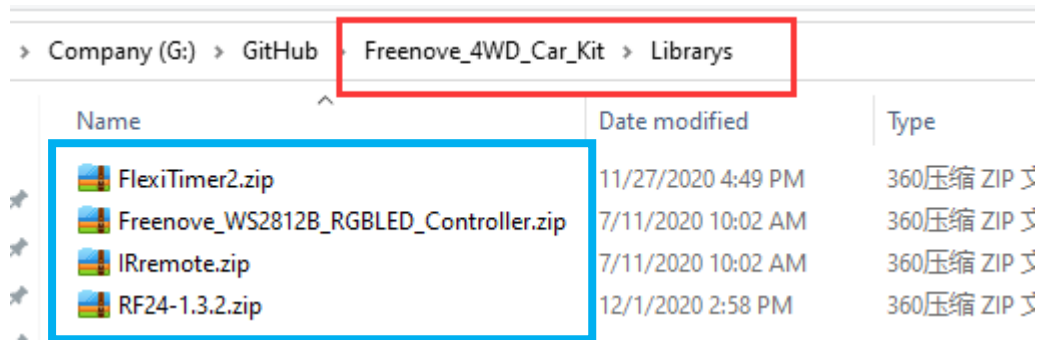
You can follow this video or the later steps to play this car:

<https://youtu.be/HxXmkmQxEjY>

Step 1 Add libraries

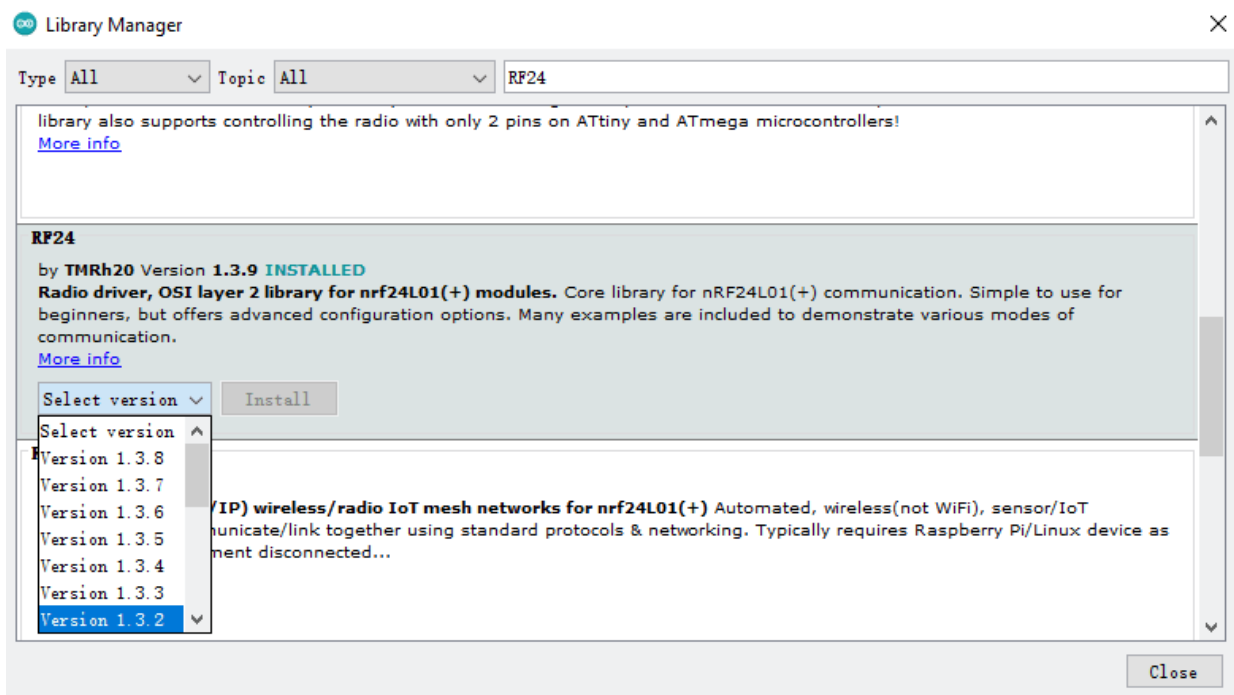


Add **all the 4 libraries** in folder **Freenove_4WD_Car_Kit\Libraris** one by one.



Name	Date modified	Type
 FlexiTimer2.zip	11/27/2020 4:49 PM	360压缩 ZIP 彡
 Freenove_WS2812B_RGBLED_Controller.zip	7/11/2020 10:02 AM	360压缩 ZIP 彡
 IRremote.zip	7/11/2020 10:02 AM	360压缩 ZIP 彡
 RF24-1.3.2.zip	12/1/2020 2:58 PM	360压缩 ZIP 彡

If you have installed RF24 libraries, you need select to install Version 1.3.2

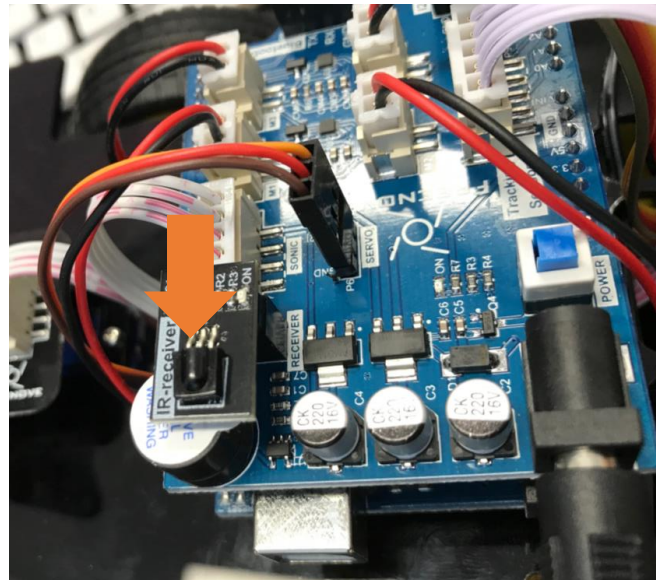
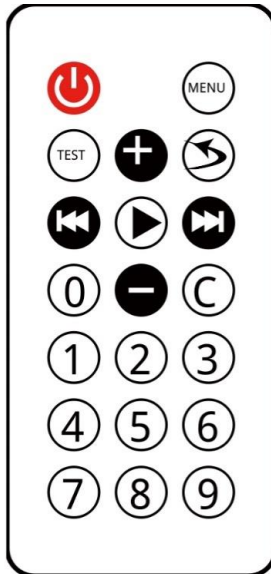


Step 2 Upload Code

If you want to use different control ways, you need to upload different codes.

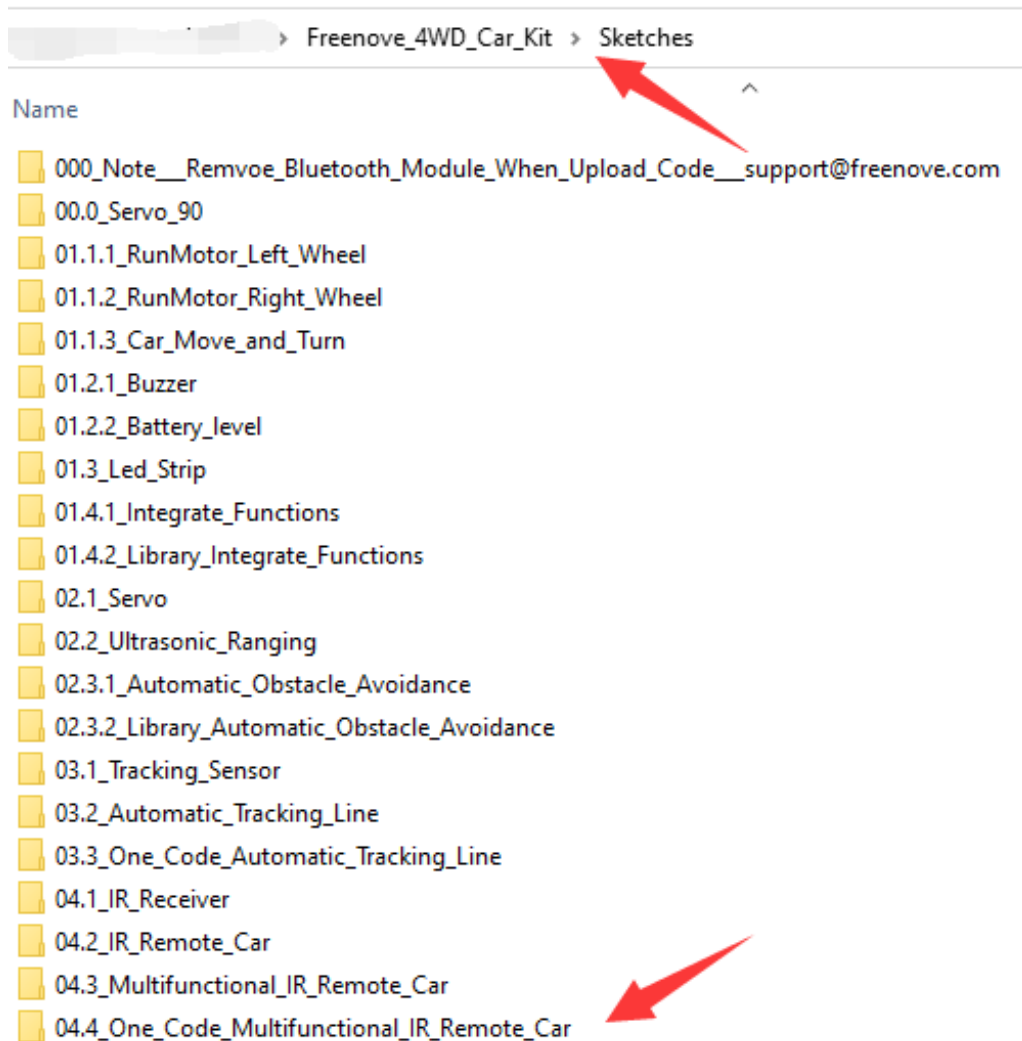
IR remote control

1) Install IR receiver to the car. And remove the plastic paper from IR remote.



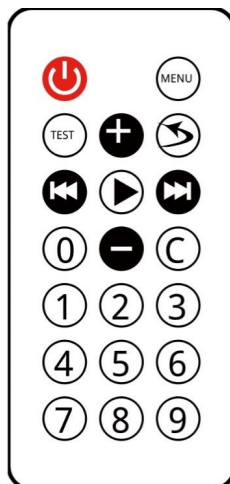
2) Upload following code to the car board. **Please remove Bluetooth Module when you upload code.**

Freenove_4WD_Car_Kit\Sketches\04.4_One_Code_Multifunctional_IR_Remote_Car.ino



After uploading successfully, you can use the IR remote control to control the car.

After the code is successfully uploaded, turn on the power of the car and use the infrared remote control to control the car and other functions. The corresponding keys and their functions are shown in the following table:



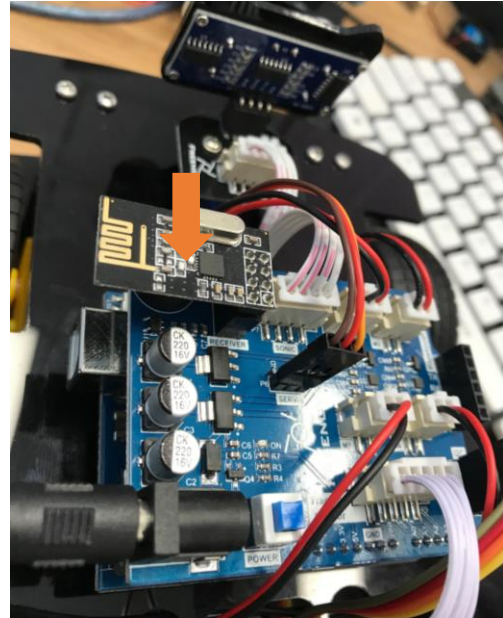
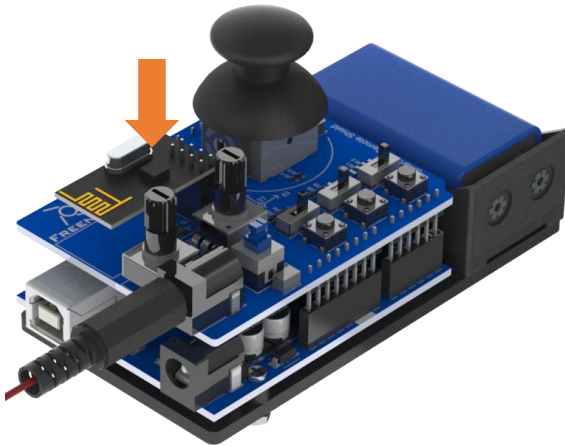
Key graph	Key define	Key code	Function
	IR_REMOTE_KEYCODE_UP	0xFF02FD	move forward
	IR_REMOTE_KEYCODE_DOWN	0xFF9867	move back
	IR_REMOTE_KEYCODE_LEFT	0xFFE01F	Turn left
	IR_REMOTE_KEYCODE_RIGHT	0xFF906F	Turn right
	IR_REMOTE_KEYCODE_CENTER	0xFFA857	Turn on buzzer
	IR_REMOTE_KEYCODE_1	0xFF30CF	Make the LED run mode 1 to scroll the rainbow color.
	IR_REMOTE_KEYCODE_4	0xFF10EF	Make LED run mode 2, changing the color of the water LED
	IR_REMOTE_KEYCODE_2	0xFF18E7	The color of the LED bar changes faster. The color is from ColorWheel.
	IR_REMOTE_KEYCODE_3	0xFF7A85	The color of the LED bar changes slower.
	IR_REMOTE_KEYCODE_5	0xFF38C7	The LED bar cycle period is decreased, and the LED bar changes at a faster speed
	IR_REMOTE_KEYCODE_6	0xFF5AA5	The LED bar cycle period is increased, and the LED bar changes at a slower speed

RF remote control

Download tutorial and code to **assemble** remote controller.

https://github.com/Freenove/Freenove_Remote_Control_Kit/archive/refs/heads/master.zip

Remove IR reviver and install RF module.



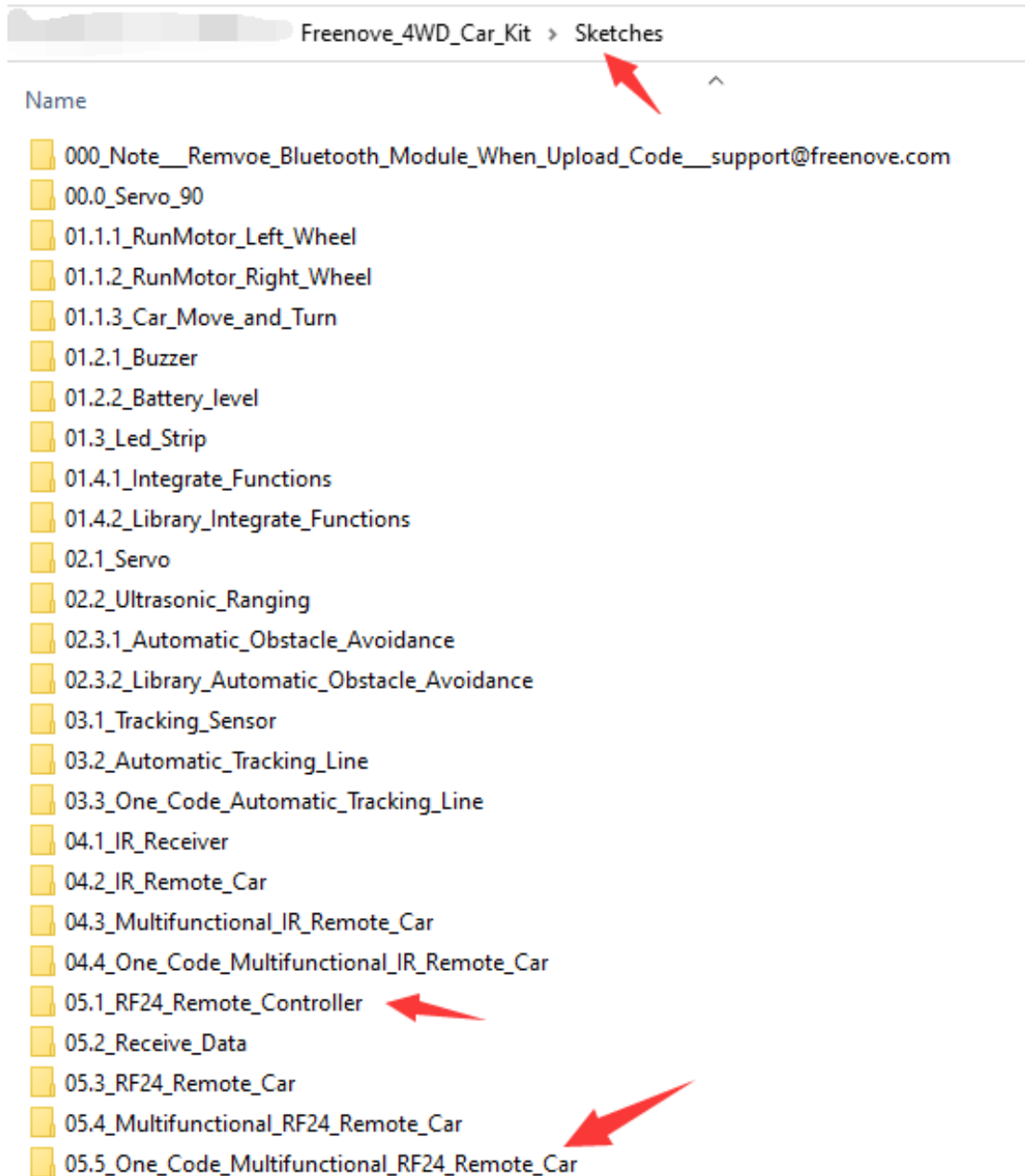
You need remove Bluetooth Module when you upload code.

Upload following code to the car board.

Freenove_4WD_Car_Kit\Sketches\05.5_One_Code_Multifunctional_RF24_Remote_Car

Upload following code to the RF remote control.

Freenove_4WD_Car_Kit\Sketches\05.1_RF24_Remote_Controller



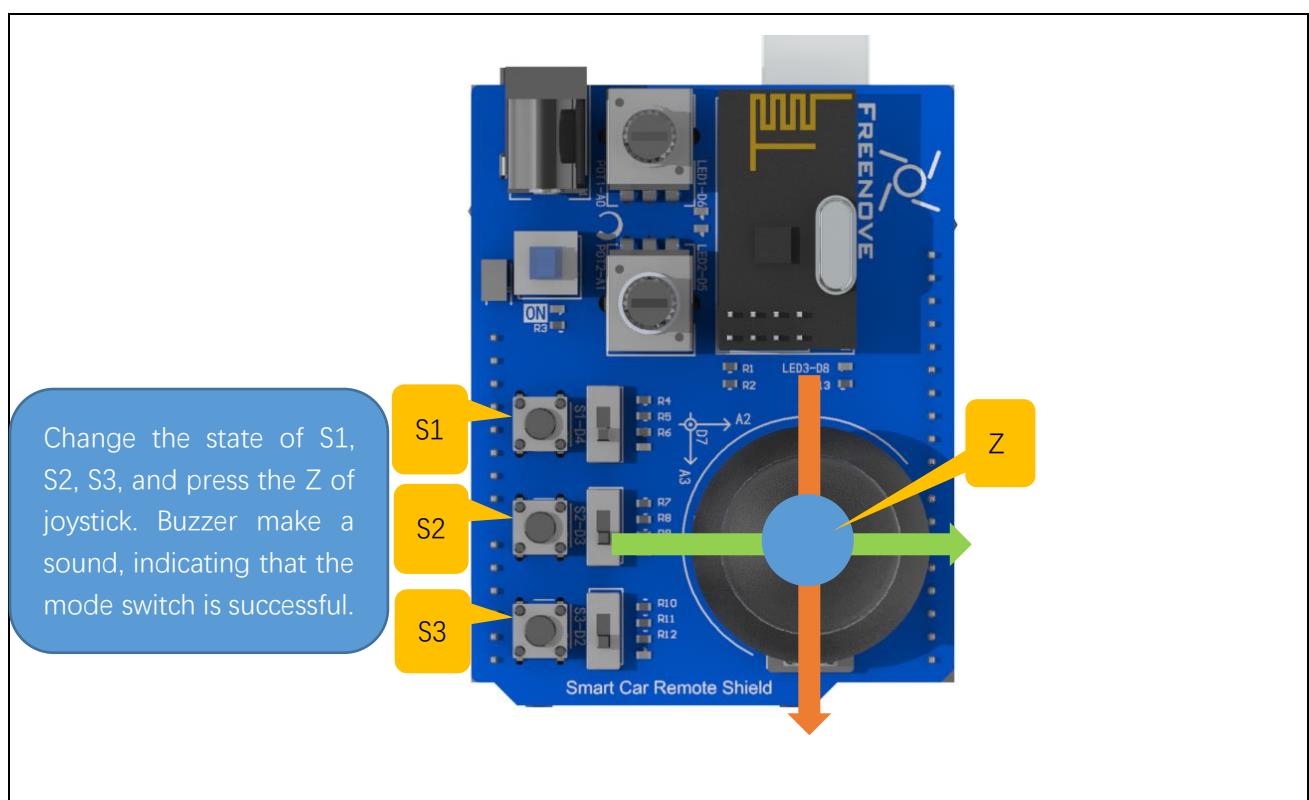
After uploading successfully, you can use the RF remote control to control the car.

Switch different modes

- 1, Change the switch state of S1, S2, and S3, and the car will stop moving.
- 2, Press the Z axis of the joystick and the buzzer B sounds to indicate that the mode is successfully switched.

The following table shows the modes indicated by the different states of the three switches S1, S2, and S3. The LED next to the switch illuminates to indicate ON and OFF state of switches. The three switches can form $2 \times 2 \times 2 = 8$ modes.

S1	S2	S3	Mode No.	Mode
ON	ON	ON	0	None
ON	ON	OFF	1	Calibrate servo mode
ON	OFF	ON	2	None
ON	OFF	OFF	3	Obstacle avoidance mode
OFF	ON	ON	4	None
OFF	ON	OFF	5	Line tracking mode
OFF	OFF	ON	6	Switch LED mode
OFF	OFF	OFF	7	Manuel control mode / Default mode



Mode 0, 2, 4

Reserved. We did not assign functions for them.

Mode 1-Calibrate servo

If your servo is not accurately mounted at 90 degrees, you can use this mode for fine adjustment (+ -10 degrees).

In this mode, you can adjust potentiometer 2 (POT2) to fine tune the angle of the servo. When you adjust the servo to the correct angle, press the Z-axis of joystick to save calibration data to EEPROM. It will be saved permanently unless it is modified.

Mode 3-obstacle avoidance, Mode 5-line tracking mode

These two modes have been learned separately in the previous project, and their running logic and codes are

consistent with the previous project.

The difference is that in this project, the car can respond to commands from the remote control at any time. Therefore, in this project, it is still necessary to communicate with the remote controller in these two modes. When the remote control signal is disconnected, the car will stop. Therefore, the normal communication between the remote control and the car should be maintained at any time. Poor communication conditions may cause these two modes to work abnormally.

Mode 6-switch LED display mode

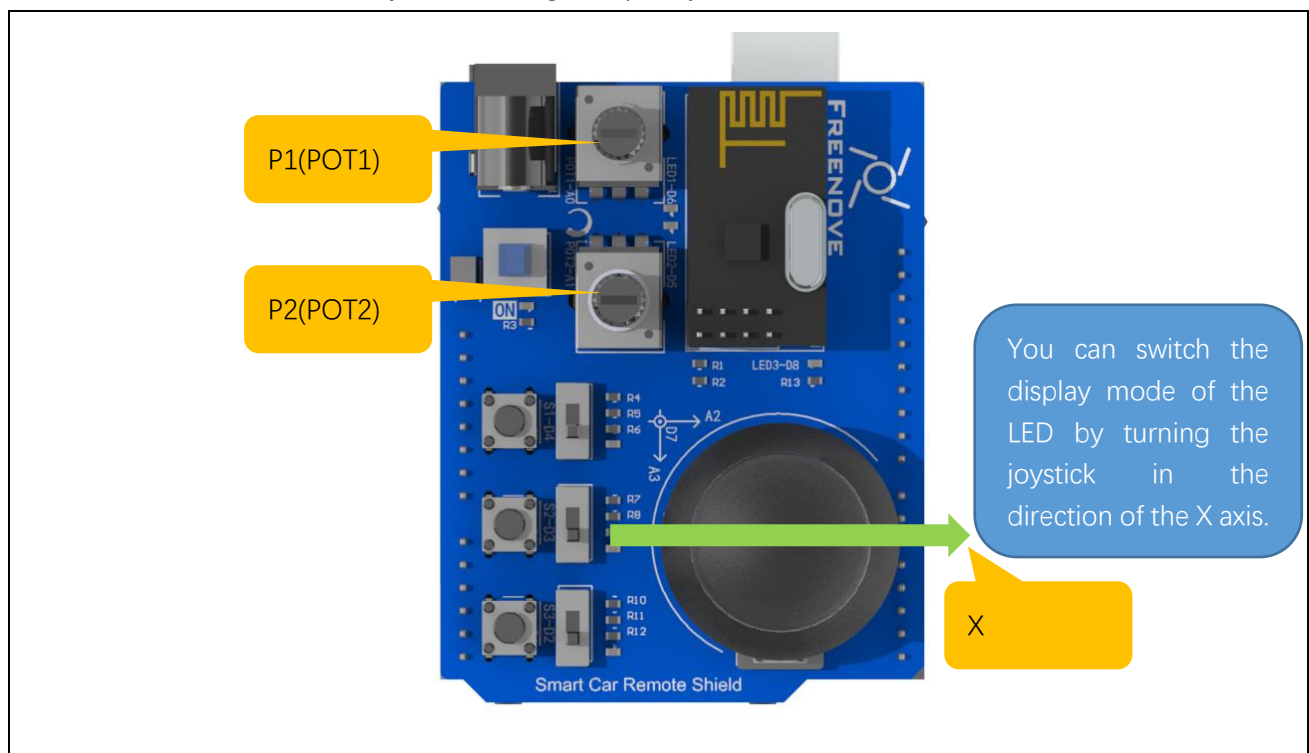
There are three display modes for the LEDs on the car, which are 0-flowing rainbow, 1-flowing water led, 2-Blink. In this mode, the display mode of the LED can be switched.

After entering this mode,

Move the joystick along the positive direction of its X-axis to switch the LED to the next mode.

Move the joystick in the negative direction of its X-axis to switch the LED to the previous mode.

In any mode, the LEDs can be adjusted with potentiometers P1 and P2. P1 is used to adjust the color change of the LED, and P2 is used to adjust the change frequency of LED.



Mode 7- manual remote mode

This mode is manual remote mode and is the default mode. This mode is consistent with the previous project "RF_Remote_Car". Use the joystick to control to move forward, move back and turn left, turn right.

App control

Download and install app. You can use app to control the robot.

You can download Freenove App through below ways:

View or download on Google Play:

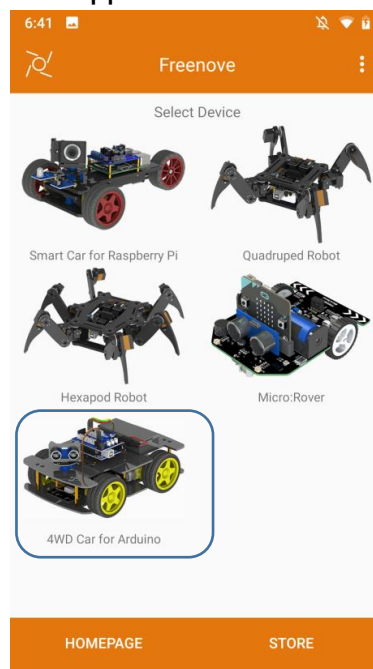
<https://play.google.com/store/apps/details?id=com.freenove.suhayl.Freenove>

Or download APK file directly

https://github.com/Freenove/Freenove_App_for_Android/raw/master/freenove.apk

Then install it on your Android phone.

For **iPhone app**, please search **freenove** in **App Store**.



Connect car to computer with USB cable.

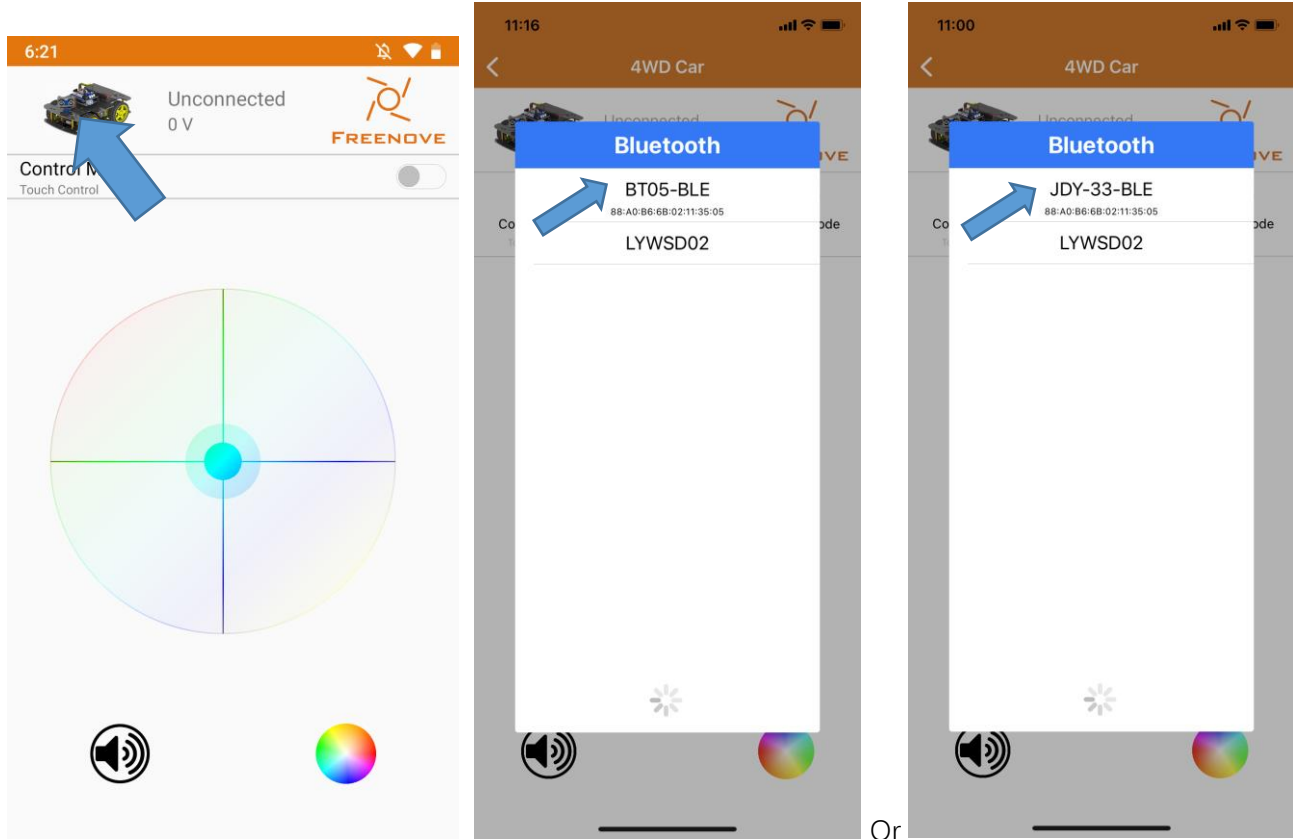
1 You need remove Bluetooth Module first when you upload code.

2 Then upload the code in **Sketches/06.4_One_Code_Multifunctional_Bluetooth_Remote_Car**.

3 After upload, reconnect the Bluetooth and reset the uno board.

Click following icon.

Click BT05 or JDY-33-BLE (Default name).



The following chapters will teach you how to control components.

Chapter 1 Control Basic Components

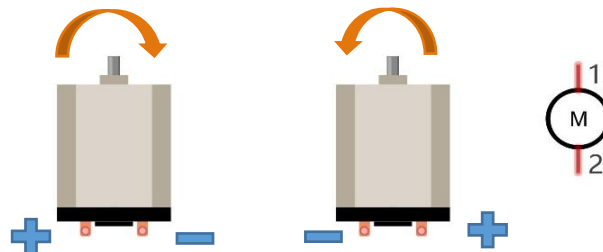
This chapter will introduce how to control motor, buzzer, RGB LED and get battery level.

If you have any concerns, please feel free to contact us via support@freenove.com

1.1 Motor

When motor is connected to the power supply, it will rotate in one direction. Reverse the polarity of power supply, the motor will rotate in the opposite direction.

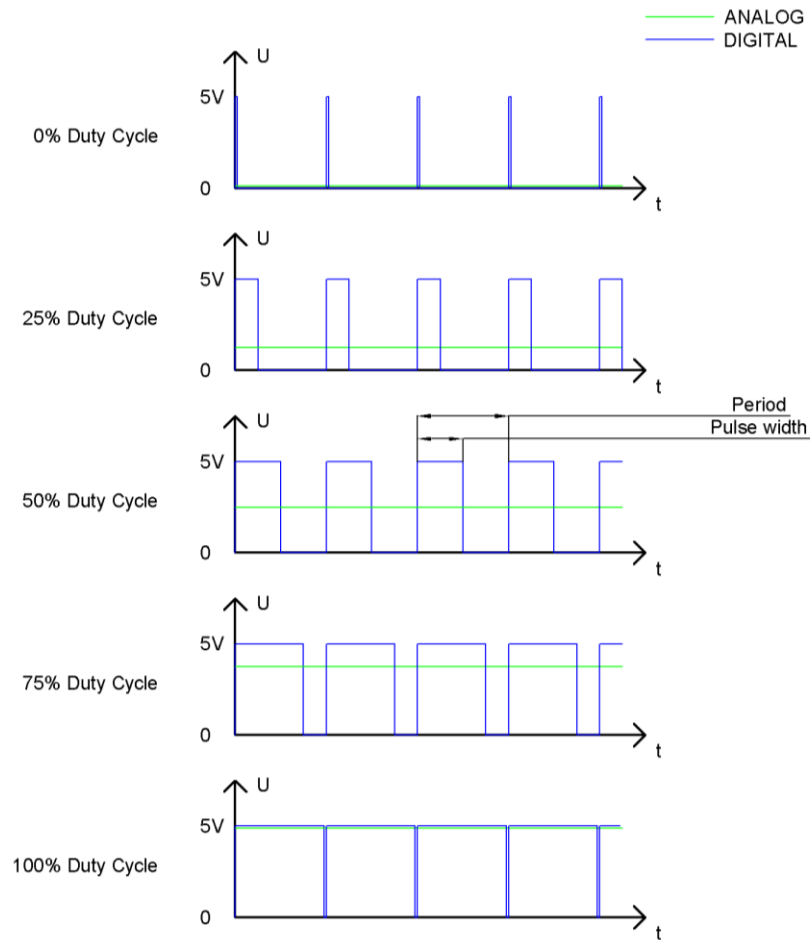
And the speed of motor depends on the voltage between two ends. The larger the voltage, the larger the speed.



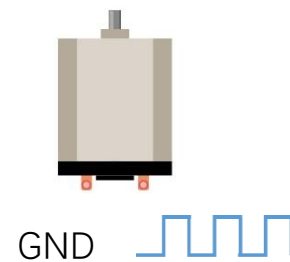
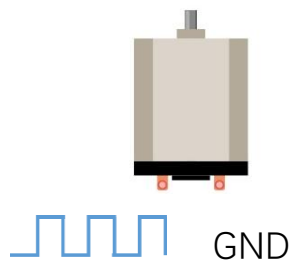
PWM

PWM, Pulse Width Modulation, uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (the reciprocal of the period is frequency). The time of high level outputs are generally called "pulse width", and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform.

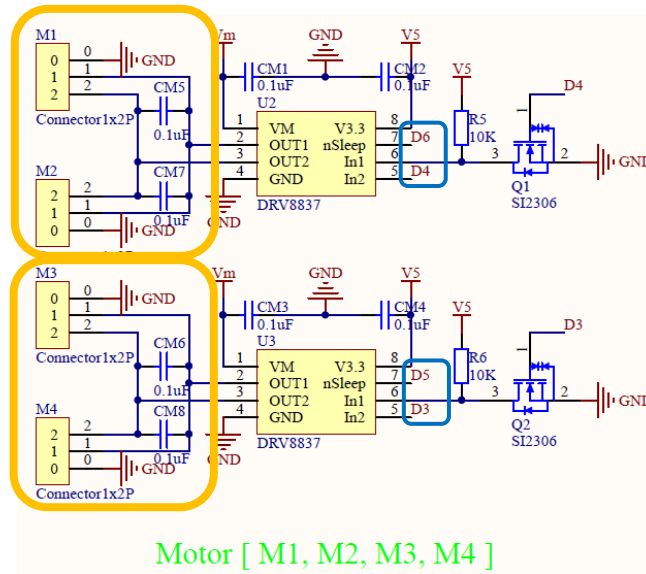
The longer the output of high levels last, the larger the duty cycle and the higher the corresponding voltage in analog signal will be. The following figures show how the analogs signal voltage vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



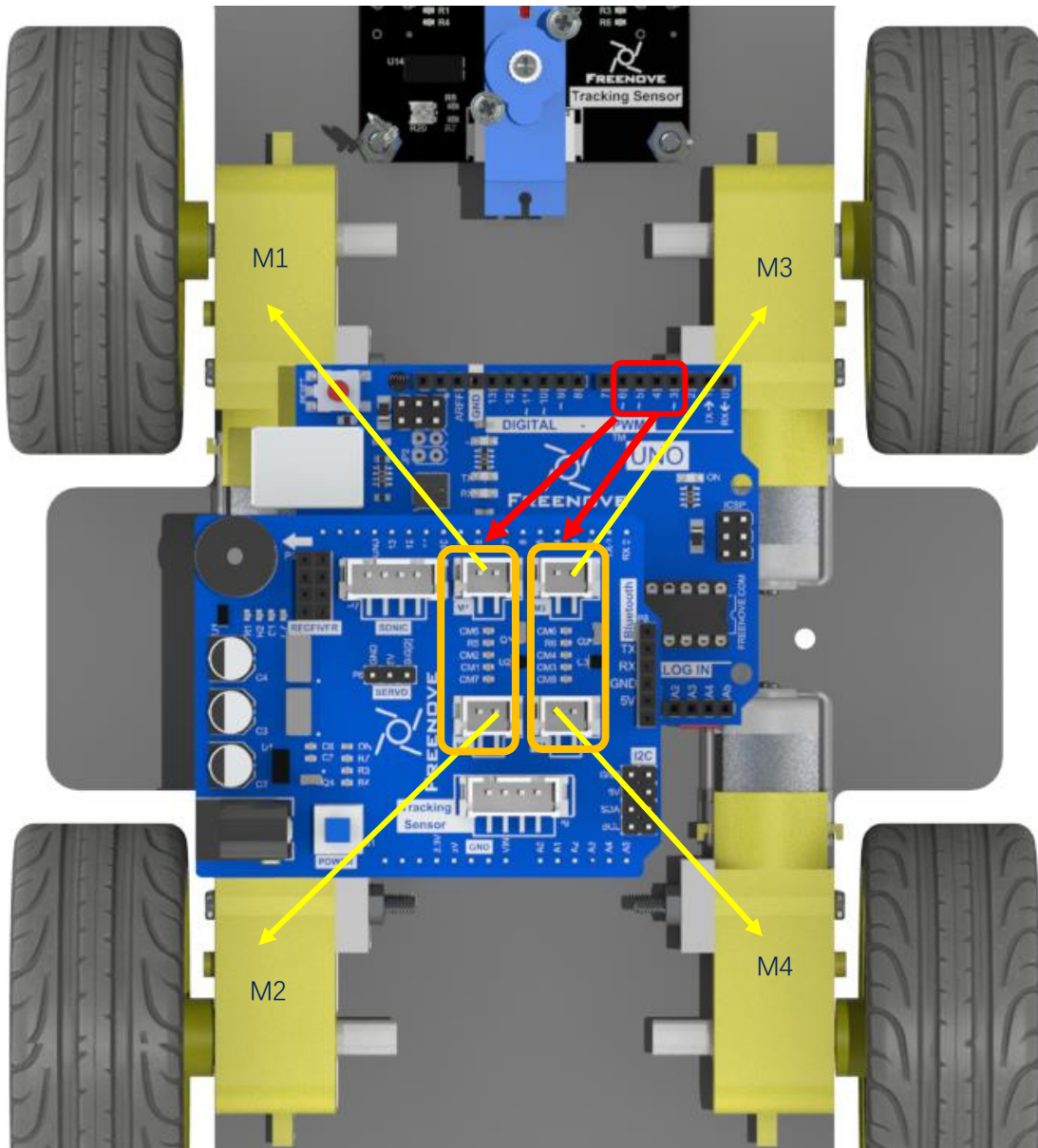
The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.



In this car, M1 and M2 are connected in parallel; M3 and M4 are connected in parallel. Schematic is below:



D3 and D5 pins of control board control M3 and M4 respectively. D4 and D6 pins of board control M1 and M2.



Code

01.1.1_RunMotor_Left_Wheel

If you don't know how to upload code to Arduino, you can refer to [First Use](#).

You need remove Bluetooth Module Module when you upload code.

Next turn off the power. And connect control board to computer with a USB cable. Upload code in the folder: Sketches\01.1.1_RunMotor_Left_Wheel

Then disconnect USB cable and put your car in a place with enough space to move. Or make the car's head stand up and observe the wheel. **The following code also requires this step.**

Turn on the power of Car. Then the left motors and wheels of car will rotate forth and back. And you can

see the speeds are different. You can turn off the power of the car to stop that.

From previous section, we have known that M1 and M2 are connected in parallel and M3 and M4 are connected in parallel. It means that **you can only control motors in the same side together but** not separately. Usually, motors of the same side don't need to rotate in different directions.

The code is below:

```
1  #define PIN_DIRECTION_RIGHT 3
2  #define PIN_DIRECTION_LEFT 4
3  #define PIN_MOTOR_PWM_RIGHT 5
4  #define PIN_MOTOR_PWM_LEFT 6
5
6  void setup() {
7      pinMode(PIN_DIRECTION_LEFT, OUTPUT);
8      pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
9      pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
10     pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
11 }
12
13 void loop() {
14     //Left motors rotates to one direction
15     digitalWrite(PIN_DIRECTION_LEFT, HIGH);
16     analogWrite(PIN_MOTOR_PWM_LEFT, 100);
17     delay(1000);    //delay(ms), 1000ms=1s
18
19     analogWrite(PIN_MOTOR_PWM_LEFT, 0); //Stop
20     delay(1000);
21
22     //left motors rotates to opposite direction
23     digitalWrite(PIN_DIRECTION_LEFT, LOW);
24     analogWrite(PIN_MOTOR_PWM_LEFT, 255);
25     delay(1000);
26
27     analogWrite(PIN_MOTOR_PWM_LEFT, 0); //Stop
28     delay(1000);
29 }
30
```

In the code, following code is uses to define pins.

```
1  #define PIN_DIRECTION_RIGHT 3
2  #define PIN_DIRECTION_LEFT 4
3  #define PIN_MOTOR_PWM_RIGHT 5
4  #define PIN_MOTOR_PWM_LEFT 6
```

As you can see in schematic above. The pin for motors cannot be modified.

#define A B, it means A is B. Use A instead of B for easy maintenance and easy reading.

You can also define pins with int type, like `int PIN_DIRECTION_RIGHT=3`; But this need more RAM space.

Usually, there are two basic main functions for Arduino code, `void setup()` and `void loop()`.

void setup(){ }

The `setup()` function is called when a sketch starts, which is used to initialize variables, pin modes, start using libraries, etc.

The `setup()` function will **only run once**, after each **power up** or **reset** of the Arduino board.

void loop(){ }

This function will loop consecutively. Code in this function will be executed again and again...

There are some other functions integrated by Arduino.

pinMode(pin, mode)

Configures the specified pin to behave either as an input or an output.

Parameters

pin: Arduino pin number to set the mode of.

mode: INPUT, OUTPUT, or INPUT_PULLUP.

digitalWrite(pin, value)

Write a HIGH or a LOW value to a digital pin.

If the pin has been configured as an OUTPUT with `pinMode()`, its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for HIGH, 0V (ground) for LOW.

Parameters

pin: Arduino pin number.

value: HIGH or LOW.

analogWrite(pin, value)

Writes an analog value (PWM wave) to a pin.

You do not need to call `pinMode()` to set the pin as an output before calling `analogWrite()`.

Parameters

pin: Arduino pin to write to. Allowed data types: int.

value: the duty cycle: between 0 (always off) and 255 (always on). Allowed data types: int.

For more details, please refer to: <https://www.arduino.cc/reference/>

Note, as to `analogWrite(pin, value)` of motors for this car, the maximum value is 255. As the motor requires a certain voltage to run, and `analogWrite(pin, 1)` doesn't meet such required voltage., according to different voltage, the value that makes the car run will be much greater than `analogWrite(pin, 1)`.

01.1.2_RunMotor_Right_Wheel

You need remove Bluetooth Module when you upload code.

Upload code in folder: Sketches\01.1.2_RunMotor_Right_Wheel.

Then put your car in a place with enough space to move. Turn ON the power of car. Then the right motors and wheels of car will rotate forth and back. And you can see that the speeds are different.

Because the left motors and right motors are mirrored. When the motors rotate with same direction, the car will react with reversed direction.

The code is below:

```
1 #define PIN_DIRECTION_RIGHT 3
2 #define PIN_DIRECTION_LEFT 4
3 #define PIN_MOTOR_PWM_RIGHT 5
4 #define PIN_MOTOR_PWM_LEFT 6
5
6 void setup() {
7     pinMode(PIN_DIRECTION_LEFT, OUTPUT);
8     pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
9     pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
10    pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
11 }
12
13 void loop() {
14     //Right motors rotate to one direction
15     digitalWrite(PIN_DIRECTION_RIGHT, HIGH);
16     analogWrite(PIN_MOTOR_PWM_RIGHT, 100);
17     delay(1000);
18     analogWrite(PIN_MOTOR_PWM_RIGHT, 0);
19     delay(1000);
20
21     //Right motors rotate to opposite direction
22     digitalWrite(PIN_DIRECTION_RIGHT, LOW);
23     analogWrite(PIN_MOTOR_PWM_RIGHT, 255);
24     delay(1000);
25     analogWrite(PIN_MOTOR_PWM_RIGHT, 0);
26     delay(1000);
27 }
28
```

01.1.3_Car_Move_and_Turn

You need remove Bluetooth Module Module when you upload code.

Upload code in folder: Sketches\01.1.3_Car_Move_and_Turn.

Then put your car in a place with enough space to move. Turn ON the power of car. Then the car will move forth and back, then turn left and turn right. After stopping for 2 second, it repeat the actions.

The code is below:

```
1 #define MOTOR_DIRECTION      0 //If the direction is reversed, change 0 to 1
2 #define PIN_DIRECTION_RIGHT 3
3 #define PIN_DIRECTION_LEFT 4
4 #define PIN_MOTOR_PWM_RIGHT 5
5 #define PIN_MOTOR_PWM_LEFT 6
6
7 void setup() {
8     pinMode(PIN_DIRECTION_LEFT, OUTPUT);
9     pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
```

```
10  pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
11  pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
12  }
13
14  void loop() {
15      //Move forward
16      motorRun(200, 200);
17      delay(1000);
18
19      //Move back
20      motorRun(-200, -200);
21      delay(1000);
22
23      //Turn left
24      motorRun(-200, 200);
25      delay(1000);
26
27      //Turn right
28      motorRun(200, -200);
29      delay(1000);
30
31      //Stop
32      motorRun(0, 0);
33      delay(2000);
34  }
35
36  void motorRun(int speedl, int speedr) {
37      int dirL = 0, dirR = 0;
38      if (speedl > 0) {
39          dirL = 0 ^ MOTOR_DIRECTION;
40      }
41      else {
42          dirL = 1 ^ MOTOR_DIRECTION;
43          speedl = -speedl;
44      }
45      if (speedr > 0) {
46          dirR = 1 ^ MOTOR_DIRECTION;
47      }
48      else {
49          dirR = 0 ^ MOTOR_DIRECTION;
50          speedr = -speedr;
51      }
52      digitalWrite(PIN_DIRECTION_LEFT, dirL);
53      digitalWrite(PIN_DIRECTION_RIGHT, dirR);
```

```
54 analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
55 analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
56 }
```

If you find that the motor is turning in the opposite direction as described in the code, please change the 0 in the code to 1.

```
1 #define MOTOR_DIRECTION 0 //If the direction is reversed, change 0 to 1
```

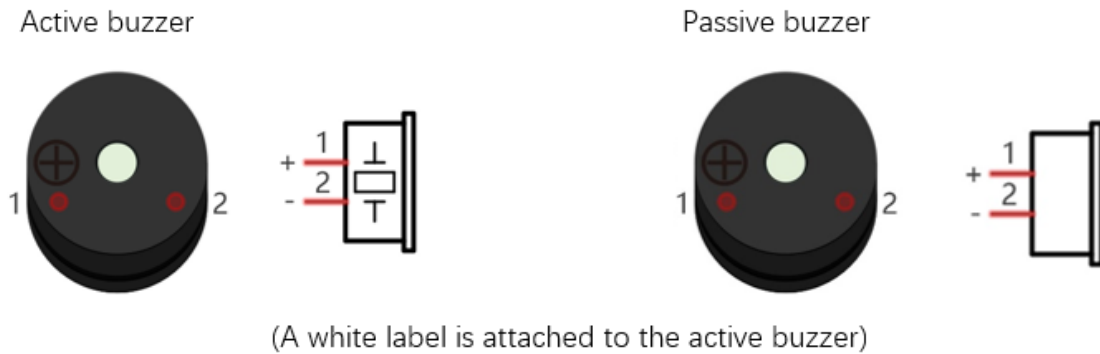
Now we will use `motorRun(200, -200)` as an example to introduce this function.

```
36 void motorRun(int speedl, int speedr) {
37     int dirL = 0, dirR = 0;
38     if (speedl > 0) {
39         dirL = 0 ^ MOTOR_DIRECTION;
40     }
41     else {
42         dirL = 1 ^ MOTOR_DIRECTION;
43         speedl = -speedl;
44     }
45     if (speedr > 0) {
46         dirR = 1 ^ MOTOR_DIRECTION;
47     }
48     else {
49         dirR = 0 ^ MOTOR_DIRECTION;
50         speedr = -speedr;
51     }
52     digitalWrite(PIN_DIRECTION_LEFT, dirL);
53     digitalWrite(PIN_DIRECTION_RIGHT, dirR);
54     analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
55     analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
56 }
```

The left motors and right motors are **mirrored**. When the motors on two side rotate with same direction, the car will turn. In order to make a car move forth or back, we need set motor on two side with different direction. So `motorRun(200, -200)` will make the car turn right.

1.2 Buzzer and Battery level

A buzzer is an audio component. There are both active and passive types of buzzers. Active buzzers have oscillator inside, these will sound as long as power is supplied. Passive buzzers require an external oscillator signal (generally using PWM with different frequencies) to make a sound.



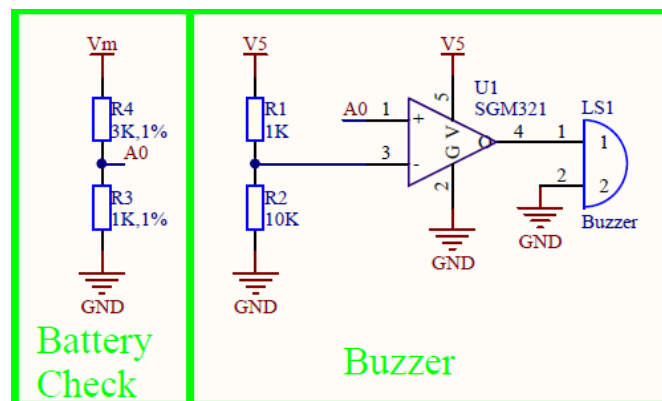
Active buzzers are easier to use. Generally, they can only make a specific sound frequency.

Passive buzzers require an external circuit to make sounds, but they can be controlled to make sounds of various frequencies. The resonant frequency of the passive buzzer is 2kHz, which means the passive buzzer is the loudest when resonant frequency is 2kHz.

In this car, the buzzer is an active buzzer. So it can only make a specific frequency of sound. Function tone() of Arduino cannot be used for this kind of buzzer.

Since the control board has limited number of IOs. Buzzer and battery level are not used frequently. So we use one IO of control board controlling buzzer and detecting battery level in different time

The schematic of buzzer and battery is below:



As you can see they use same IO A0.

Battery level should be $V_m = 4 \cdot V_0$. V_0 is the voltage detected by A0 of control board.

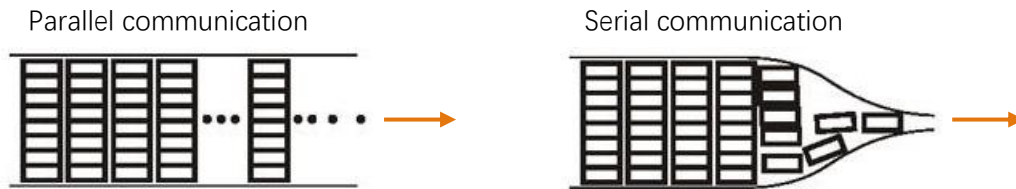
The maximum battery level is 8.4v ($V_0 = 2.1V$).

The right circuit means if $V_0 > 4.5V$, the buzzer will make sound.

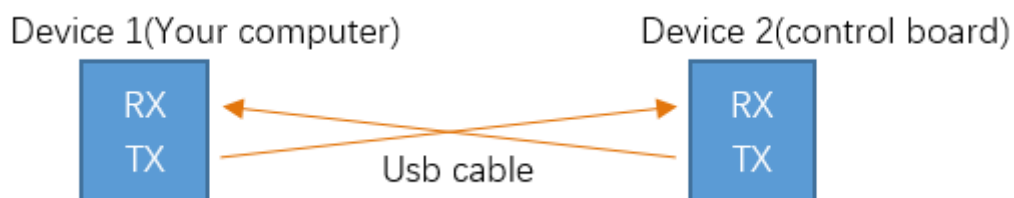
If V_0 is within(0~2.1v), it is used for battery. If $V_0 > 4.5V$, it is used for buzzer.

Serial Communication

Serial communication uses one data cable to transfer data one bit by another in turn. Parallel communication means that the data is transmitted simultaneously on multiple cables. Serial communication takes only a few cables to exchange information between systems, which is especially suitable for computers to computers, long distance communication between computers and peripherals. Parallel communication is faster, but it requires more cables and higher cost, so it is not appropriate for long distance communication.



Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections of two devices use is as follows:



For serial communication, the **baud rate in both sides must be the same**. The baud rates commonly used are 9600 and 115200.

Computer identifies serial devices connected to your computer as COMx. We can use the Serial Monitor window of Arduino Software to communicate with Freenove control board.

Code

01.2.1_Buzzer

You need remove Bluetooth Module Module when you upload code.

Upload code in Sketches\01.2.1_Buzzer. Then turn the switch of car.

You will hear b-bbbb----bbbb----bbbb----

The code in setup() is only executed once. Code in loop() will be executed circularly.

```

1  #define PIN_BATTERY    A0
2  #define PIN_BUZZER     A0
3
4  void setup() {
5      pinMode(PIN_BUZZER, OUTPUT);
6      digitalWrite(PIN_BUZZER, HIGH);
7      delay(100);
8      digitalWrite(PIN_BUZZER, LOW);
9  }
10

```

```

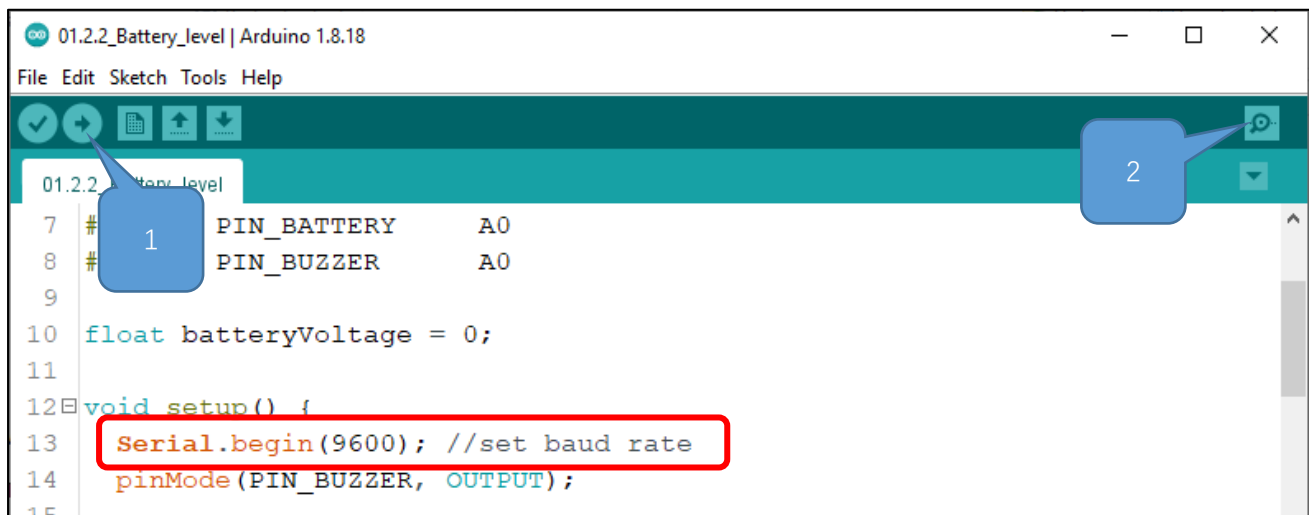
11 void loop() {
12     for (int i = 0; i < 4; i++) {
13         digitalWrite(PIN_BUZZER, HIGH); //turn on buzzer
14         delay(100);
15         digitalWrite(PIN_BUZZER, LOW); //turn off buzzer
16         delay(100);
17     }
18     delay(500);
19 }

```

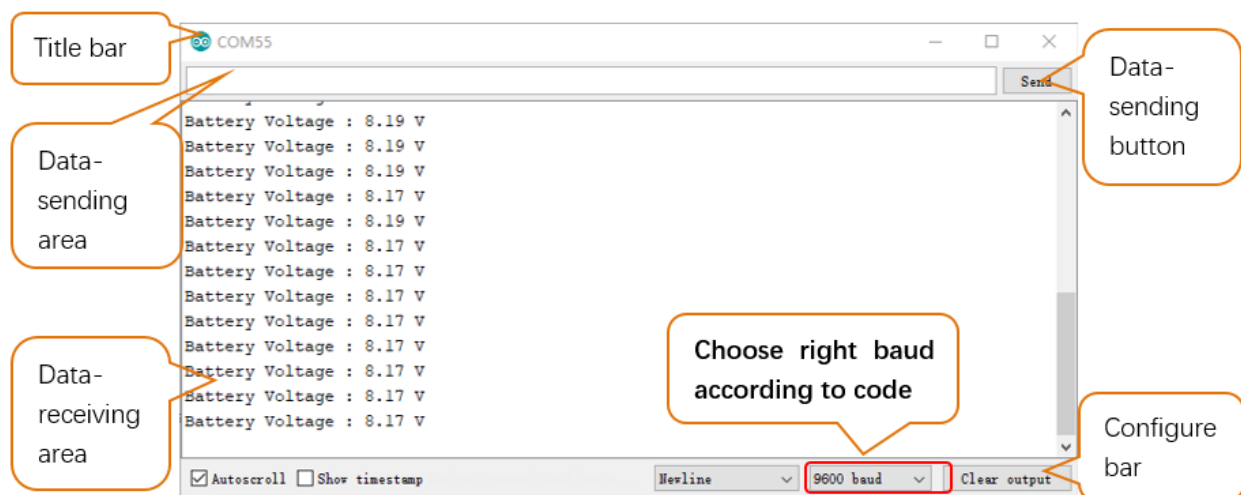
01.2.2_Battery_level

Upload code in Sketches\01.2.2_Battery_level. Don't disconnect the USB cable. Then turn on the switch of the car.

And then click the Serial Monitor icon to open the Serial Monitor window.



Then you will hear buzzer making sounds twice. And then see the Interface of Serial Monitor window as follows. If you can't open it, make sure Freenove control board had been connected to the computer, and choose the right serial port in the menu bar "Tools-Port".



The code is below:

```
1  #define PIN_BATTERY    A0
2  #define PIN_BUZZER     A0
3
4  float batteryVoltage = 0;
5
6  void setup() {
7      Serial.begin(9600); //set baud rate
8      pinMode(PIN_BUZZER, OUTPUT);
9
10     digitalWrite(PIN_BUZZER, HIGH);
11     delay(500);
12     digitalWrite(PIN_BUZZER, LOW);
13     delay(500);
14     digitalWrite(PIN_BUZZER, HIGH);
15     delay(500);
16     digitalWrite(PIN_BUZZER, LOW);
17 }
18
19 void loop() {
20     //Because the buzzer and battery voltage detection use A0 pin together,
21     //the buzzer must be turned off when reading battery voltage,
22     //otherwise the read battery voltage is incorrect.
23
24     //Turn off buzzer
25     pinMode(PIN_BUZZER, false);
26     digitalWrite(PIN_BUZZER, false);
27
28     //print battery voltage
29     if (getBatteryVoltage()) {
30         Serial.print("Battery Voltage : ");
31         Serial.print(batteryVoltage);
32         Serial.println(" V");
33     }
34     else {
35         Serial.print("Port busy! Please turn off buzzer before reading battery voltage.");
36     }
37     delay(1000);
38 }
39
40 bool getBatteryVoltage() {
41     pinMode(PIN_BATTERY, INPUT);
42     int batteryADC = analogRead(PIN_BATTERY);
43     if (batteryADC < 614) // 614/1023* 5=3V, reasonable Voltage:<2.1V
```

```

44 {
45     batteryVoltage = batteryADC / 1023.0 * 5.0 * 4;
46     return true;
47 } else {
48     return false;
49 }
50 }

```

In this code, different Data Types are mentioned: float, bool, void.

If you are not familiar with data types, please refer to <https://www.arduino.cc/reference/>

VARIABLES

Arduino data types and constants.

Constants	Conversion	float	Variable Scope & Qualifiers
Floating Point Constants	(unsigned int)	int	const
Integer Constants	(unsigned long)	long	scope
HIGH LOW	byte()	short	static
INPUT OUTPUT INPUT_PULLUP	char()	size_t	volatile
LED_BUILTIN	float()	string	
true false	int()	unsigned char	Utilities
	long()	unsigned int	PROGMEM
	word()	unsigned long	sizeof()
		void	
		word	
	Data Types		
	String()		
	array		
	bool		
	boolean		
	byte		
	char		
	double		

The function is defined in form: `data type function name ()`.

If it is void function name(), after it is executed, it will return nothing.

```

28 //print battery voltage
29 if (getBatteryVoltage()) {
30     Serial.print("Battery Voltage : ");
31     Serial.print(batteryVoltage);
32     Serial.println(" V");
33 }
34 else {
35     Serial.print("Port busy! Please turn off buzzer before reading battery voltage.");
36 }
37 delay(1000);
38 }
39
40 bool getBatteryVoltage() {
41     pinMode(PIN_BATTERY, INPUT);
42     int batteryADC = analogRead(PIN_BATTERY);
43     if (batteryADC < 614) // 614/1023* 5=3V, reasonable Voltage:<2.1V
44     {
45         batteryVoltage = batteryADC / 1023.0 * 5.0 * 4;
46         return true;
47     }else {
48         return false;
49     }
50 }

```

Code is always executed line by line. When `if (getBatteryVoltage())` is executed, the program will turn to execute `bool getBatteryVoltage(){ }`. After return, the program will execute code in `if { }`.

Serial.print()

Syntax is below:

`Serial.print(val)`

`Serial.print(val, format)`

Serial: serial port object. See the list of available serial ports for each board on the Serial main page.

val: the value to print. Allowed data types: any data type.

The only difference is that `Serial.println()` will send a "\n". content of next print() or println() will be shown in a new line.

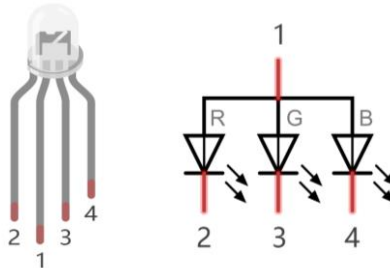
analogRead(pin)

It will map input voltages between 0 and the operating voltage(5V or 3.3V) into integer values between 0 and 1023. For 5v as an example, `ADCvalue= analogRead( pin)`

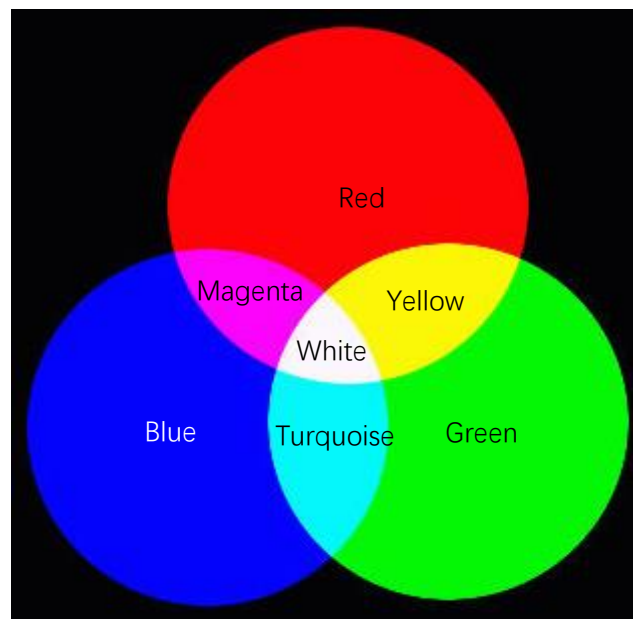
If you want to get the voltage value. It should be $V = \text{ADCvalue} / 1023 * 5V$.

1.3 RGB

A RGB LED has 3 LEDs integrated into one LED component. It can respectively emit Red, Green and Blue light. In order to do this, it requires 4 pins (this is also how you identify it). The long pin (1) is the common which is the Cathode (+) or positive lead, the other 3 are the Anodes (-) or negative leads. A rendering of a RGB LED and its electronic symbol are shown below. We can make RGB LED emit various colors of light and brightness by controlling the 3 Anodes (2, 3 & 4) of the RGB LED

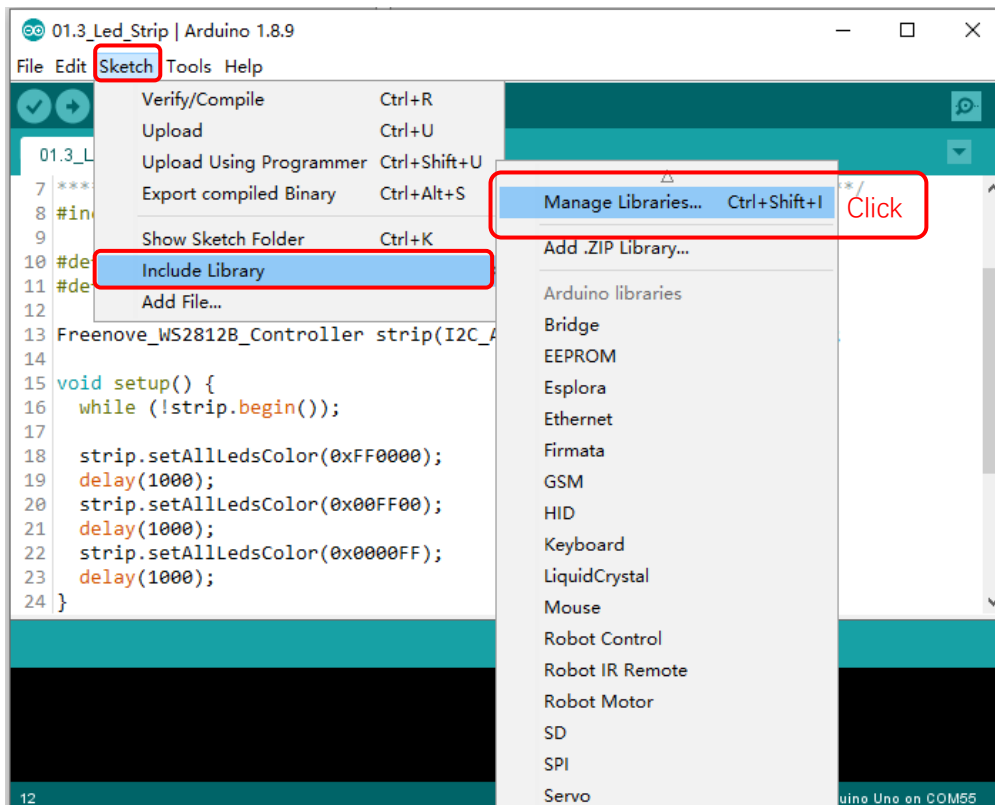


Red, Green, and Blue light are called 3 Primary Colors when discussing light (Note: for pigments such as paints, the 3 Primary Colors are Red, Blue and Yellow). When you combine these three Primary Colors of light with varied brightness, they can produce almost any color of visible light. Computer screens, single pixels of cell phone screens, neon lamps, etc. can all produce millions of colors due to this phenomenon.

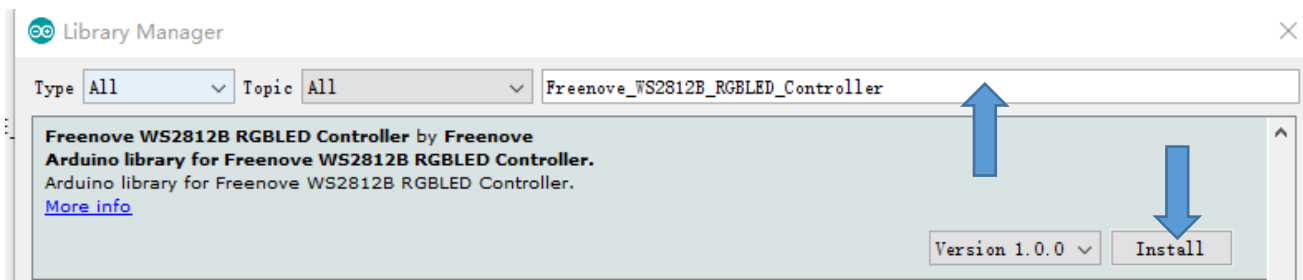


We know from previous section that, control board controls LEDs to emit a total of 256(0-255) different brightness with PWM. So, through the combination of three different colors of LEDs, RGB LED can emit $256^3=16777216$ Colors, 16Million colors.

Include Library



Enter “Freenove_WS2812B_RGBLED_Controller” and press “Enter” key to search.
Find and **install** the latest version (now it is version 1.0.0).



Then go back and upload the code.

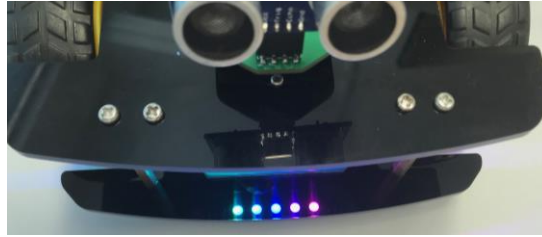
Code

01.3_Led_Strip

You need remove Bluetooth Module Module when you upload code.

Upload code in Sketches\01.3_Led_Strip. Then turn the power of car.

You can find a proper view to observe LED colors on the bottom acrylic board.



You will see that all LEDs show red, green, blue, yellow in turn. Then all LEDs turn off.

Then first 5 LEDs show different colors one by one. Then all LEDs show the same color and the color changes smoothly. Then different LEDs show different colors and they change smoothly.

The code is below:

```

1  #include "Freenove_WS2812B_RGBLED_Controller.h"
2
3  #define I2C_ADDRESS  0x20
4  #define LEDS_COUNT   10 //it defines number of LEDs
5
6  Freenove_WS2812B_Controller strip(I2C_ADDRESS, LEDS_COUNT, TYPE_GRB); //initialization
7
8  void setup() {
9      while (!strip.begin());           //judge if initialization success
10
11      strip.setAllLedsColor(0xFF0000); //Set all LED color to red
12      delay(2000);
13      strip.setAllLedsColor(0x00FF00); //set all LED color to green
14      delay(2000);
15      strip.setAllLedsColor(0x0000FF); //set all LED color to blue
16      delay(2000);
17      strip.setAllLedsColor(255, 255, 0); //set all LED color to yellow. This is just different
18  form of rgb value.
19      delay(2000);
20      strip.setAllLedsColor(0, 0, 0);   //set all LED off.
21      delay(2000);
22
23      strip.setLedColor(0, 255, 0, 0);  //set the NO.0 LED to red
24      delay(1000);
25      strip.setLedColor(1, 0, 255, 0);  //set the NO.1 LED to green
26      delay(1000);
27      strip.setLedColor(2, 0, 0, 255);  //set the NO.2 LED to blue
28      delay(1000);
29      strip.setLedColor(3, 255, 255, 0); //set the NO.3 LED to yellow
30      delay(1000);
31      strip.setLedColor(4, 255, 0, 255); //set the NO.4 LED to purple
32      delay(1000);
33  }
34

```

```

35 void loop() {
36     for (int k = 0; k < 255; k = k + 2) {
37         strip.setAllLedsColorData(strip.Wheel(k)); // set color data for all LED
38         strip.show();                               // show the color set before
39         delay(50);
40     }
41     delay(3000);
42
43     for (int j = 0; j < 255; j += 2) {
44         for (int i = 0; i < LEDS_COUNT; i++) {
45             strip.setLedColorData(i, strip.Wheel(i * 256 / LEDS_COUNT + j)); //set color data for
46 LED one by one
47         }
48         strip.show(); // show the color set
49         delay(50);
50     }
51 }

```

First, you need install and include the library.

```
#include "Freenove_WS2812B_RGBLED_Controller.h"
```

Then set number of LED. The maximum is 255. But it is limited by power supply.

```
#define LEDS_COUNT 10 //it defines number of LEDs
```

There are two ways to set LED color.

A, set LED color directly.

1) set all LED colors

strip.setAllLedsColor(rgb value)

rgb value should be Hexadecimal value, like `strip.setAllLedsColor(0x0000FF)`.

strip.setAllLedsColor(rvalue, gvalue, bvalue)

This is another form, Like `strip.setAllLedsColor(255, 255, 0)`.

You can find more rgb values and colors here https://www.rapidtables.com/web/color/RGB_Color.html

2) set color for one LED

strip.setLedColor(LED_Index, rgb_value) or **strip.setLedColor(LED_Index, rvalue, gvalue, bvalue)**

LED No. starts from 0, like `strip.setLedColor(0, 255, 0, 0)`.

B, Set color data first. Then use show function.

This is a more **efficient** way to set colors for many LEDs. It will make the code run faster than before.

1) set all LED color

strip.setAllLedsColorData(rgb value) or **strip.setAllLedsColorData(rvalue, gvalue, bvalue)**

After you complete setting color, just add **strip.show()**

2) set color for one LED

strip.setLedColorData(LED_Index, rgb value) or **strip.setLedColorData(LED_Index, rvalue, gvalue, bvalue)**

After you complete setting color, just add **strip.show()**

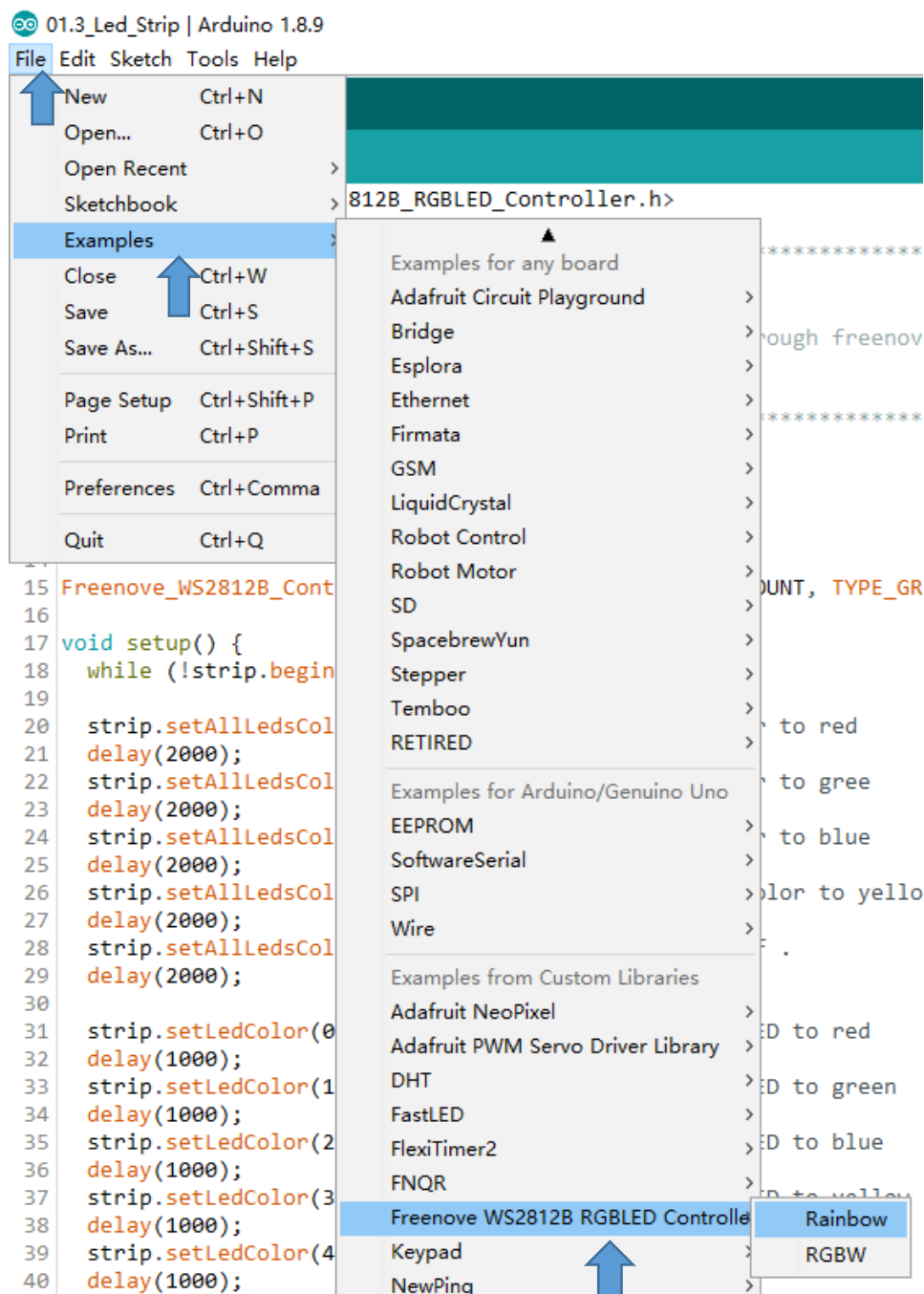
strip.Wheel(value) value can be 0~255.

Different values correspond to different colors. You can take **strip.Wheel(value)** as a rgb value to use in the

code, like `strip.setAllLedsColorData(strip.Wheel(k)).`



You can find some examples of the library below:



1.4 Integrate functions

Now we will write a code, which include all previous functions.

Code

01.4.1_Integrate_Functions

You need remove Bluetooth Module Module first when you upload code.

Upload code in Sketches\01.4.1_Integrate_Functions. **Don't separate the files in the folder.**

Then put your car in a place with enough space to move. Turn ON the power of car.

```
1  #include "Freenove_WS2812B_RGBLED_Controller.h"
2
3  #define I2C_ADDRESS  0x20 //define I2C address of LEDs
4  #define LEDS_COUNT   10  //it defines number of LEDs
5
6  #define MOTOR_DIRECTION  0 //If the direction is reversed, change 0 to 1
7  //define motor pins
8  #define PIN_DIRECTION_LEFT  4
9  #define PIN_DIRECTION_RIGHT 3
10 #define PIN_MOTOR_PWM_LEFT  6
11 #define PIN_MOTOR_PWM_RIGHT 5
12
13 #define PIN_BATTERY      A0
14 #define PIN_BUZZER       A0
15 #define MOTOR_PWM_DEAD   10
16 float batteryVoltage = 0;
17 bool isBuzzered = false;
18
19 Freenove_WS2812B_Controller strip(I2C_ADDRESS, LEDS_COUNT, TYPE_GRB); //initialization
20
21 void setup() {
22     while (!strip.begin()); //judge if initialization successes
23
24     Serial.begin(9600); //set baud rate
25     pinsSetup();        //set pin mode (output or input )
26
27     strip.setAllLedsColor(0x00FF00); //set all LED color to green
28     motorRun(100, 100); //Car move forward
29     alarm(4, 1);        //4 beat, 1 repeat
30     resetCarAction();   //Stop the car and turn off the buzzer
31
32     strip.setAllLedsColor(0xFF0000); //Set all LED color to red
33     motorRun(-100, -100); //Car move back
```

```
34   setBuzzer(true);           //turn on buzzer
35   delay(1000);
36   resetCarAction();         //Stop the car and turn off the buzzer
37 }
38
39 void loop() {
40   if (getBatteryVoltage() == true) {
41     Serial.print("voltage: ");
42     Serial.println(batteryVoltage);
43   }
44
45   int r = random(0, 255); //a random value between 0~255
46   int g = random(0, 255); //a random value between 0~255
47   int b = random(0, 255); //a random value between 0~255
48   strip.setAllLedsColor(r, g, b); //Set all LED color with current r, g, b value
49   delay(1000);
50 }
51
52 void pinsSetup() {
53   pinMode(PIN_DIRECTION_LEFT, OUTPUT);
54   pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
55   pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
56   pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
57   setBuzzer(false); //Turn off the Buzzer
58 }
59
60 void motorRun(int speedl, int speedr) {
61   int dirL = 0, dirR = 0;
62   if (speedl > 0) {
63     dirL = 0 ^ MOTOR_DIRECTION;
64   }
65   else {
66     dirL = 1 ^ MOTOR_DIRECTION;
67     speedl = -speedl;
68   }
69   if (speedr > 0) {
70     dirR = 1 ^ MOTOR_DIRECTION;
71   }
72   else {
73     dirR = 0 ^ MOTOR_DIRECTION;
74     speedr = -speedr;
75   }
76
77   speedl = constrain(speedl, 0, 255);
```

```
78 speedr = constrain(speedr, 0, 255);
79
80 if (abs(speedl) < MOTOR_PWM_DEAD && abs(speedr) < MOTOR_PWM_DEAD) {
81     speedl = 0;
82     speedr = 0;
83 }
84 digitalWrite(PIN_DIRECTION_LEFT, dirL);
85 digitalWrite(PIN_DIRECTION_RIGHT, dirR);
86 analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
87 analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
88 }
89
90 bool getBatteryVoltage() {
91     if (!isBuzzered) {
92         pinMode(PIN_BATTERY, INPUT);
93         int batteryADC = analogRead(PIN_BATTERY);
94         if (batteryADC < 614)    // 3V/12V ,Voltage read: <2.1V/8.4V
95         {
96             batteryVoltage = batteryADC / 1023.0 * 5.0 * 4;
97             return true;
98         }
99     }
100     return false;
101 }
102
103 void setBuzzer(bool flag) {
104     isBuzzered = flag;
105     pinMode(PIN_BUZZER, flag);
106     digitalWrite(PIN_BUZZER, flag);
107 }
108
109 void alarm(u8 beat, u8 repeat) {
110     beat = constrain(beat, 1, 9);
111     repeat = constrain(repeat, 1, 255);
112     for (int j = 0; j < repeat; j++) {
113         for (int i = 0; i < beat; i++) {
114             setBuzzer(true);
115             delay(100);
116             setBuzzer(false);
117             delay(100);
118         }
119         delay(500);
120     }
121 }
```

```

122
123 void resetCarAction() {
124     motorRun(0, 0);
125     setBuzzer(false);
126 }

```

01.4.2_Library_Integrate_Functions

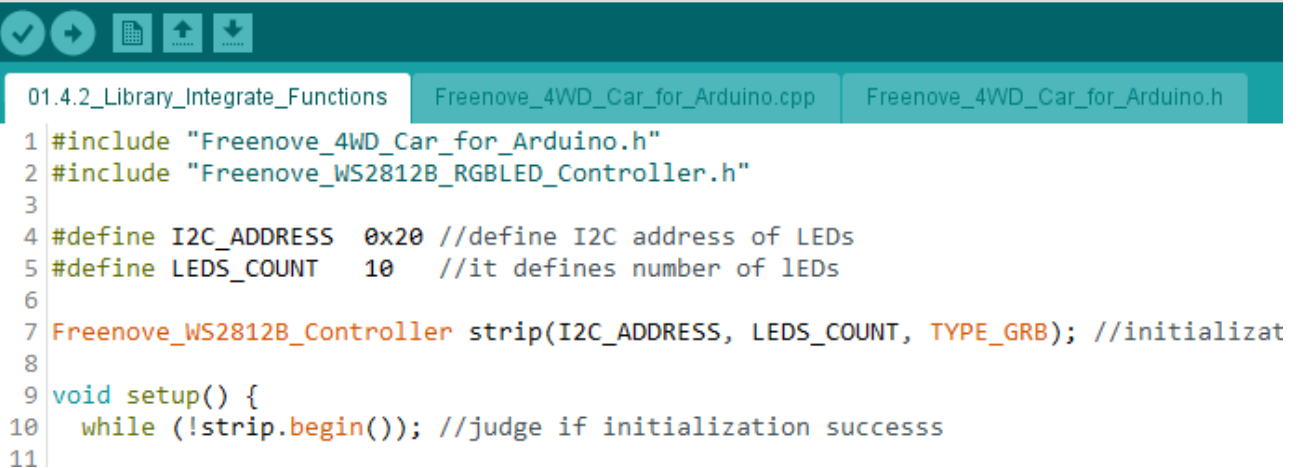
Upload code in sketches\01.4.2_Library_Integrate_Functions, which has the same function with code **01.4.1_Integrate_Functions**. The only difference is that we write some codes into library. We will use this library later.

If libraries don't work on your computer, it may be related to your computer system.

Don't worry. We also provide one code for later projects.

01.4.2_Library_Integrate_Functions | Arduino 1.8.9

File Edit Sketch Tools Help



```

01.4.2_Library_Integrate_Functions  Freenove_4WD_Car_for_Arduino.cpp  Freenove_4WD_Car_for_Arduino.h
1 #include "Freenove_4WD_Car_for_Arduino.h"
2 #include "Freenove_WS2812B_RGBLED_Controller.h"
3
4 #define I2C_ADDRESS 0x20 //define I2C address of LEDs
5 #define LEDS_COUNT 10 //it defines number of LEDs
6
7 Freenove_WS2812B_Controller strip(I2C_ADDRESS, LEDS_COUNT, TYPE_GRB); //initializat
8
9 void setup() {
10     while (!strip.begin()); //judge if initialization successss
11

```

The main code is as below. The Freenove_4WD_Car_for_Arduino.h has defined some pins, variables and functions. We can use them directly. We do not need redefine them again.

```

1 #include "Freenove_4WD_Car_for_Arduino.h"
2 #include "Freenove_WS2812B_RGBLED_Controller.h"
3
4 #define I2C_ADDRESS 0x20 //define I2C address of LEDs
5 #define LEDS_COUNT 10 //it defines number of LEDs
6 Freenove_WS2812B_Controller strip(I2C_ADDRESS, LEDS_COUNT, TYPE_GRB); //initialization
7
8 void setup() {
9     while (!strip.begin()); //judge if initialization successses
10     Serial.begin(9600); //set baud rate
11     pinsSetup(); //set pin mode (output or input )
12
13     strip.setAllLedsColor(0x00FF00); //set all LED color to green
14     motorRun(100, 100); //Car move forward
15     alarm(4, 1); //4 beat, 1 repeat

```

```

16  resetCarAction();    //Stop the car and turn off the buzzer
17
18  strip.setAllLedsColor(0xFF0000); //Set all LED color to red
19  motorRun(-100, -100); //Car move back
20  setBuzzer(true);     //turn on buzzer
21  delay(1000);
22  resetCarAction();    //Stop the car and turn off the buzzer
23 }
24
25 void loop() {
26     if (getBatteryVoltage() == true) {
27         Serial.print("voltage: ");
28         Serial.println(batteryVoltage);
29     }
30     int r = random(0, 255); //a random value between 0~255
31     int g = random(0, 255); //a random value between 0~255
32     int b = random(0, 255); //a random value between 0~255
33     strip.setAllLedsColor(r, g, b); //Set all LED color with current r, g, b value
34     delay(1000);
35 }

```

In this code, Freenove_4WD_Car_for_Arduino library is used.

```
#include "Freenove_4WD_Car_for_Arduino.h"
```

And you can see, we did not define pins and functions in the main. But we can use them.

Now let's learn a new skill, using library.

Open the directory Sketches\01.4.2_Library_Integrate_Functions. You will see .cpp and .h file. They are **library files**.

When use the library in new code, you just need put this two files into the same directory of your code.

 01.4.2_Library_Integrate_Functions.ino
 Freenove_4WD_Car_for_Arduino.cpp
 Freenove_4WD_Car_for_Arduino.h

This library will be used latter, which will make programing more efficient.

Now we will introduce content in the library.

First, check **Freenove_4WD_Car_for_Arduino.h**.

This file defines some pins and function.

After you write the code `#include "Freenove_4WD_Car_for_Arduino.h"`

You will be able to use the pins define and functions in Freenove_4WD_Car_for_Arduino.h. And you don't need to define the pins and build the functions any more in your code.

```

1  // Freenove_4WD_Car_for_Arduino.h
2  #ifndef _FREENOVE_4WD_CAR_FOR_ARDUINO_h
3  #define _FREENOVE_4WD_CAR_FOR_ARDUINO_h

```

```

4  #if defined(ARDUINO) && ARDUINO >= 100
5      #include "arduino.h"
6  #else
7      #include "WProgram.h"
8  #endif
9  #define PIN_SERVO                2
10 #define MOTOR_DIRECTION           0 //If the direction is reversed, change 0 to 1
11 //define motor pin
12 #define PIN_DIRECTION_LEFT        4
13 #define PIN_DIRECTION_RIGHT       3
14 #define PIN_MOTOR_PWM_LEFT        6
15 #define PIN_MOTOR_PWM_RIGHT       5
16
17 //define ultrasonic module pin
18 #define PIN_SONIC_TRIG             7
19 #define PIN_SONIC_ECHO             8
20
21 #define PIN_IRREMOTE_RECV         9
22 //define SPI pin
23 #define PIN_SPI_CE                 9
24 #define PIN_SPI_CSN                10
25 #define PIN_SPI_MOSI               11
26 #define PIN_SPI_MISO               12
27 #define PIN_SPI_SCK               13
28
29 #define PIN_BATTERY                A0
30 #define PIN_BUZZER                 A0
31
32 //define tracking sensor pin
33 #define PIN_TRACKING_LEFT          A1
34 #define PIN_TRACKING_CENTER        A2
35 #define PIN_TRACKING_RIGHT         A3
36 #define MOTOR_PWM_DEAD            5
37
38 //define functions
39 // all following content are used in integrated code above
40 extern float batteryVoltage;
41 void pinsSetup();
42 void motorRun(int speedl, int speedr);
43 bool getBatteryVoltage();
44 void setBuzzer(bool flag);
45 void alarm(u8 beat, u8 repeat);
46 void resetCarAction();
47 #endif

```

Then check **Freenove_4WD_Car_for_Arduino.cpp**.

Its name should be the same as with .h file. We define functions in .h file and implement its content in .cpp. You can see how the function work.

```
1  #include "Freenove_4WD_Car_for_Arduino.h"
2  float batteryVoltage = 0;
3  bool isBuzzered = false;
4
5  void pinsSetup() {
6      //define motor pin
7      pinMode(PIN_DIRECTION_LEFT, OUTPUT);
8      pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
9      pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
10     pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
11
12     //define ultrasonic module pin
13     pinMode(PIN_SONIC_TRIG, OUTPUT);
14     pinMode(PIN_SONIC_ECHO, INPUT);
15
16     //define tracking sensor pin
17     pinMode(PIN_TRACKING_LEFT, INPUT);
18     pinMode(PIN_TRACKING_RIGHT, INPUT);
19     pinMode(PIN_TRACKING_CENTER, INPUT);
20     setBuzzer(false); // turn off buzzer
21 }
22
23 void motorRun(int speedl, int speedr) {
24     int dirL = 0, dirR = 0;
25     if (speedl > 0) {
26         dirL = 0 ^ MOTOR_DIRECTION;
27     }
28     else {
29         dirL = 1 ^ MOTOR_DIRECTION;
30         speedl = -speedl;
31     }
32     if (speedr > 0) {
33         dirR = 1 ^ MOTOR_DIRECTION;
34     }
35     else {
36         dirR = 0 ^ MOTOR_DIRECTION;
37         speedr = -speedr;
38     }
39     speedl = constrain(speedl, 0, 255); // speedl absolute value should be within 0~255
40     speedr = constrain(speedr, 0, 255); // speedr absolute value should be within 0~255
41     digitalWrite(PIN_DIRECTION_LEFT, dirL);
```

```
42 digitalWrite(PIN_DIRECTION_RIGHT, dirR);
43 analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
44 analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
45 }
46
47 bool getBatteryVoltage() {
48     if (!isBuzzered) {
49         pinMode(PIN_BATTERY, INPUT);
50         int batteryADC = analogRead(PIN_BATTERY);
51         if (batteryADC < 614) // 3V/12V ,Voltage read: <2.1V/8.4V
52         {
53             batteryVoltage = batteryADC / 1023.0 * 5.0 * 4;
54             return true;
55         }
56     }
57     return false;
58 }
59 void setBuzzer(bool flag) {
60     isBuzzered = flag;
61     pinMode(PIN_BUZZER, flag);
62     digitalWrite(PIN_BUZZER, flag);
63 }
64 void alarm(u8 beat, u8 repeat) {
65     beat = constrain(beat, 1, 9);
66     repeat = constrain(repeat, 1, 255);
67     for (int j = 0; j < repeat; j++) {
68         for (int i = 0; i < beat; i++) {
69             setBuzzer(true);
70             delay(100);
71             setBuzzer(false);
72             delay(100);
73         }
74         delay(500);
75     }
76 }
77 void resetCarAction() {
78     motorRun(0, 0);
79     setBuzzer(false);
80 }
```

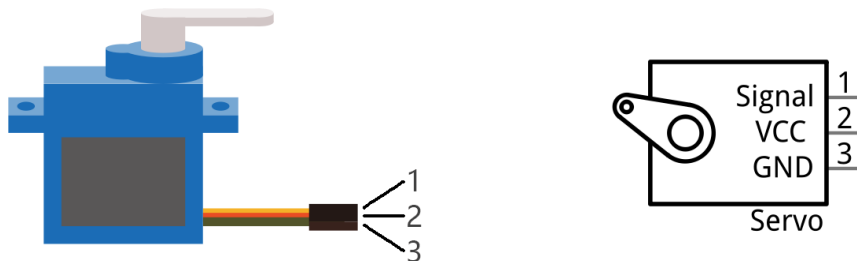
Chapter 2 Obstacle avoidance

In this chapter, we will introduce servo and ultrasonic module. And then make a car that can automatically obstacles.

If you have any concerns, please feel free to contact us via support@freenove.com

2.1 Servo

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their "horn". Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.



We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the servo. The lasting time 0.5ms-2.5ms of PWM single cycle high level corresponds to the servo angle 0 degrees - 180 degrees linearly. Part of the corresponding values are as follows:

High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	90 degree
2ms	135 degree
2.5ms	180 degree

When you change the servo signal, the servo will rotate to the designated position.

Code

02.1_Servo

You need remove Bluetooth Module Module first when you upload code.

Upload code in Sketches\02.1_Servo. Then turn on the power of car.

Then you will see servo sweep among 45°, 90° and 135°.

The code is below:

```
1 #include "Servo.h" //servo library
```

```
2
3 #define PIN_SERVO 2 //define servo pin
4
5 Servo; //create servo object to control a servo
6 char servoOffset = 0; // change the value to Calibrate servo
7
8 void setup() {
9     servo.attach(PIN_SERVO); //initialize servo
10    servo.write(90 + servoOffset); //Calibrate servo
11 }
12
13 void loop() {
14     servo.write(45); //make servo rotate to 45°
15     delay(1000); //delay 1000ms
16
17     servo.write(90); //make servo rotate to 90°
18     delay(1000);
19
20     servo.write(135); //make servo rotate to 135°
21     delay(1000);
22
23     servo.write(90); //make servo rotate to 90°
24     delay(1000);
25 }
26
```

Servo library

There are some functions of this library:

servo.attach(pin)

Attach the Servo variable to a pin.

pin: servo pin it is an initialization step to control a servo.

servo.write(angle)

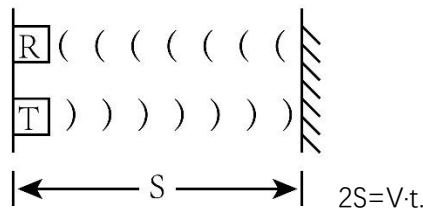
Writes a value to the servo, controlling the shaft accordingly.

angle: the value to write to the servo, from 0 to 180

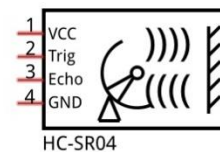
For more details, please refer to: <https://www.arduino.cc/en/Reference/Servo>

2.2 Ultrasonic ranging module

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will reflect when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (Δt) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about $v=340\text{m/s}$, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: $s=vt/2$.



The HC-SR04 Ultrasonic Ranging Module integrates a both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the HC SR04 Ultrasonic Ranging Module are shown below:

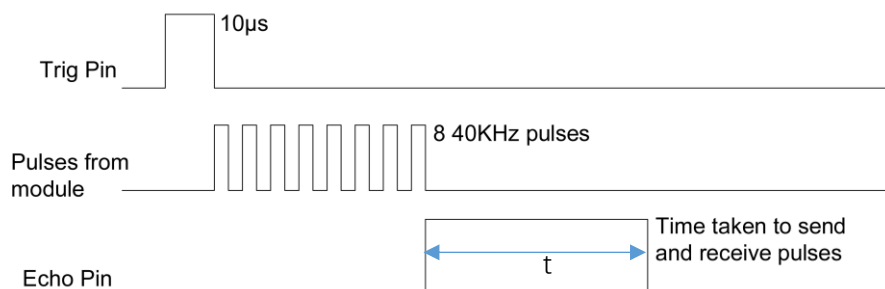


Pin description:

Pin name	Pin number	Description
Vcc	1	Positive electrode of power supply, the voltage is 5V
Trig	2	Triger pin
Echo	3	Echo pin
Gnd	4	Negative electrode of power supply

Instructions for use:

Output a high-level pulse in Trig pin lasting for least $10\mu\text{s}$, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled **up**. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled **down**. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, $s=vt/2$. This is done constantly.



Code

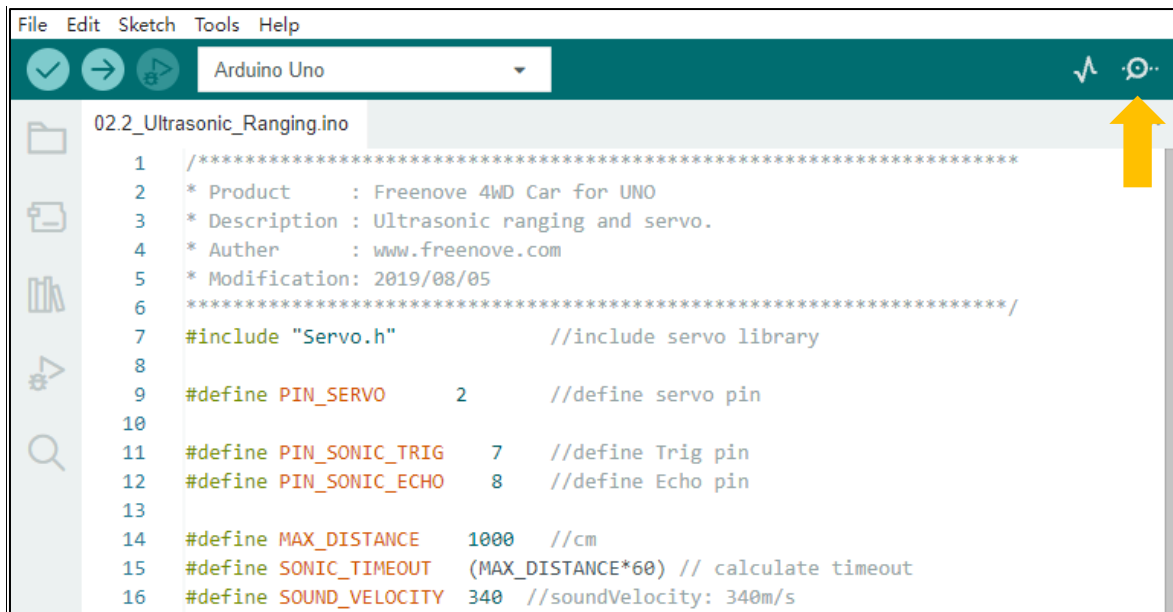
02.2_Ultrasonic_Ranging

This project is used to set servo different angles and detect obstacle distance at each angle.

You need remove Bluetooth Module Module first when you upload code.

Upload code in Sketches\02.2_Ultrasonic_Ranging.

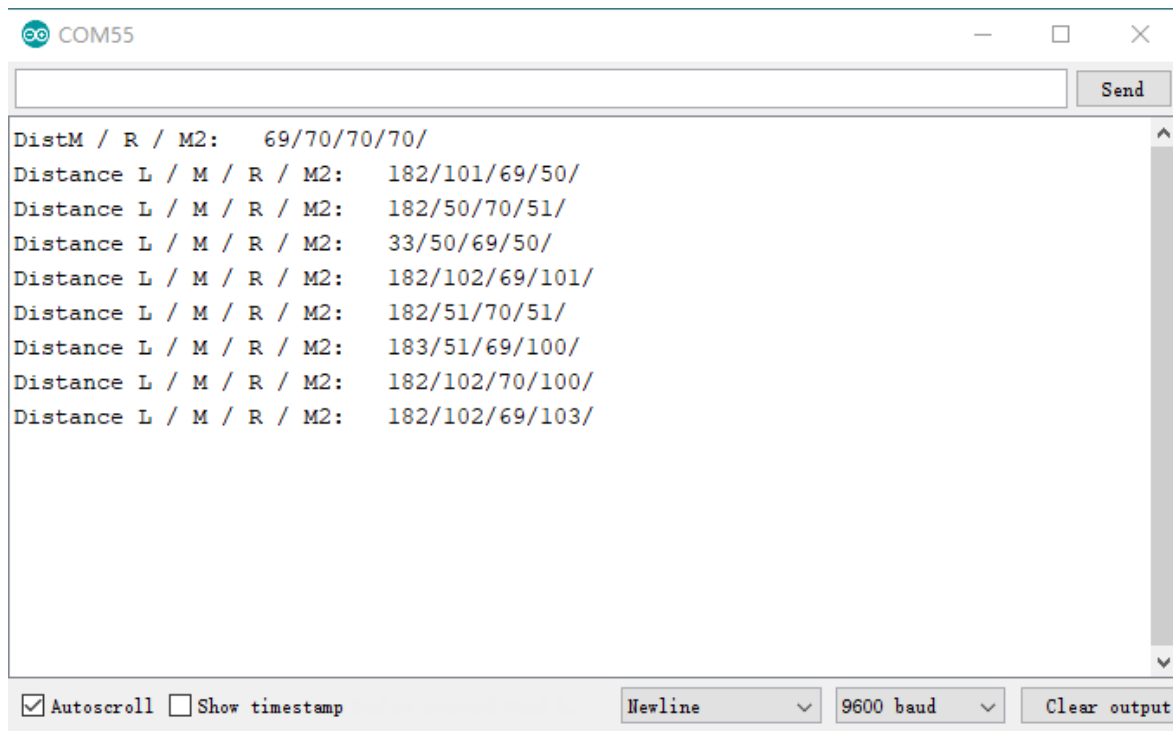
Then turn on the power of car. Then open the serial monitor.



```

File Edit Sketch Tools Help
Arduino Uno
02.2_Ultrasonic_Ranging.ino
1  /*****
2  * Product   : Freenove 4WD Car for UNO
3  * Description : Ultrasonic ranging and servo.
4  * Author    : www.freenove.com
5  * Modification: 2019/08/05
6  *****/
7  #include "Servo.h"          //include servo library
8
9  #define PIN_SERVO          2      //define servo pin
10
11 #define PIN_SONIC_TRIG      7      //define Trig pin
12 #define PIN_SONIC_ECHO      8      //define Echo pin
13
14 #define MAX_DISTANCE        1000   //cm
15 #define SONIC_TIMEOUT       (MAX_DISTANCE*60) // calculate timeout
16 #define SOUND_VELOCITY      340    //soundVelocity: 340m/s
  
```

Then you will see servo sweep among 45°, 90°, 135°. And ultrasonic module detects different distance at different angles. All the distance values are printed below:



```

COM55
Send
DistM / R / M2: 69/70/70/70/
Distance L / M / R / M2: 182/101/69/50/
Distance L / M / R / M2: 182/50/70/51/
Distance L / M / R / M2: 33/50/69/50/
Distance L / M / R / M2: 182/102/69/101/
Distance L / M / R / M2: 182/51/70/51/
Distance L / M / R / M2: 183/51/69/100/
Distance L / M / R / M2: 182/102/70/100/
Distance L / M / R / M2: 182/102/69/103/
Autoscroll Show timestamp Newline 9600 baud Clear output
  
```

The code is below:

```
1  #include "Servo.h"           //include servo library
2  #define PIN_SERVO           2       //define servo pin
3  #define PIN_SONIC_TRIG      7       //define Trig pin
4  #define PIN_SONIC_ECHO      8       //define Echo pin
5  #define MAX_DISTANCE        1000    //cm
6  #define SONIC_TIMEOUT       (MAX_DISTANCE*60) // calculate timeout
7  #define SOUND_VELOCITY      340     //sound Velocity: 340m/s
8
9  Servo servo;                 //create servo object
10 char servoOffset = 0;        //change the value to Calibrate servo
11 u8 distance[4];              //define an array with type u8(same to unsigned char)
12
13 void setup() {
14     Serial.begin(9600);
15     pinMode(PIN_SONIC_TRIG, OUTPUT); // set trigPin to output mode
16     pinMode(PIN_SONIC_ECHO, INPUT);  // set echoPin to input mode
17     servo.attach(PIN_SERVO);         //initialize servo
18     servo.write(90 + servoOffset);   // change servoOffset to Calibrate servo
19 }
20 void loop() {
21     servo.write(45);
22     delay(1000);
23     distance[0] = getSonar(); //get ultrasonic value and save it into distance[0]
24
25     servo.write(90);
26     delay(1000);
27     distance[1] = getSonar();
28
29     servo.write(135);
30     delay(1000);
31     distance[2] = getSonar();
32
33     servo.write(90);
34     delay(1000);
35     distance[3] = getSonar();
36
37     Serial.print("Distance L / M / R / M2:  "); //Left/Middle/Right/Middle2
38     for (int i = 0; i < 4; i++) {
39         Serial.print(distance[i]); //print ultrasonic in 45° , 90° , 135° , 90°
40         Serial.print("/");
41     }
42     Serial.print('\n'); //next content will be printed in new line
43 }
```

```

44
45 float getSonar() {
46     unsigned long pingTime;
47     float distance;
48     digitalWrite(PIN_SONIC_TRIG, HIGH); // make trigPin output high level lasting for 10 μs to
49     trigger HC_SR04,
50     delayMicroseconds(10);
51     digitalWrite(PIN_SONIC_TRIG, LOW);
52     pingTime = pulseIn(PIN_SONIC_ECHO, HIGH, SONIC_TIMEOUT); // Wait HC-SR04 returning to the
53     high level and measure out this waiting time
54     if (pingTime != 0)
55         distance = (float)pingTime * SOUND_VELOCITY / 2 / 10000; // calculate the distance
56     according to the time
57     else
58         distance = MAX_DISTANCE;
59     return distance; // return the distance value
60 }

```

```

#define MAX_DISTANCE 1000 //cm
#define SONIC_TIMEOUT (MAX_DISTANCE*60) // calculate timeout
#define SOUND_VELOCITY 340 //sound Velocity: 340m/s

```

First calculate t for max distance.

$340(t/1000000)/2 = \text{MAX_DISTANCE} / 100$ $t = 58.8 * \text{MAX_DISTANCE}$ (Unit of t is μs)

We set timeout to $\text{MAX_DISTANCE} * 60$, a little larger, since the maximum distance of ultrasonic module is much larger.

```
pingTime = pulseIn(PIN_SONIC_ECHO, HIGH, SONIC_TIMEOUT); // Wait HC-SR04 returning to the high level
```

If time of high level lasting time of echo is larger than `SONIC_TIMEOUT`, it will return 0. Then pingTime=0.

pulseIn(pin, value, timeout)

pin: the number of the Arduino pin on which you want to read the pulse.

value: type of pulse to be read: either HIGH or LOW.

timeout (optional): the number of microseconds to wait for the pulse to start; default is one second.

For more details, please refer to:

<https://www.arduino.cc/reference/en/language/functions/advanced-io/pulsein/>

Array

An array is a collection of variables that are accessed with an index number.

Define an array in Arduino.

Data type **Array name** **[Number of elements]**

u8 distance[3]; define an array named distance, its data type is u8, which has 3 elements.

The index starts from 0.

u8= **unsigned char** Its range is $0 \sim 2^8$, namely 0~255.

Range of u16 is 0~65535.

For more details about array, please refer to:

<https://www.arduino.cc/reference/en/language/variables/data-types/array/>

2.3 Automatic Obstacle Avoidance Car

Code

02.3.1_Automatic_Obstacle_Avoidance

You need remove Bluetooth Module first when you upload code.

Upload code in Sketches\02.3.1_Automatic_Obstacle_Avoidance.

Then disconnect USB cable. Turn on the power of car. And put the car in a place with enough space to move.

The code is below:

```
1  #include <Servo.h>
2  #define PIN_SERVO          2
3  #define MOTOR_DIRECTION    0 //If the direction is reversed, change 0 to 1
4  #define PIN_DIRECTION_LEFT 4
5  #define PIN_DIRECTION_RIGHT 3
6  #define PIN_MOTOR_PWM_LEFT 6
7  #define PIN_MOTOR_PWM_RIGHT 5
8
9  #define PIN_SONIC_TRIG      7
10 #define PIN_SONIC_ECHO      8
11
12 #define PIN_BATTERY          A0
13
14 #define OBSTACLE_DISTANCE    40
15 #define OBSTACLE_DISTANCE_LOW 15
16
17 #define MAX_DISTANCE         1000
18 #define SONIC_TIMEOUT        (MAX_DISTANCE*60)
19 #define SOUND_VELOCITY       340 //soundVelocity: 340m/s
20
21 Servo servo;
22 char servoOffset = 0;
23 int speedOffset;//batteryVoltageCompensationToSpeed
24
25 void setup() {
26     pinMode(PIN_DIRECTION_LEFT, OUTPUT);
27     pinMode(PIN_MOTOR_PWM_LEFT, OUTPUT);
28     pinMode(PIN_DIRECTION_RIGHT, OUTPUT);
29     pinMode(PIN_MOTOR_PWM_RIGHT, OUTPUT);
30
31     pinMode(PIN_SONIC_TRIG, OUTPUT); // set trigPin to output mode
32     pinMode(PIN_SONIC_ECHO, INPUT); // set echoPin to input mode
33     servo.attach(PIN_SERVO);
34     calculateVoltageCompensation();
```

```

35 }
36
37 void loop() {
38     updateAutomaticObstacleAvoidance();
39 }
40
41 void updateAutomaticObstacleAvoidance() {
42     int distance[3], tempDistance[3][5], sumDisntance;
43     static u8 leftToRight = 0, servoAngle = 0, lastServoAngle = 0; //
44     const u8 scanAngle[2][3] = { {150, 90, 30}, {30, 90, 150} };
45
46     for (int i = 0; i < 3; i++)
47     {
48         servoAngle = scanAngle[leftToRight][i];
49         servo.write(servoAngle);
50         if (lastServoAngle != servoAngle) {
51             delay(130);
52         }
53         lastServoAngle = servoAngle;
54         for (int j = 0; j < 5; j++) {
55             tempDistance[i][j] = getSonar();
56             delayMicroseconds(2 * SONIC_TIMEOUT);
57             sumDisntance += tempDistance[i][j];
58         }
59         if (leftToRight == 0) {
60             distance[i] = sumDisntance / 5;
61         }
62         else {
63             distance[2 - i] = sumDisntance / 5;
64         }
65         sumDisntance = 0;
66     }
67     leftToRight = (leftToRight + 1) % 2;
68
69     if (distance[1] < OBSTACLE_DISTANCE) { //Too little distance ahead
70         if (distance[0] > distance[2] && distance[0] > OBSTACLE_DISTANCE) { //Left distance is
71             greater than right distance
72             motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
73             delay(100);
74             motorRun(-(150 + speedOffset), (150 + speedOffset));
75         }
76         else if (distance[0] < distance[2] && distance[2] > OBSTACLE_DISTANCE)
77         { //Right distance is greater than left distance
78             motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back

```

```

79     delay(100);
80     motorRun((150 + speedOffset), -(150 + speedOffset));
81 }
82 else {                                     //Get into the dead corner, move back, then turn.
83     motorRun(-(150 + speedOffset), -(150 + speedOffset));
84     delay(100);
85     motorRun(-(150 + speedOffset), (150 + speedOffset));
86 }
87 }
88 else {                                     //No obstacles ahead
89     if (distance[0] < OBSTACLE_DISTANCE_LOW) { //Obstacles on the left front.
90         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
91         delay(100);
92         motorRun((180 + speedOffset), (50 + speedOffset));
93     }
94     else if (distance[2] < OBSTACLE_DISTANCE_LOW) { //Obstacles on the right front.
95         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
96         delay(100);
97         motorRun((50 + speedOffset), (180 + speedOffset));
98     }
99     else {                                     //Cruising
100         motorRun((80 + speedOffset), (80 + speedOffset));
101     }
102 }
103 }
104
105 float getSonar() {
106     unsigned long pingTime;
107     float distance;
108     digitalWrite(PIN_SONIC_TRIG, HIGH); // make trigPin output high level lasting for 10 μs to
109     trigger HC_SR04,
110     delayMicroseconds(10);
111     digitalWrite(PIN_SONIC_TRIG, LOW);
112     pingTime = pulseIn(PIN_SONIC_ECHO, HIGH, SONIC_TIMEOUT); // Wait HC-SR04 returning to the
113     high level and measure out this waiting time
114     if (pingTime != 0)
115         distance = (float)pingTime * SOUND_VELOCITY / 2 / 10000; // calculate the distance
116     according to the time
117     else
118         distance = MAX_DISTANCE;
119     return distance; // return the distance value
120 }
121
122 void calculateVoltageCompensation() {

```

```

123     float voltageOffset = 8.4 - getBatteryVoltage();
124     speedOffset = voltageOffset * 20;
125 }
126
127 void motorRun(int speedl, int speedr) {
128     int dirL = 0, dirR = 0;
129     if (speedl > 0) {
130         dirL = 0 ^ MOTOR_DIRECTION;
131     }
132     else {
133         dirL = 1 ^ MOTOR_DIRECTION;
134         speedl = -speedl;
135     }
136     if (speedr > 0) {
137         dirR = 1 ^ MOTOR_DIRECTION;
138     }
139     else {
140         dirR = 0 ^ MOTOR_DIRECTION;
141         speedr = -speedr;
142     }
143     digitalWrite(PIN_DIRECTION_LEFT, dirL);
144     digitalWrite(PIN_DIRECTION_RIGHT, dirR);
145     analogWrite(PIN_MOTOR_PWM_LEFT, speedl);
146     analogWrite(PIN_MOTOR_PWM_RIGHT, speedr);
147 }
148
149
150 float getBatteryVoltage() {
151     pinMode(PIN_BATTERY, INPUT);
152     int batteryADC = analogRead(PIN_BATTERY);
153     float batteryVoltage = batteryADC / 1023.0 * 5.0 * 4;
154     return batteryVoltage;
155 }

```

We use `analogWrite(PIN_MOTOR, PWM value)` to make motor work. The PWM value represents different speeds when the voltage of battery changes. So we set a `speedOffset` to compensate the difference.

```

1 void calculateVoltageCompensation() {
2     float voltageOffset = 8.4 - getBatteryVoltage();
3     speedOffset = voltageOffset * 20;
4 }

```

20 is tested value. You can try to test what is the most proper value.

We need control servo to 150°, 90°, 30°, 90°, 150°...

Get 5 ultrasonic module values (distance) at one angle. Then calculate the average distance.

```

1  int distance[3], tempDistance[3][5], sumDisntance;
2  static u8 leftToRight = 0, servoAngle = 0, lastServoAngle = 0;  //
3  const u8 scanAngle[2][3] = { {150, 90, 30}, {30, 90, 150} };
4  for (int i = 0; i < 3; i++)
5  {
6      servoAngle = scanAngle[leftToRight][i];
7      servo.write(servoAngle);
8      if (lastServoAngle != servoAngle) {
9          delay(130);
10     }
11     lastServoAngle = servoAngle;
12     for (int j = 0; j < 5; j++) {
13         tempDistance[i][j] = getSonar();
14         delayMicroseconds(2 * SONIC_TIMEOUT);
15         sumDisntance += tempDistance[i][j];
16     }
17     if (leftToRight == 0) {
18         distance[i] = sumDisntance / 5;
19     }
20     else {
21         distance[2 - i] = sumDisntance / 5;
22     }
23     sumDisntance = 0;
24 }
25 leftToRight = (leftToRight + 1) % 2;
26

```

Then make the car react according to the distances above. About its logic, please refer to flow chart.

```

1  //make the car react according to the distances
2  if (distance[1] < OBSTACLE_DISTANCE) {          //obstacle in front
3      if (distance[0] > distance[2] && distance[0] > OBSTACLE_DISTANCE) {    //Left distance is
4  greater than right distance
5      motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back a little
6      delay(100);
7      motorRun(-(150 + speedOffset), (150 + speedOffset));
8      delay(50);
9      }
10     else if (distance[0] < distance[2] && distance[2] > OBSTACLE_DISTANCE)
11     {
12         //Right distance is greater than left distance
13         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back a little
14         delay(100);
15         motorRun((150 + speedOffset), -(150 + speedOffset)); //Turn right
16         delay(50);
17     }
18 }

```

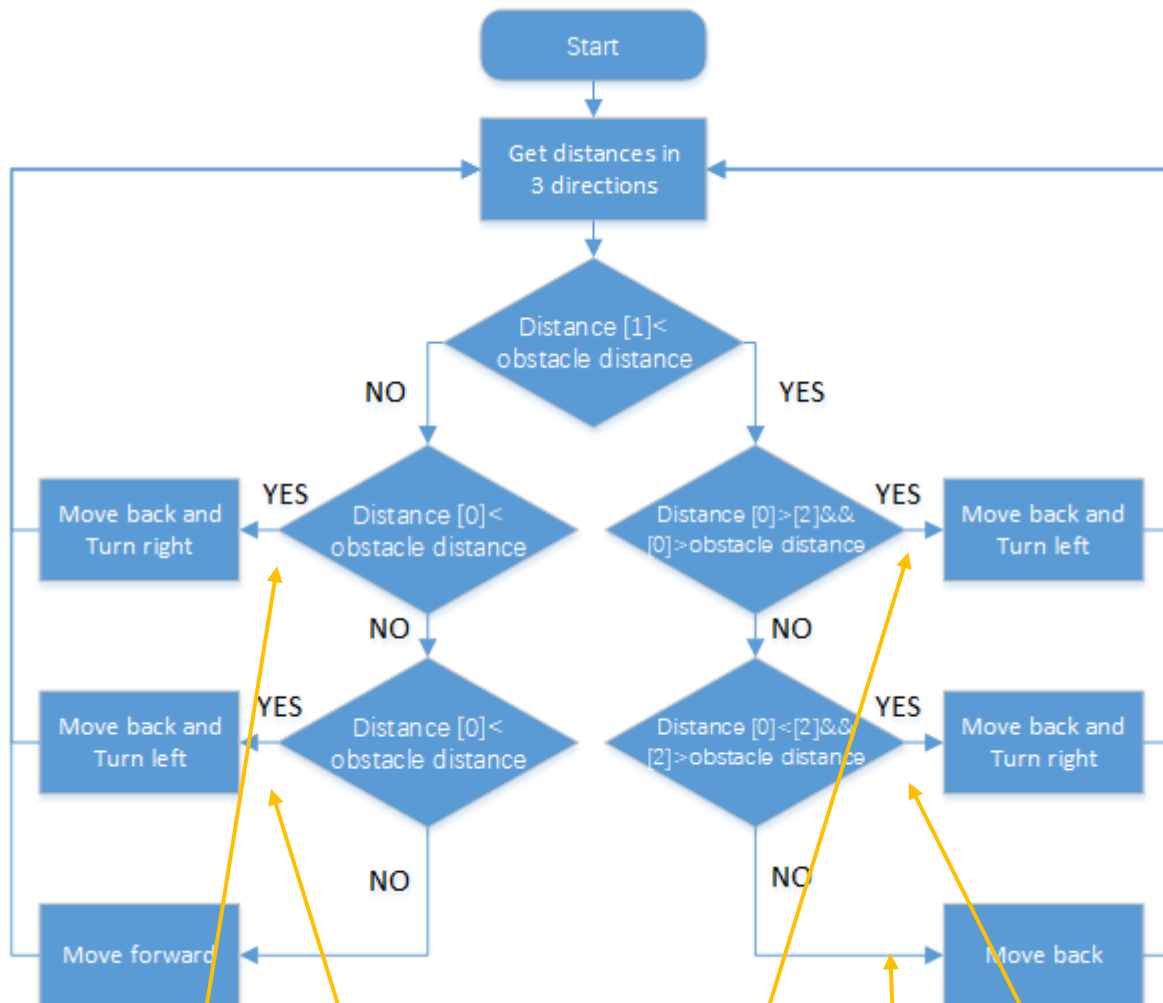
```
17     else {                                     //Get into the dead corner, move back a little, then spin.
18         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
19         delay(100);
20         motorRun(-(150 + speedOffset), (150 + speedOffset)); //Turn left
21         delay(50);
22     }
23 }
24 else {                                         //No obstacles ahead
25     if (distance[0] < OBSTACLE_DISTANCE_LOW) { //Obstacles on the left front.
26         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
27         delay(100);
28         motorRun((180 + speedOffset), (50 + speedOffset)); //Turn right with large radius
29         delay(50);
30     }
31     else if (distance[2] < OBSTACLE_DISTANCE_LOW) { //Obstacles on the right front.
32         motorRun(-(150 + speedOffset), -(150 + speedOffset)); //Move back
33         delay(100);
34         motorRun((50 + speedOffset), (180 + speedOffset)); //Turn left with large radius
35         delay(50);
36     }
37     else {                                     //Cruising, no obstacle closed
38         motorRun((80 + speedOffset), (80 + speedOffset));
39     }
40 }
41 }
42 }
```

Code logic is as flow chart below:

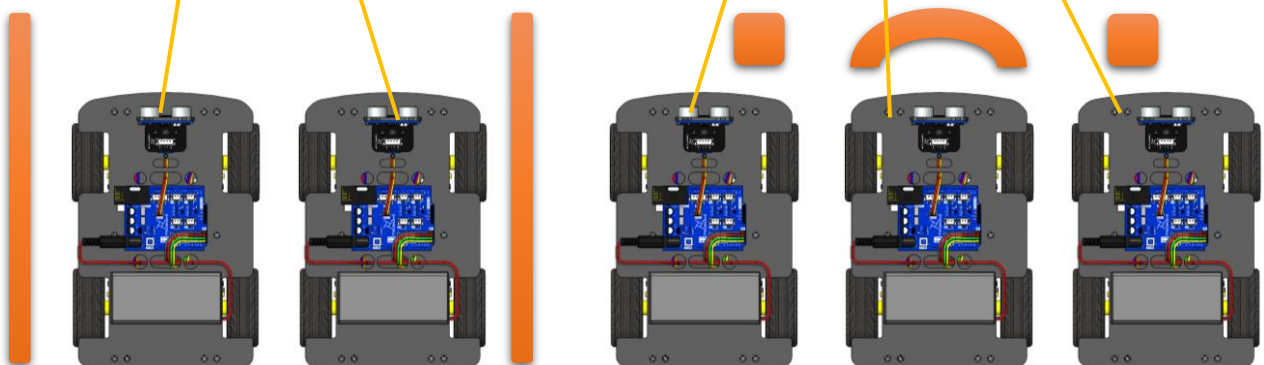
Distance[0] is the left distance to obstacle.

Distance[1] is the center distance to obstacle.

Distance[2] is the right distance to obstacle.



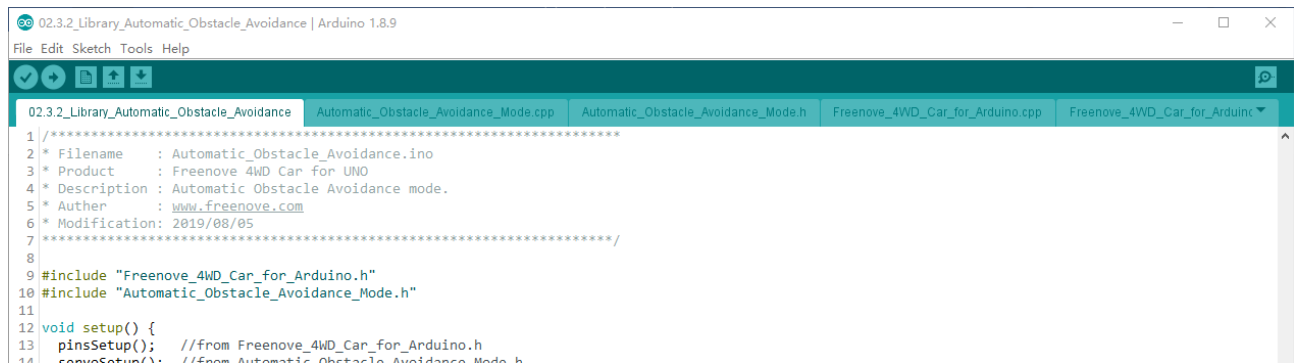
This project consider following



02.3.2_Library_Automatic_Obstacle_Avoidance

Code in sketches\02.3.2_Library_Automatic_Obstacle_Avoidance includes a library

The code function and content is similar to previous section. There are little difference. Previous code is more simple.



```

02.3.2_Library_Automatic_Obstacle_Avoidance | Arduino 1.8.9
File Edit Sketch Tools Help

02.3.2_Library_Automatic_Obstacle_Avoidance Automatic_Obstacle_Avoidance_Mode.cpp Automatic_Obstacle_Avoidance_Mode.h Freenove_4WD_Car_for_Arduino.cpp Freenove_4WD_Car_for_Arduino.h

1 /*****
2  * Filename   : Automatic_Obstacle_Avoidance.ino
3  * Product    : Freenove 4WD Car for UNO
4  * Description : Automatic Obstacle Avoidance mode.
5  * Author     : www.freenove.com
6  * Modification: 2019/08/05
7  *****/
8
9 #include "Freenove_4WD_Car_for_Arduino.h"
10 #include "Automatic_Obstacle_Avoidance_Mode.h"
11
12 void setup() {
13   pinsSetup(); //from Freenove_4WD_Car_for_Arduino.h
14   servoSetup(); //from Automatic_Obstacle_Avoidance_Mode.h

```

There are 4 labels.

Freenove_4WD_Car_for_Arduino.h has introduced in section [01.4.2 Library Integrate Functions](#)

The main label is **02.3.2_Library_Automatic_Obstacle_Avoidance.ino**

Its code is below.

```

1 #include "Freenove_4WD_Car_for_Arduino.h"
2 #include "Automatic_Obstacle_Avoidance_Mode.h"
3
4 void setup() {
5   pinsSetup(); //from Freenove_4WD_Car_for_Arduino.h
6   servoSetup(); //from Automatic_Obstacle_Avoidance_Mode.h
7   oa_CalculateVoltageCompensation(); //Automatic_Obstacle_Avoidance_Mode.h
8 }
9
10 void loop() {
11   updateAutomaticObstacleAvoidance(); //Automatic_Obstacle_Avoidance_Mode.h
12 }

```

The code is very simple. The function from other labels.

Code in label **Automatic_Obstacle_Avoidance_Mode.h**

```

1 #ifndef _AUTOMATIC_OBSTACLE_AVOIDANCE_h
2 #define _AUTOMATIC_OBSTACLE_AVOIDANCE_h
3
4 #if defined(ARDUINO) && ARDUINO >= 100
5 #include "arduino.h"
6 #else
7 #include "WProgram.h"
8 #endif
9 #include <EEPROM.h>
10 #include "Servo.h"
11 #include "Freenove_4WD_Car_for_Arduino.h"
12

```

```

13 #define OA_SERVO_CENTER                (90)
14 #define OA_SCAN_ANGLE_INTERVAL 50
15 #define OA_SCAN_ANGLE_MIN      (OA_SERVO_CENTER - OA_SCAN_ANGLE_INTERVAL)
16 #define OA_SCAN_ANGLE_MAX      (OA_SERVO_CENTER + OA_SCAN_ANGLE_INTERVAL)
17 #define OA_WAITTING_SERVO_TIME 130
18
19 #define OA_CRUISE_SPEED          (110 + oa_VoltageCompensationToSpeed)
20
21 #define OA_ROTATY_SPEED_LOW      (120 + oa_VoltageCompensationToSpeed)
22 #define OA_ROTATY_SPEED_NORMAL  (150 + oa_VoltageCompensationToSpeed)
23 #define OA_ROTATY_SPEED_HIGH    (180 + oa_VoltageCompensationToSpeed)
24
25 #define OA_TURN_SPEED_LV4        (180 + oa_VoltageCompensationToSpeed)
26 #define OA_TURN_SPEED_LV1       (50 + oa_VoltageCompensationToSpeed )
27
28 #define OA_BACK_SPEED_LOW        (110 + oa_VoltageCompensationToSpeed)
29 #define OA_BACK_SPEED_NORMAL     (150 + oa_VoltageCompensationToSpeed)
30 #define OA_BACK_SPEED_HIGH       (180 + oa_VoltageCompensationToSpeed)
31
32 #define OA_OBSTACLE_DISTANCE     40
33 #define OA_OBSTACLE_DISTANCE_LOW 15
34
35 #define OA_SPEED_OFFSET_PER_V    35
36
37 #define OA_SERVO_OFFSET_ADDR_IN_EEPROM 0
38
39 #define MAX_DISTANCE             1000
40 #define SONIC_TIMEOUT            (MAX_DISTANCE*60)
41 #define SOUND_VELOCITY           340    //soundVelocity: 340m/s
42
43
44 extern Servo servo;
45 extern int servoOffset;
46
47 void servoSetup(); //initialize the servo
48 void setServoOffset(int offset); //Set servo offset
49 void writeServo(u8 n); //set servo to an angle, offset has been considered
50 void writeServoOffsetToEEPROM();
51 void getServoOffsetFromEEPROM();
52
53 float getSonar();
54 void oa_CalculateVoltageCompensation(); // Calculate Voltage Compensation
55 void updateAutomaticObstacleAvoidance();
56 #endif

```

.h file used to define pins, variables and functions.

You can get information of what you can use from .h file.

The detailed implement code of the functions is included in .cpp file.

```

1  #include "Automatic_Obstacle_Avoidance_Mode.h"
2
3  Servo servo;
4  char servoOffset = 0;
5  int oa_VoltageCompensationToSpeed;
6
7  void servoSetup() {
8      getServoOffsetFromEEPROM();
9      servo.attach(PIN_SERVO);
10     servo.write(90 + servoOffset);
11 }
12
13 void setServoOffset(char offset) {
14     servoOffset = offset = constrain(offset, -100, 100);
15     servo.write(90 + offset);
16 }
17
18 void writeServo(u8 n)
19 {
20     servo.write(90 + servoOffset);
21 }
22
23 void writeServoOffsetToEEPROM() {
24     servo.write(90 + servoOffset);
25     EEPROM.write(OA_SERVO_OFFSET_ADDR_IN_EEPROM, servoOffset);
26 }
27
28 void getServoOffsetFromEEPROM() {
29     servoOffset = EEPROM.read(OA_SERVO_OFFSET_ADDR_IN_EEPROM);
30     servoOffset = constrain(servoOffset, -10, 10);
31 }
32
33 float getSonar() {
34     unsigned long pingTime;
35     float distance;
36     digitalWrite(PIN_SONIC_TRIG, HIGH); // make trigPin output high level lasting for 10 μs
37     to trigger HC_SR04,
38     delayMicroseconds(10);
39     digitalWrite(PIN_SONIC_TRIG, LOW);
40     pingTime = pulseIn(PIN_SONIC_ECHO, HIGH, SONIC_TIMEOUT); // Wait HC-SR04 returning to the

```

```
41 high level and measure out this waiting time
42     if (pingTime != 0)
43         distance = (float)pingTime * SOUND_VELOCITY / 2 / 10000; // calculate the distance
44 according to the time
45     else
46         distance = MAX_DISTANCE;
47     return distance; // return the distance value
48 }
49
50 void oa_CalculateVoltageCompensation() {
51     getBatteryVoltage();
52     float voltageOffset = BAT_VOL_STANDARD - batteryVoltage;
53     oa_VoltageCompensationToSpeed = voltageOffset * OA_SPEED_OFFSET_PER_V;
54 }
55
56 void updateAutomaticObstacleAvoidance() {
57     int distance[3], tempDistance[3][5], sumDisntance;
58     static u8 cnt = 0, servoAngle = 0, lastServoAngle = 0; //
59     if (cnt == 0) {
60         for (int i = 0; i < 3; i++) {
61             servoAngle = OA_SCAN_ANGLE_MAX - i * OA_SCAN_ANGLE_INTERVAL + servoOffset;
62             servo.write(servoAngle);
63             if (lastServoAngle != servoAngle) {
64                 delay(OA_WAITTING_SERVO_TIME);
65             }
66             lastServoAngle = servoAngle;
67             for (int j = 0; j < 5; j++) {
68                 tempDistance[i][j] = getSonar();
69                 delayMicroseconds(2 * SONIC_TIMEOUT);
70                 sumDisntance += tempDistance[i][j];
71             }
72             distance[i] = sumDisntance / 5;
73             sumDisntance = 0;
74         }
75         cnt++;
76     }
77     else {
78         for (int i = 2; i > 0; i--) {
79             servoAngle = OA_SCAN_ANGLE_MAX - i * OA_SCAN_ANGLE_INTERVAL + servoOffset;
80             servo.write(servoAngle);
81             if (lastServoAngle != servoAngle) {
82                 delay(OA_WAITTING_SERVO_TIME);
83             }
84             lastServoAngle = servoAngle;
```

```

85         for (int j = 0; j < 5; j++) {
86             tempDistance[i][j] = getSonar();
87             delayMicroseconds(2 * SONIC_TIMEOUT);
88             sumDistance += tempDistance[i][j];
89         }
90         distance[i] = sumDistance / 5;
91         sumDistance = 0;
92     }
93     cnt = 0;
94 }
95
96 if (distance[1] < OA_OBSTACLE_DISTANCE) {                //Too little distance ahead
97     if (distance[0] > OA_OBSTACLE_DISTANCE || distance[2] > OA_OBSTACLE_DISTANCE) {
98         motorRun(-OA_BACK_SPEED_LOW, -OA_BACK_SPEED_LOW);    //Move back a little
99         delay(100);
100        if (distance[0] > distance[2]) {                //Left distance is greater than
101    right distance
102            motorRun(-OA_ROTATY_SPEED_LOW, OA_ROTATY_SPEED_LOW);
103        }
104        else {                //Right distance is greater than
105    left distance
106            motorRun(OA_ROTATY_SPEED_LOW, -OA_ROTATY_SPEED_LOW);
107        }
108    }
109    else {                //Get into the dead corner, move
110    back a little, then spin.
111        motorRun(-OA_BACK_SPEED_HIGH, -OA_BACK_SPEED_HIGH);
112        delay(100);
113        motorRun(-OA_ROTATY_SPEED_NORMAL, OA_ROTATY_SPEED_NORMAL);
114    }
115 }
116 // following code is different from last section. Last section is more simple
117 else {                //No obstacles ahead
118     if (distance[0] < OA_OBSTACLE_DISTANCE) {                //Obstacles on the left front.
119         if (distance[0] < OA_OBSTACLE_DISTANCE_LOW) {        //Very close to the left front
120    obstacle.
121            motorRun(-OA_BACK_SPEED_LOW, -OA_BACK_SPEED_LOW);    //Move back
122            delay(100);
123        }
124        motorRun(OA_TURN_SPEED_LV4, OA_TURN_SPEED_LV1);
125    }
126    else if (distance[2] < OA_OBSTACLE_DISTANCE) {                //Obstacles on the right
127    front.
128        if (distance[2] < OA_OBSTACLE_DISTANCE_LOW) {        //Very close to the right

```

```

129 front obstacle.
130         motorRun(-OA_BACK_SPEED_LOW, -OA_BACK_SPEED_LOW);    //Move back
131         delay(100);
132     }
133     motorRun(OA_TURN_SPEED_LV1, OA_TURN_SPEED_LV4);
134 }
135 else {                                     //Cruising
136     motorRun(OA_CRUISE_SPEED, OA_CRUISE_SPEED);
137 }
138 }
139 }

```

Since the servo may not be installed to 90°, we need adjust it. But we don't want to adjust it every time, so we can save the servoOffset in a place and next time, we can use it directly.. The microcontroller on the Arduino and Genuino boards have 1024 bytes of EEPROM.

It is useful when we use remote control to calibrate the servo.

```

1 void writeServoOffsetToEEPROM() {
2     servo.write(90 + servoOffset);
3     EEPROM.write(OA_SERVO_OFFSET_ADDR_IN_EEPROM, servoOffset);
4 }
5
6 void getServoOffsetFromEEPROM() {
7     servoOffset = EEPROM.read(OA_SERVO_OFFSET_ADDR_IN_EEPROM);
8     servoOffset = constrain(servoOffset, -10, 10);
9 }

```

EEPROM

The microcontroller on the Arduino and Genuino boards have 1024 bytes of EEPROM.

EEPROM.write(address, val); write val to address

value = EEPROM.read(address); read the data saved in address

for example,

EEPROM.write(0, 3);

EEPROM.write(1, 5);

EEPROM.write(2, 8);

Value0= EEPROM.read(0); then Value0 = 3

Value1 = EEPROM.read(2); then Value2 = 8

For more information, refer to <https://www.arduino.cc/en/Reference/EEPROM>

Other code logic is similar to previous section.

Chapter 3 Line tracking

In this chapter, we will introduce line-tracking sensor and make a line-tracking car.

If you have any concerns, please feel free to contact us via support@freenove.com

3.1 Line tracking sensor

There are three Reflective Optical Sensors on this car. When the infrared light emitted by infrared diode shines on the surface of different objects, the sensor will receive light with different intensities after reflection.

As we know, black objects absorb light better. So when black lines are drawn on the white plane, the sensor can detect the difference. The sensor can also be called Line Tracking Sensor.

Warning:

Reflective Optical Sensor (including Line Tracking Sensor) should be avoided using in environment with infrared interference, like sunlight. Sunlight contains a lot of invisible light such as infrared and ultraviolet. Under environment with intense sunlight, Reflective Optical Sensor cannot work normally.

The following table shows the values of all cases when three Tracking Sensors detect objects of different colors. Among them, black objects or no objects were detected to represent 1, and white objects were detected to represent 0.

Left	Middle	Right	Value(binary)	Value(decimal)
0	0	0	000	0
0	0	1	001	1
0	1	0	010	2
0	1	1	011	3
1	0	0	100	4
1	0	1	101	5
1	1	0	110	6
1	1	1	111	7

Code

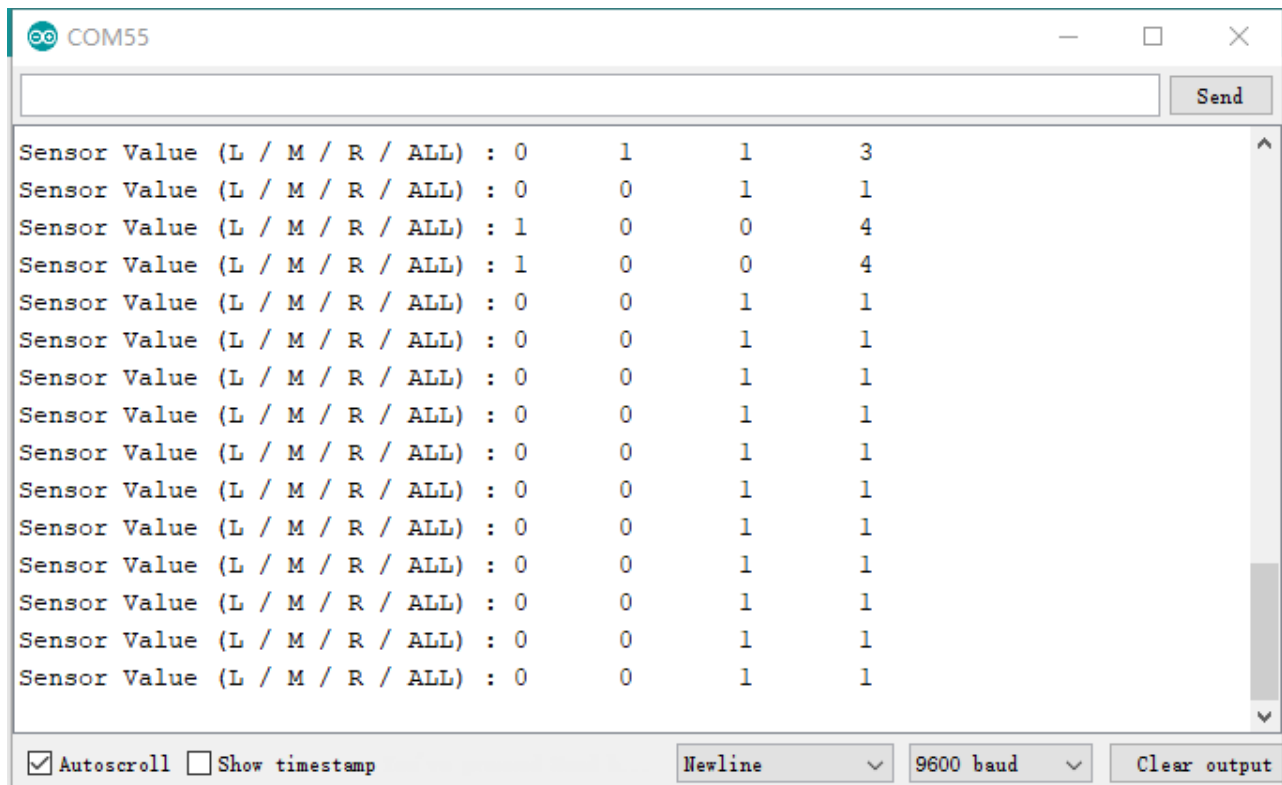
03.1_Tracking_Sensor

You need remove Bluetooth Module Module first when you upload code.

Upload code in Sketches\03.1_Tracking_Sensor, turn on the power, and open the serial monitor. Put something black under the sensor. Move it and you will see different LEDs light up.



The monitor is shown below:



The code is below:

```

1  #define PIN_TRACKING_LEFT  A1
2  #define PIN_TRACKING_CENTER A2
3  #define PIN_TRACKING_RIGHT A3
4
5  u8 sensorValue[4]; //define a array, u8 = unsigned char, 0~255.
6
7  void setup() {
8      Serial.begin(9600); //set baud rate
9      pinMode(PIN_TRACKING_LEFT, INPUT); //
10     pinMode(PIN_TRACKING_RIGHT, INPUT); //
11     pinMode(PIN_TRACKING_CENTER, INPUT); //
12 }
13

```

```

14 void loop() {
15     sensorValue[0] = digitalRead(PIN_TRACKING_LEFT);
16     sensorValue[1] = digitalRead(PIN_TRACKING_CENTER);
17     sensorValue[2] = digitalRead(PIN_TRACKING_RIGHT);
18     sensorValue[3] = sensorValue[0] << 2 | sensorValue[1] << 1 | sensorValue[2];
19     Serial.print("Sensor Value (L / M / R / ALL) : ");
20     for (int i = 0; i < 4; i++) {
21         Serial.print(sensorValue[i]);
22         Serial.print(' '); //means Tab
23     }
24     Serial.print('\n'); //means new line
25     delay(500);
26 }
27

```

Bitwise Operators

There are some Bitwise Operators.

<< (bitshift left)

If sensorValue[0] =1, sensorValue[1]= 1, sensorValue[2]=1

sensorValue[0] << 2 then sensorValue[0]=100, (Binary), namely 4(Decimal)

sensorValue[1] << 1 then sensorValue[0]=010, (Binary), namely 2(Decimal)

sensorValue[2]=001 (Binary)

| (bitwise or)

The code turns to: 100 | 010 | 001 =111 (Binary), namely 7(Decimal)

>> (bitshift right)

& (bitwise and)

^ (bitwise xor)

~ (bitwise not)

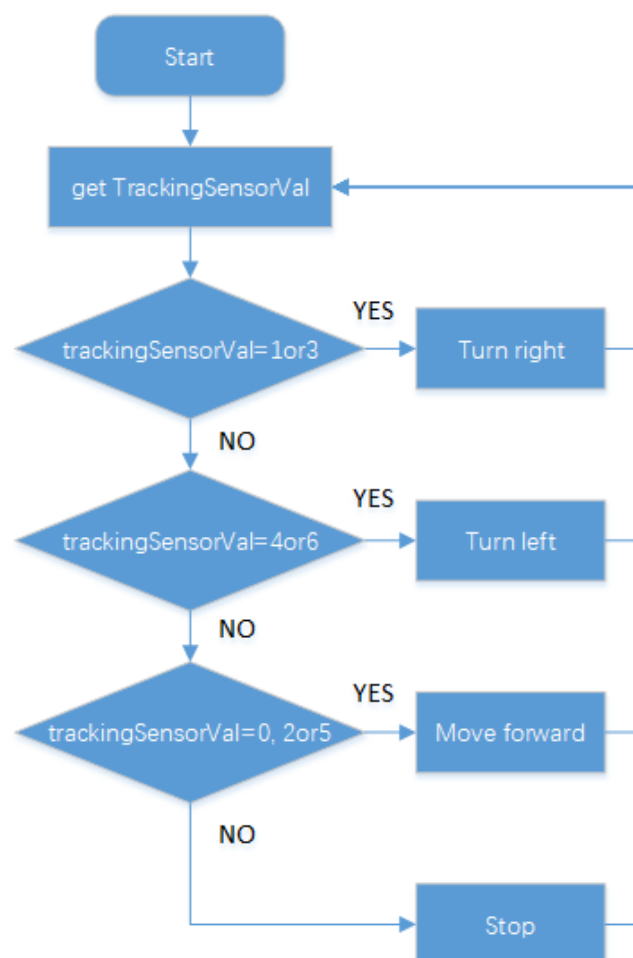
For more information, please refer to: <https://www.arduino.cc/reference/>

3.2 Line tracking car

The car will make different actions according to the value transmitted by the line-tracking sensor.
When

Left	Middle	Right	Value(binary)	Value(decimal)	Action
0	0	0	000	0	move forward
0	0	1	001	1	Turn Right
0	1	0	010	2	Move Forward
0	1	1	011	3	Turn Right
1	0	0	100	4	Turn Left
1	0	1	101	5	Move Forward
1	1	0	110	6	Turn Left
1	1	1	111	7	Stop

Flow chart of line tracking car is as below:

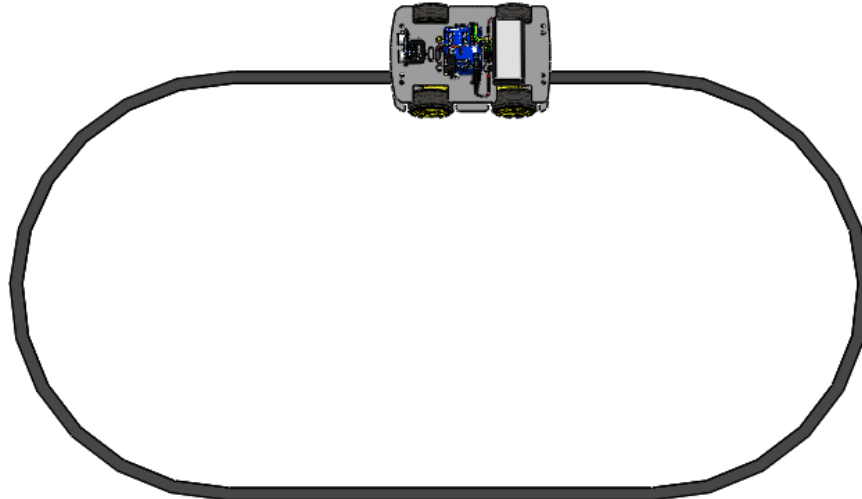


Code

03.2_Automatic_Tracking_Line

You need remove Bluetooth Module first when you upload code.

Upload code in **Sketches\03.3_One_Code_Automatic_Tracking_Line**. Turn on the power. Use a black tape to build a line and then put your car on it as below.



You can also choose 03.2_Automatic_Tracking_Line. The code is same.

The code is below.

```

1  #include "Freenove_4WD_Car_for_Arduino.h"
2
3  #define TK_STOP_SPEED          0
4  #define TK_FORWARD_SPEED      (90 + tk_VoltageCompensationToSpeed )
5
6  //define different speed levels
7  #define TK_TURN_SPEED_LV4     (180 + tk_VoltageCompensationToSpeed )
8  #define TK_TURN_SPEED_LV3     (150 + tk_VoltageCompensationToSpeed )
9  #define TK_TURN_SPEED_LV2     (-140 + tk_VoltageCompensationToSpeed )
10 #define TK_TURN_SPEED_LV1     (-160 + tk_VoltageCompensationToSpeed )
11
12 int tk_VoltageCompensationToSpeed; //define Voltage Speed Compensation
13
14 void setup() {
15     pinsSetup(); //set up pins, from Freenove_4WD_Car_for_Arduino.h library
16     getTrackingSensorVal(); //Calculate Voltage speed Compensation
17 }
18
19 void loop() {
20     u8 trackingSensorVal = 0;
21     trackingSensorVal = getTrackingSensorVal(); //get sensor value
22

```

```
23  switch (trackingSensorVal)
24  {
25      case 0:    //000
26          motorRun(TK_FORWARD_SPEED, TK_FORWARD_SPEED); //car move forward
27          break;
28      case 7:    //111
29          motorRun(TK_STOP_SPEED, TK_STOP_SPEED); //car stop
30          break;
31      case 1:    //001
32          motorRun(TK_TURN_SPEED_LV4, TK_TURN_SPEED_LV1); //car turn
33          break;
34      case 3:    //011
35          motorRun(TK_TURN_SPEED_LV3, TK_TURN_SPEED_LV2); //car turn right
36          break;
37      case 2:    //010
38      case 5:    //101
39          motorRun(TK_FORWARD_SPEED, TK_FORWARD_SPEED); //car move forward
40          break;
41      case 6:    //110
42          motorRun(TK_TURN_SPEED_LV2, TK_TURN_SPEED_LV3); //car turn left
43          break;
44      case 4:    //100
45          motorRun(TK_TURN_SPEED_LV1, TK_TURN_SPEED_LV4); //car turn right
46          break;
47      default:
48          break;
49  }
50 }
51
52 void tk_CalculateVoltageCompensation() {
53     getBatteryVoltage(); //from Freenove_4WD_Car_for_Arduino.h library
54     float voltageOffset = 7 - batteryVoltage;
55     tk_VoltageCompensationToSpeed = 30 * voltageOffset;
56 }
57
58 //when black line on one side is detected, the value of the side will be 0, or the value is 1
59 u8 getTrackingSensorVal() {
60     u8 trackingSensorVal = 0;
61     trackingSensorVal = (digitalRead(PIN_TRACKING_LEFT) == 1 ? 1 : 0) << 2 |
62     (digitalRead(PIN_TRACKING_CENTER) == 1 ? 1 : 0) << 1 | (digitalRead(PIN_TRACKING_RIGHT) == 1 ?
63     1 : 0) << 0;
64     return trackingSensorVal;
65 }
```

Exp1 ? Exp2 : Exp3;

If Exp1 is true, the result of this code is Exp2.

If Exp1 is false, the result of this code is Exp3.

For example

If y=8;

var = (y < 10) ? 30 : 40; then var=30

If y=10

var = (y < 10) ? 30 : 40; then var=40

switch...case

```
switch (var) {  
  case 1:  
    //do something when var equals 1  
    break;  
  case 2:  
    //do something when var equals 2  
    break;  
  default:  
    // if nothing else matches, do the default  
    // default is optional  
    break;  
}
```

For more information, please refer to:

<https://www.arduino.cc/reference/en/language/structure/control-structure/switchcase/>

Chapter 4 IR control

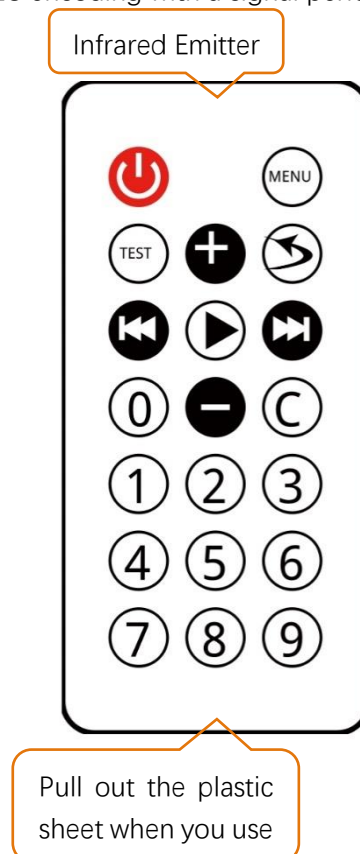
In this chapter, we will introduce infrared remote and receiver and make an IR remote control car.

If you have any concerns, please feel free to contact us via support@freenove.com

4.1 Infrared remote and receiver

Infrared(IR) remote control is a device with a certain number of buttons. Pressing down different buttons will make the infrared emission tube, which is located in the front of the remote control, send infrared with different encodings. Infrared remote control technology is widely used, such as TV, air conditioning, etc. Thus, it makes it possible for you to switch TV programs and adjust the temperature of the air conditioning when you are away from them. The remote control we use is shown below:

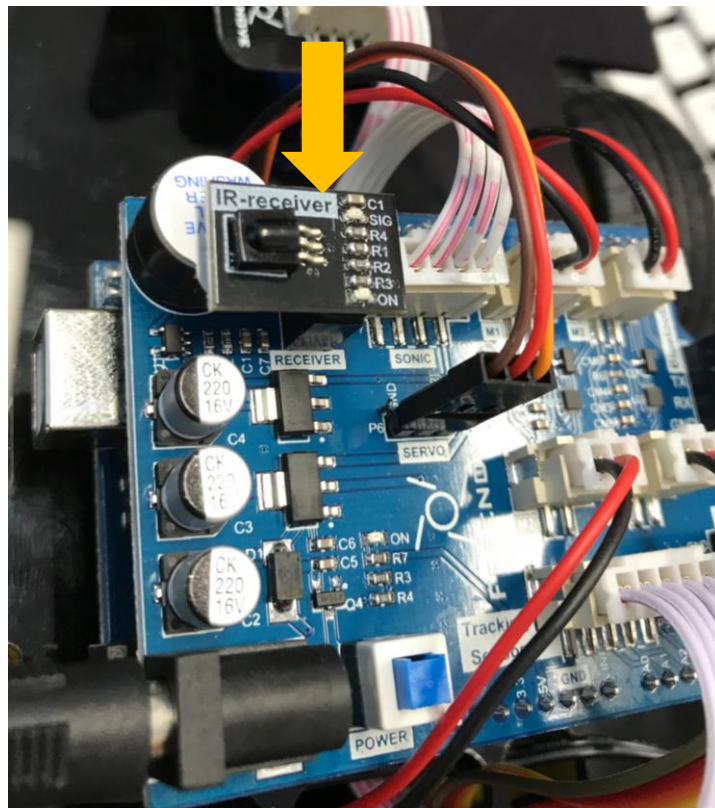
This infrared remote control uses NEC encoding with a signal period of 108ms.



Infrared(IR) receiver is a component, which can receive the infrared light, so we can use it to detect the signal emitted by the infrared remote control. DATA pin here outputs the received infrared signal.



Install IR receiver on the car as below:



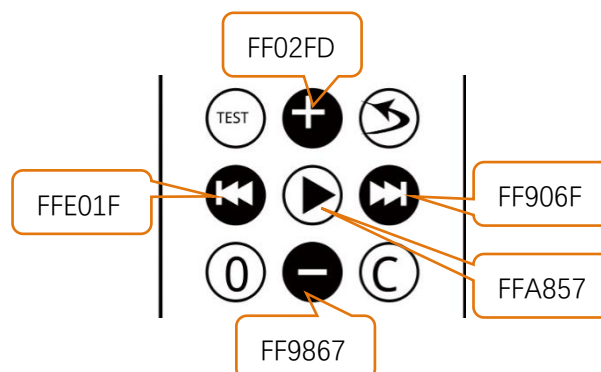
Code

04.1_IR_Receiver

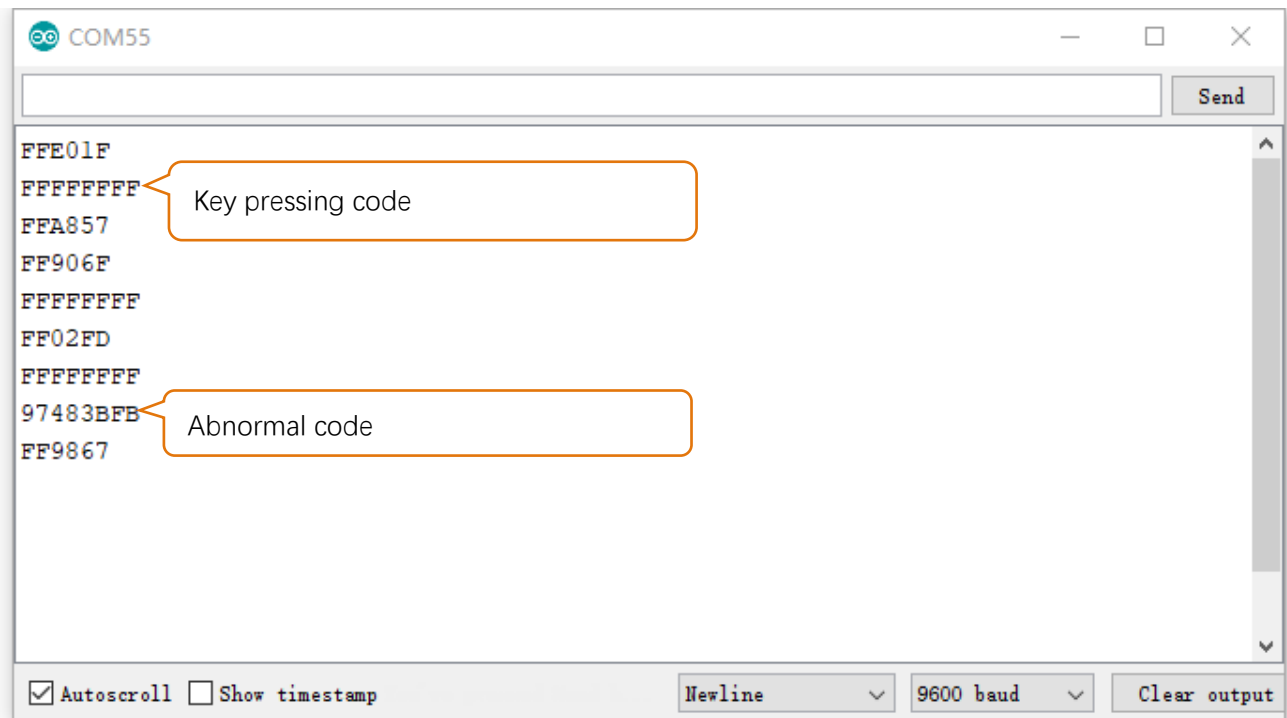
You need remove Bluetooth Module Module first when you upload code.

Then upload code in Sketches\04.1_IR_Receiver, turn on the power and open the serial monitor.

And then **pull off the plastic piece closed to the battery**, and press following keys in IR remote.



After receiving the signal from Infrared Emitter, the controller will recognize which key has been pressed. And print it in the monitor as below



When you press one key lasting for some time, FFFFFFFF will be shown. It indicates that the key is in pressing state.

IR communication is not very stable, so it is easily to be interfered. When used, the transmitting LED of remote control must be aimed at the receiver. Otherwise, it may result in no data being received or incorrect data being received.

In addition, sometimes the code received is abnormal. Except key codes and FFFFFFFF, other codes received are abnormal. When we process the code, we need skip this kind of data.

Different key code corresponds to different key.

```

1  #include <IRremote.h>           // include IRremote library
2  #define PIN_IRREMOTE_RECV  9    // Infrared receiving pin
3  IRrecv irrecv(PIN_IRREMOTE_RECV); // Create a class object used to receive
4  decode_results results;         // Create a decoding results class object
5
6  void setup()
7  {
8      Serial.begin(9600);          // Initialize the serial port and set the baud rate to 9600
9      irrecv.enableIRIn();         // Start the receiver
10 }
11
12 void loop() {
13     if (irrecv.decode(&results)) { // Waiting for decoding
14         Serial.println(results.value, HEX); // Print out the decoded results
15         irrecv.resume();             // Receive the next value
16     }
17     delay(100);
18 }

```

4.2 IR Remote Car

We will use IR remote to control the car to make some actions.

Code

04.2_IR_Remote_Car

You need remove Bluetooth Module Module first when you upload code.

First, upload the code in Sketches\04.2_IR_Remote_Car, and turn on the power of car.

Key graph	Key define	Key code	Function
	IR_REMOTE_KEYCODE_UP	0xFF02FD	move forward
	IR_REMOTE_KEYCODE_DOWN	0xFF9867	move back
	IR_REMOTE_KEYCODE_LEFT	0xFFE01F	Turn left
	IR_REMOTE_KEYCODE_RIGHT	0xFF906F	Turn right
	IR_REMOTE_KEYCODE_CENTER	0xFFA857	Turn on buzzer

The code is below:

```

1  #include <IRremote.h>
2  #include "Freenove_4WD_Car_for_Arduino.h"
3
4  //define key, the code cannot be changed.
5  #define IR_REMOTE_KEYCODE_UP      0xFF02FD
6  #define IR_REMOTE_KEYCODE_DOWN    0xFF9867
7  #define IR_REMOTE_KEYCODE_LEFT    0xFFE01F
8  #define IR_REMOTE_KEYCODE_RIGHT   0xFF906F
9  #define IR_REMOTE_KEYCODE_CENTER  0xFFA857
10
11 #define IR_UPDATE_TIMEOUT          120
12 #define IR_CAR_SPEED               180
13
14 IRrecv irrecv(PIN_IRREMOTE_RECV);
15 decode_results results;
16 u32 currentKeyCode, lastKeyCode;
17 bool isStopFromIR = false;
18 u32 lastIRUpdateTime = 0;
19
20 void setup() {
```

```
21  irrecv.enableIRIn(); // Start the receiver
22  }
23
24  void loop() {
25    if (irrecv.decode(&results)) {
26      isStopFromIR = false;
27      currentKeyCode = results.value;
28      if (currentKeyCode != 0xFFFFFFFF) {
29        lastKeyCode = currentKeyCode;
30      }
31      switch (lastKeyCode) {
32        case IR_REMOTE_KEYCODE_UP:
33          motorRun(IR_CAR_SPEED, IR_CAR_SPEED); //move forward
34          break;
35        case IR_REMOTE_KEYCODE_DOWN:
36          motorRun(-IR_CAR_SPEED, -IR_CAR_SPEED); //move back
37          break;
38        case IR_REMOTE_KEYCODE_LEFT:
39          motorRun(-IR_CAR_SPEED, IR_CAR_SPEED); //turn left
40          break;
41        case IR_REMOTE_KEYCODE_RIGHT:
42          motorRun(IR_CAR_SPEED, -IR_CAR_SPEED); //turn right
43          break;
44        case IR_REMOTE_KEYCODE_CENTER:
45          setBuzzer(true);           //turn on buzzer
46          break;
47        default:
48          break;
49      }
50      irrecv.resume(); // Receive the next value
51      lastIRUpdateTime = millis(); //write down current time
52    }
53    else {
54      if (millis() - lastIRUpdateTime > IR_UPDATE_TIMEOUT) {
55        if (!isStopFromIR) {
56          isStopFromIR = true;
57          motorRun(0, 0);
58          setBuzzer(false);
59        }
60        lastIRUpdateTime = millis();
61      }
62    }
63  }
64
```

Since the emission period of the infrared remote control is 108 ms, the signal timeout time is set to a rough value of 120 ms. If the timeout is less than 108 ms, the data will be read ahead of time and cannot be read. It will cause a misunderstanding that the signal is lost. If the timeout time is set too large, such as 1000ms, the delay of car action will be more serious.

```

11  #define IR_UPDATE_TIMEOUT    120
    .....
54      if (millis() - lastIRUpdateTime > IR_UPDATE_TIMEOUT) {
55          if (!isStopFromIR) {
56              isStopFromIR = true;
57              motorRun(0, 0);
58              setBuzzer(false);
59          }
60          lastIRUpdateTime = millis();
61      }
    .....

```

You can change value of `IR_CAR_SPEED` to change speed of the car in range (0~255). In fact, since motor need enough voltage to make the car move so the minimum value is not 0.

```

12  #define IR_CAR_SPEED        180

```

millis()

Returns the number of milliseconds passed since the Arduino board began running the **current** program

```

    lastIRUpdateTime = millis(); //write down current time
}

```

```

else {

```

```

    if (millis() - lastIRUpdateTime)

```

`millis() - lastIRUpdateTime` is the time from `lastIRUpdateTime` to current line.

For more details, please refer to:

<https://www.arduino.cc/reference/en/language/functions/time/millis/>

4.3 Multifunctional IR Remote Car

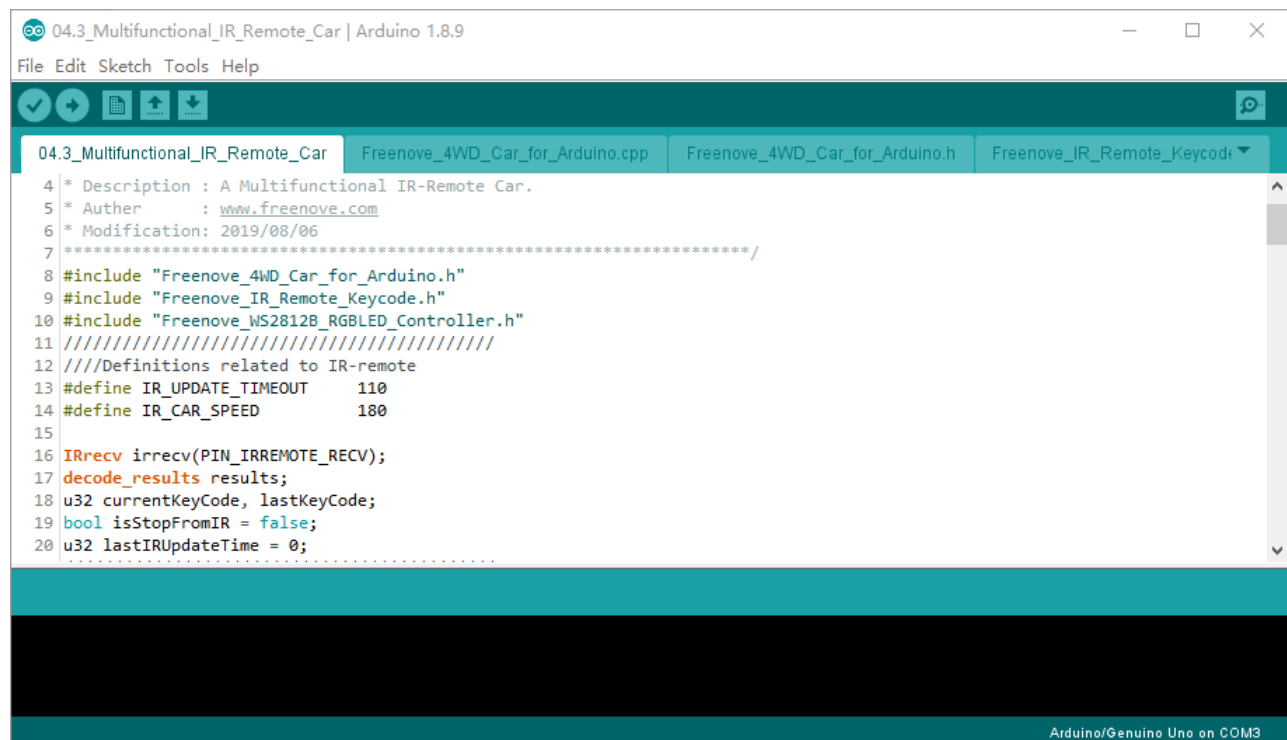
In this project, we will add a LED bar function compared to the last section, so the infrared remote control car will have more functions.

Upload Code and Running

You need remove Bluetooth Module Module first when you upload code.

Open the code in **Sketches\04.4_One_Code_Multifunctional_IR_Remote_Car**. Upload it into car.

You can also choose to Sketches\04.3_Multifunctional_IR_Remote_Car

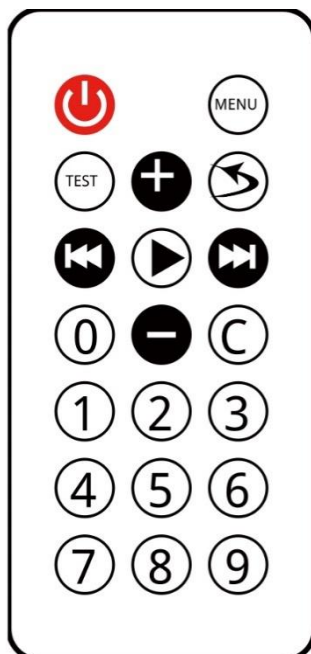


The screenshot shows the Arduino IDE interface with the file '04.3_Multifunctional_IR_Remote_Car' open. The code is for an Arduino 1.8.9 and includes comments and preprocessor directives for a multifunctional IR remote car. The code is as follows:

```
4 * Description : A Multifunctional IR-Remote Car.
5 * Author      : www.freenove.com
6 * Modification: 2019/08/06
7 *****
8 #include "Freenove_4WD_Car_for_Arduino.h"
9 #include "Freenove_IR_Remote_Keycode.h"
10 #include "Freenove_WS2812B_RGBLED_Controller.h"
11 //////////////////////////////////////////////////
12 //Definitions related to IR-remote
13 #define IR_UPDATE_TIMEOUT    110
14 #define IR_CAR_SPEED        180
15
16 IRrecv irrecv(PIN_IRREMOTE_RECV);
17 decode_results results;
18 u32 currentKeyCode, lastKeyCode;
19 bool isStopFromIR = false;
20 u32 lastIRUpdateTime = 0;
21 .....
```

The IDE status bar at the bottom indicates 'Arduino/Genuino Uno on COM3'.

After the code is successfully uploaded, turn on the power of the car and use the infrared remote control to control the car and other functions. The corresponding keys and their functions are shown in the following table:



Key graph	Key define	Key code	Function
	IR_REMOTE_KEYCODE_UP	0xFF02FD	move forward
	IR_REMOTE_KEYCODE_DOWN	0xFF9867	move back
	IR_REMOTE_KEYCODE_LEFT	0xFFE01F	Turn left
	IR_REMOTE_KEYCODE_RIGHT	0xFF906F	Turn right
	IR_REMOTE_KEYCODE_CENTER	0xFFA857	Turn on buzzer
	IR_REMOTE_KEYCODE_1	0xFF30CF	Make the LED run mode 1 to scroll the rainbow color.
	IR_REMOTE_KEYCODE_4	0xFF10EF	Make LED run mode 2, changing the color of the water LED
	IR_REMOTE_KEYCODE_2	0xFF18E7	The color of the LED bar changes faster. The color is from ColorWheel.
	IR_REMOTE_KEYCODE_3	0xFF7A85	The color of the LED bar changes slower.
	IR_REMOTE_KEYCODE_5	0xFF38C7	The LED bar cycle period is decreased, and the LED bar changes at a faster speed
	IR_REMOTE_KEYCODE_6	0xFF5AA5	The LED bar cycle period is increased, and the LED bar changes at a slower speed

Code

You need remove Bluetooth Module Module first when you upload code.

This project has many labels.

"Freenove_IR_Remote_Keycode.h" is used to save key code of IR remote control.

Freenove_IR_Remote_Keycode.h

```

1 // Freenove_IR_Remote_Keycode.h
2 #ifndef _FREENOVE_IR_REMOTE_KEYCODE_h
3 #define _FREENOVE_IR_REMOTE_KEYCODE_h
4 #if defined(ARDUINO) && ARDUINO >= 100
5     #include "arduino.h"
6 #else
7     #include "WProgram.h"
8 #endif
9 #include <IRremote.h>
10
11 #define IR_REMOTE_KEYCODE_POWER    0xFFA25D
12 #define IR_REMOTE_KEYCODE_MENU     0xFF629D
13 #define IR_REMOTE_KEYCODE_MUTE     0xFFE21D
14
15 #define IR_REMOTE_KEYCODE_MODE     0xFF22DD
16 #define IR_REMOTE_KEYCODE_UP       0xFF02FD
17 #define IR_REMOTE_KEYCODE_BACK     0xFFC23D
18
19 #define IR_REMOTE_KEYCODE_LEFT     0xFFE01F
20 #define IR_REMOTE_KEYCODE_CENTER    0xFFA857
21 #define IR_REMOTE_KEYCODE_RIGHT    0xFF906F
22
23 #define IR_REMOTE_KEYCODE_0        0xFF6897
24 #define IR_REMOTE_KEYCODE_DOWN     0xFF9867
25 #define IR_REMOTE_KEYCODE_OK       0xFFB04F
26
27 #define IR_REMOTE_KEYCODE_1        0xFF30CF
28 #define IR_REMOTE_KEYCODE_2        0xFF18E7
29 #define IR_REMOTE_KEYCODE_3        0xFF7A85
30
31 #define IR_REMOTE_KEYCODE_4        0xFF10EF
32 #define IR_REMOTE_KEYCODE_5        0xFF38C7
33 #define IR_REMOTE_KEYCODE_6        0xFF5AA5
34
35 #define IR_REMOTE_KEYCODE_7        0xFF42BD
36 #define IR_REMOTE_KEYCODE_8        0xFF4AB5
37 #define IR_REMOTE_KEYCODE_9        0xFF52AD

```



```
43         break;
44     case IR_REMOTE_KEYCODE_DOWN:
45         motorRun(-IR_CAR_SPEED, -IR_CAR_SPEED);
46         break;
47     case IR_REMOTE_KEYCODE_LEFT:
48         motorRun(-IR_CAR_SPEED, IR_CAR_SPEED);
49         break;
50     case IR_REMOTE_KEYCODE_RIGHT:
51         motorRun(IR_CAR_SPEED, -IR_CAR_SPEED);
52         break;
53     case IR_REMOTE_KEYCODE_CENTER:
54         setBuzzer(true);
55         break;
56     case IR_REMOTE_KEYCODE_0:
57         break;
58     case IR_REMOTE_KEYCODE_1:
59         stripDisplayMode = 1;
60         break;
61     case IR_REMOTE_KEYCODE_2:
62         colorStep += 5;
63         if (colorStep > 100)
64         {
65             colorStep = 100;
66         }
67         break;
68     case IR_REMOTE_KEYCODE_3:
69         colorStep -= 5;
70         if (colorStep < 5)
71         {
72             colorStep = 5;
73         }
74         break;
75     case IR_REMOTE_KEYCODE_4:
76         stripDisplayMode = 0;
77         break;
78     case IR_REMOTE_KEYCODE_5:
79         stripDisplayDelay -= 20;
80         if (stripDisplayDelay < 20)
81         {
82             stripDisplayDelay = 20;
83         }
84         break;
85     case IR_REMOTE_KEYCODE_6:
86         stripDisplayDelay += 20;
```

```
87         if (stripDisplayDelay > 300)
88         {
89             stripDisplayDelay = 300;
90         }
91         break;
92     }
93     irrecv.resume(); // Receive the next value
94     lastIRUpdateTime = millis();
95 }
96 else {
97     if (millis() - lastIRUpdateTime > IR_UPDATE_TIMEOUT)
98     {
99         if (!isStopFromIR) {
100             isStopFromIR = true;
101             motorRun(0, 0);
102             setBuzzer(false);
103         }
104         lastIRUpdateTime = millis();
105     }
106 }
107 switch (stripDisplayMode)
108 {
109 case 0:
110     if (millis() - lastStripUpdateTime > stripDisplayDelay)
111     {
112         for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
113             strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
114         }
115         strip.show();
116         colorPos += colorStep;
117         lastStripUpdateTime = millis();
118     }
119     break;
120 case 1:
121     if (millis() - lastStripUpdateTime > stripDisplayDelay)
122     {
123         strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
124         currentLedIndex++;
125         if (currentLedIndex == STRIP_LEDS_COUNT)
126         {
127             currentLedIndex = 0;
128             colorPos += colorStep;
129         }
130         lastStripUpdateTime = millis();
```

```
131     }
132     break;
133     default:
134         break;
135     }
136 }
```

Compared to the previous project, this code has more switch-case branches of the remote key code for controlling the light bar.

```
1  ....
2      case IR_REMOTE_KEYCODE_1:
3          stripDisplayMode = 1;
4          break;
5      case IR_REMOTE_KEYCODE_2:
6          colorStep += 5;
7          if (colorStep > 100)
8          {
9              colorStep = 100;
10         }
11         break;
12     case IR_REMOTE_KEYCODE_3:
13         colorStep -= 5;
14         if (colorStep < 5)
15         {
16             colorStep = 5;
17         }
18         break;
19     case IR_REMOTE_KEYCODE_4:
20         stripDisplayMode = 0;
21         break;
22     case IR_REMOTE_KEYCODE_5:
23         stripDisplayDelay -= 20;
24         if (stripDisplayDelay < 20)
25         {
26             stripDisplayDelay = 20;
27         }
28         break;
29     case IR_REMOTE_KEYCODE_6:
30         stripDisplayDelay += 20;
31         if (stripDisplayDelay > 300)
32         {
33             stripDisplayDelay = 300;
34         }
35         break;
```

.....

In the loop(), control the behavior of the light bar based on these variables above.

The variable **stripDisplayMode** is used to control the display mode of the LED.

The variable **stripDisplayDelay** is used to control the cycle time of the LED, that is, the speed of change.

The variable **colorStep** is used to control the color value change for each iteration.

```
1  switch (stripDisplayMode)
2  {
3      case 0:
4          if (millis() - lastStripUpdateTime > stripDisplayDelay)
5          {
6              for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
7                  strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
8              }
9              strip.show();
10             colorPos += colorStep;
11             lastStripUpdateTime = millis();
12         }
13         break;
14     case 1:
15         if (millis() - lastStripUpdateTime > stripDisplayDelay)
16         {
17             strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
18             currentLedIndex++;
19             if (currentLedIndex == STRIP_LEDS_COUNT)
20             {
21                 currentLedIndex = 0;
22                 colorPos += colorStep;
23             }
24             lastStripUpdateTime = millis();
25         }
26         break;
27     default:
28         break;
29 }
```

Chapter 5 RF Remote Control

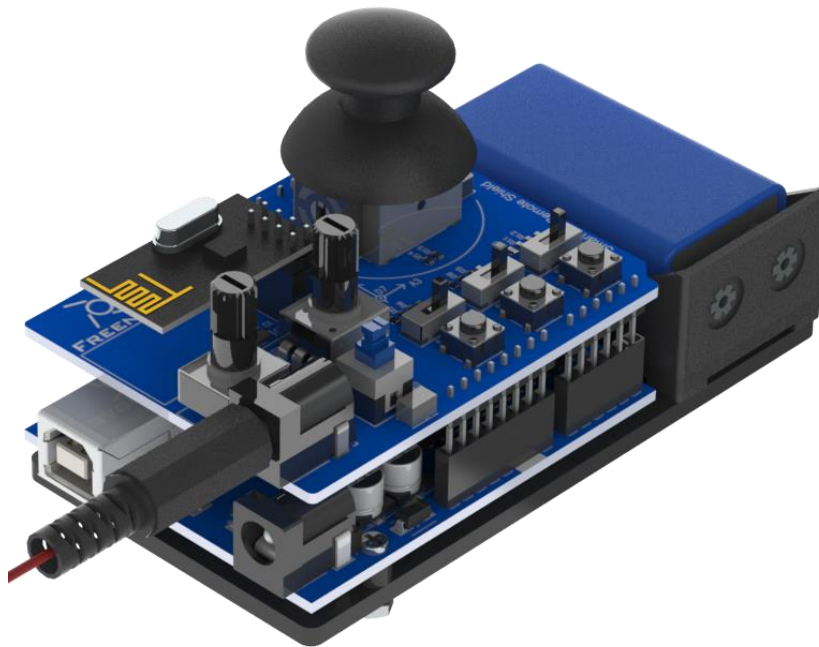
If you purchased the kit without RF remote control, you can skip this chapter.

If you have any concerns, please feel free to contact us via support@freenove.com

5.1 Remote Control

Download tutorial and code to **assemble** remote controller.

https://github.com/Freenove/Freenove_Remote_Control_Kit/archive/master.zip

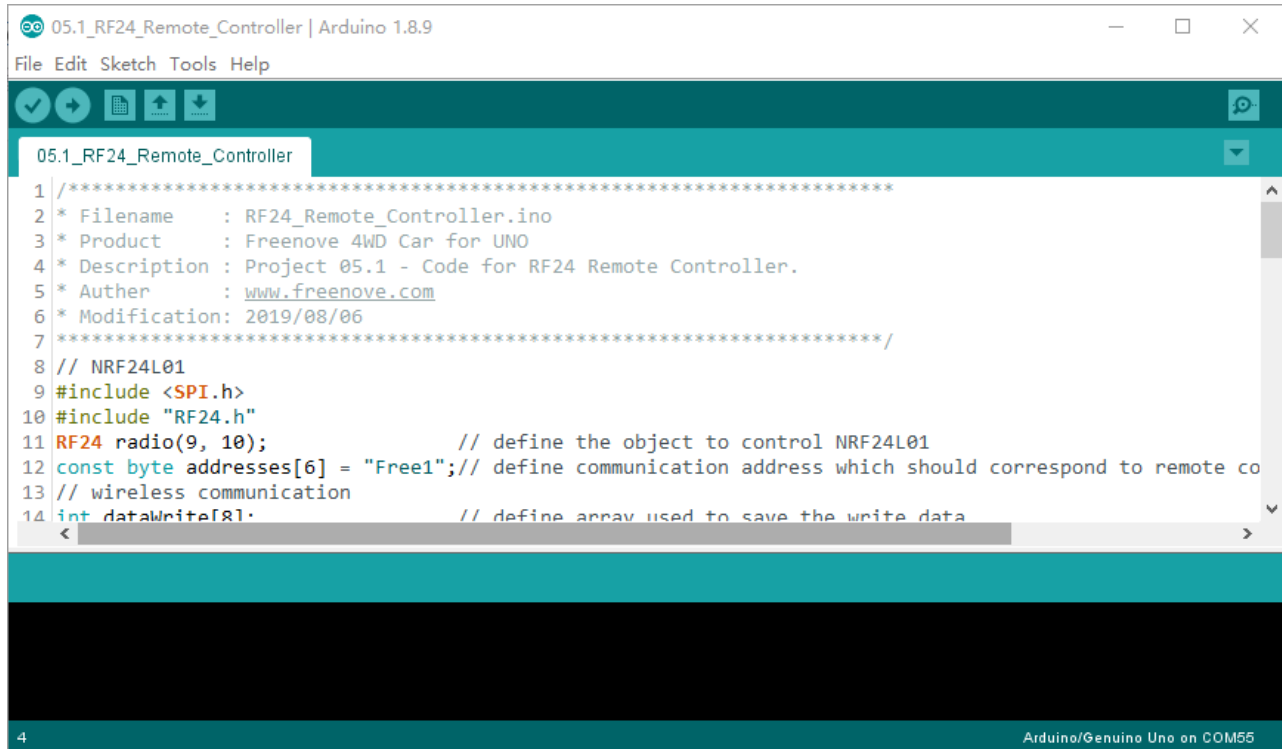


For all resources, please refer to https://github.com/Freenove/Freenove_Remote_Control_Kit

Upload Code and Running

Connect remote controller to computer. And upload code in Sketches\05.1_RF24_Remote_Controller.

Then turn on the power of RF remote control. **You need remove Bluetooth Module first when you upload code.**



05.1_RF24_Remote_Controller

The code is below:

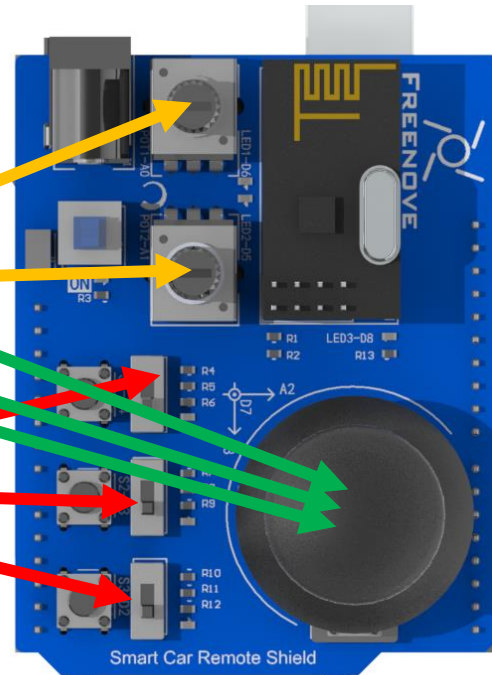
```

1 // RF24L01
2 #include <SPI.h>
3 #include "RF24.h"
4 RF24 radio(9, 10);           // define the object to control RF24L01
5 const byte addresses[6] = "Free1"; // define communication address which should correspond to
6 the control board of car
7 // wireless communication
8 int dataWrite[8];           // define array used to save the write data
9 // pin
10 const int pot1Pin = A0,      // define POT1 Potentiometer
11          pot2Pin = A1,      // define POT2 Potentiometer
12          joystickXPin = A2,  // define pin for direction X of joystick
13          joystickYPin = A3,  // define pin for direction Y of joystick
14          joystickZPin = 7,    // define pin for direction Z of joystick
15          s1Pin = 4,          // define pin for S1
16          s2Pin = 3,          // define pin for S2
17          s3Pin = 2,          // define pin for S3
  
```

```

18         led1Pin = 6,           // define pin for LED1 which is close to POT1 and used to
19 indicate the state of POT1
20         led2Pin = 5,           // define pin for LED2 which is close to POT2 and used to
21 indicate the state of POT2
22         led3Pin = 8;           // define pin for LED3 which is close to RF24L01 and used to
23 indicate the state of RF24L01
24
25 void setup() {
26     // RF24L01
27     radio.begin();              // initialize RF24
28     radio.setPALevel(RF24_PA_MAX); // set power amplifier (PA) level
29     radio.setDataRate(RF24_1MBPS); // set data rate through the air
30     radio.setRetries(0, 15);      // set the number and delay of retries
31     radio.openWritingPipe(addresses); // open a pipe for writing
32     radio.openReadingPipe(1, addresses); // open a pipe for reading
33     radio.stopListening();        // stop listening for incoming messages
34
35     // pin
36     pinMode(joystickZPin, INPUT); // set led1Pin to input mode
37     pinMode(s1Pin, INPUT);        // set s1Pin to input mode
38     pinMode(s2Pin, INPUT);        // set s2Pin to input mode
39     pinMode(s3Pin, INPUT);        // set s3Pin to input mode
40     pinMode(led1Pin, OUTPUT);     // set led1Pin to output mode
41     pinMode(led2Pin, OUTPUT);     // set led2Pin to output mode
42     pinMode(led3Pin, OUTPUT);     // set led3Pin to output mode
43 }
44
45 void loop()
46 {
47     // put the values of rocker, switch and pot
48     dataWrite[0] = analogRead(pot1Pin); // save
49     dataWrite[1] = analogRead(pot2Pin); // save
50     dataWrite[2] = analogRead(joystickXPin); //
51     dataWrite[3] = analogRead(joystickYPin); //
52     dataWrite[4] = digitalRead(joystickZPin); //
53     dataWrite[5] = digitalRead(s1Pin); //
54     dataWrite[6] = digitalRead(s2Pin); //
55     dataWrite[7] = digitalRead(s3Pin); //
56
57     // write radio data
58     if (radio.write(dataWrite, sizeof(dataWrite)
59     {
60         digitalWrite(led3Pin, HIGH);
61         delay(20);

```



```

62     digitalWrite(led3Pin, LOW);
63 }
64
65 // make LED emit different brightness of light according to analog of potentiometer
66 analogWrite(led1Pin, map(dataWrite[0], 0, 1023, 0, 255));
67 analogWrite(led2Pin, map(dataWrite[1], 0, 1023, 0, 255));
68 }

```

```

5 const byte addresses[6] = "Free1";

```

You can change the address if you have two Freenove remote to control different cars, like Free2. But it is **necessary** to keep it same to the address in code in car.

```

48 dataWrite[0] = analogRead(pot1Pin); // (0~1023) save data of Potentiometer 1
49 dataWrite[1] = analogRead(pot2Pin); // (0~1023) save data of Potentiometer 2
50 dataWrite[2] = analogRead(joystickXPin); // (0~1023) save data of direction X of joystick
51 dataWrite[3] = analogRead(joystickYPin); // (0~1023) save data of direction Y of joystick
52 dataWrite[4] = digitalRead(joystickZPin); // (0,1) save data of direction Z of joystick
53 dataWrite[5] = digitalRead(s1Pin); // (0,1) save data of switch 1
54 dataWrite[6] = digitalRead(s2Pin); // (0,1) save data of switch 2
55 dataWrite[7] = digitalRead(s3Pin); // (0,1) save data of switch 3

```

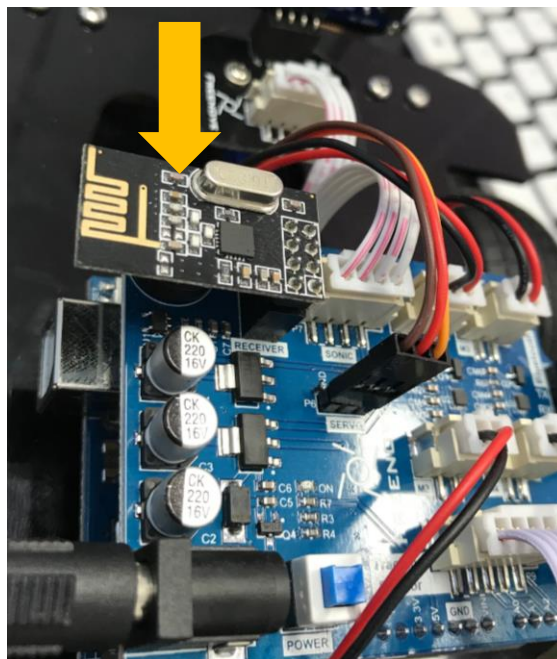
When joystick is in center place, its value is about x=512, y=512

When joystick is pressed, z=0; when it is not pressed, z=1.

Switches are the same, when switch is on, s=0; when switch is off, s=1.

5.2 Receive RF Remote Control Data

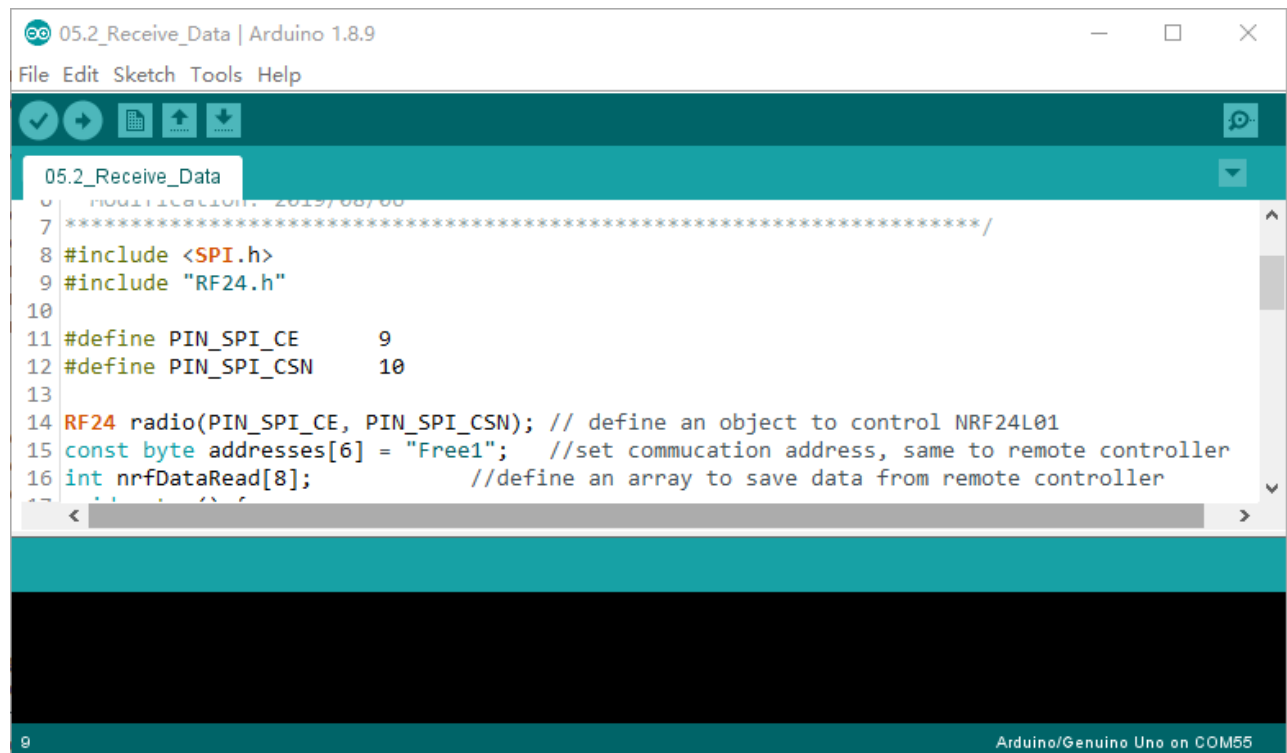
Install RF module on the car. RF module and IR receiver use the same receiver pins.



Upload Code and Running

Then connect car control board to computer. And upload code in Sketches\05.2_Receive_Data.

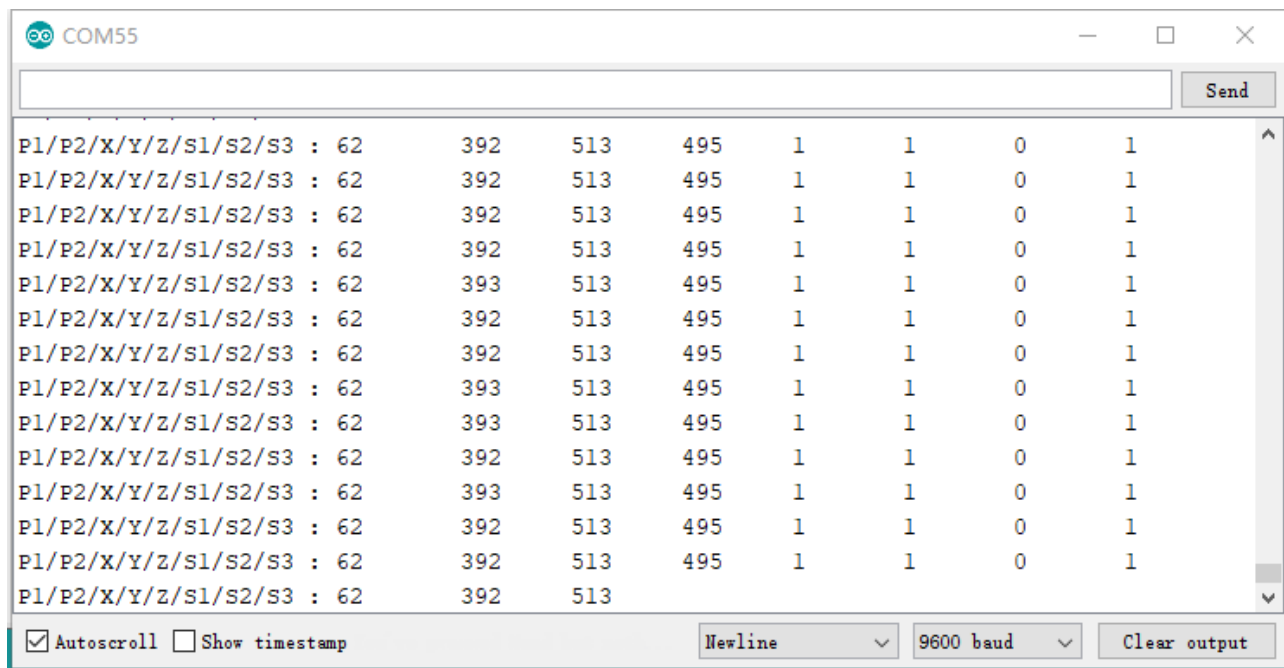
You need remove Bluetooth Module first when you upload code.



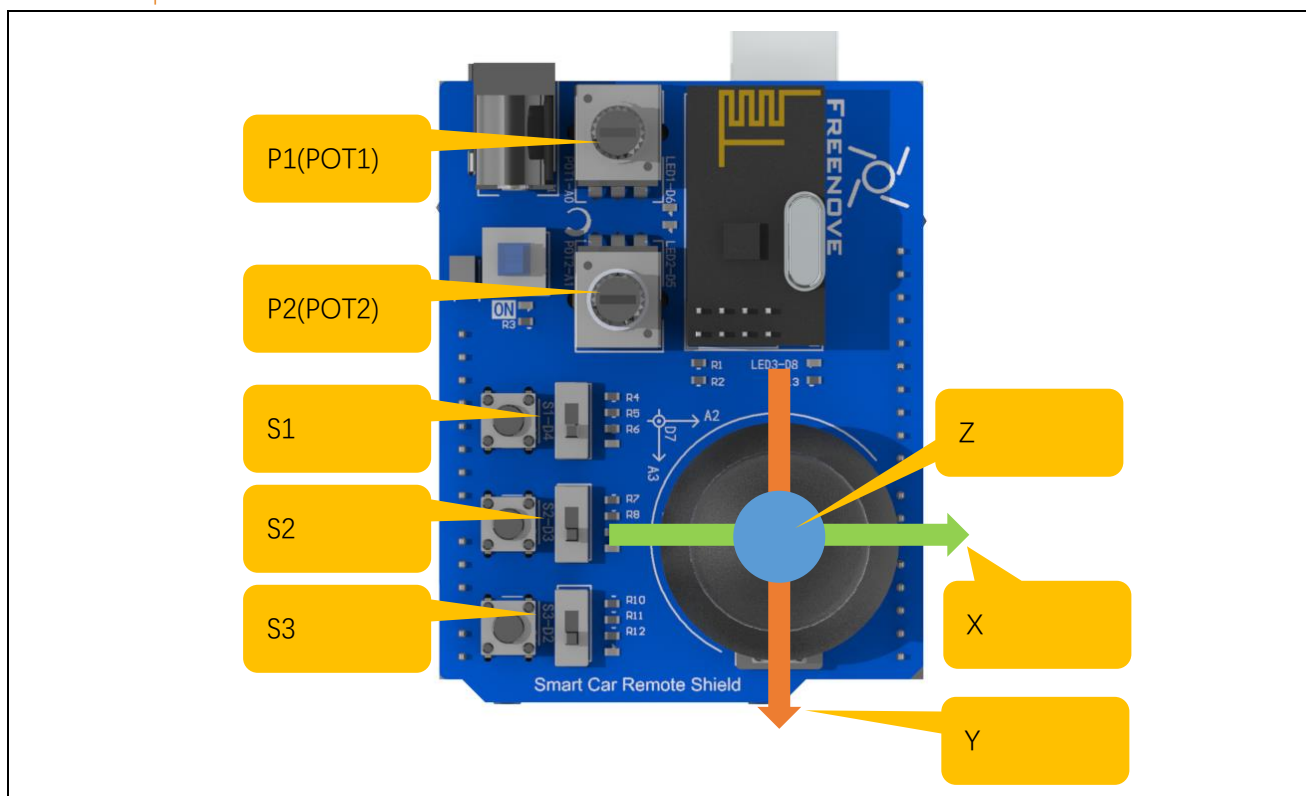
Power the remote controller.

Keep car control board connected to computer via USB cable and open the serial monitor. You will see contents as below.

Operate the remote controller and observe the data change.



Data to components



Code

05.2_Receive_Data

The code is below.

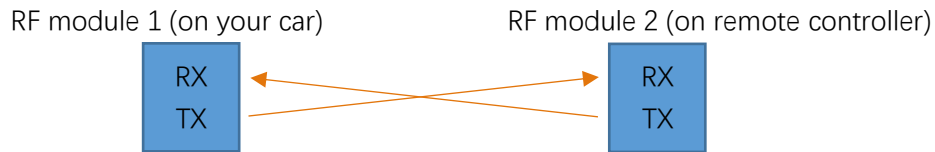
```
1  #include <SPI.h>
2  #include "RF24.h"
3
4  #define PIN_SPI_CE      9
5  #define PIN_SPI_CSN     10
6
7  RF24 radio(PIN_SPI_CE, PIN_SPI_CSN); // define an object to control RF24L01
8  const byte addresses[6] = "Free1";  //set commutation address, same to remote controller
9  int RFDataRead[8];                //define an array to save data from remote controller
10 void setup() {
11     Serial.begin(9600);
12
13     // RF24L01 initialization steps
14     if (radio.begin()) {            // initialize RF24
15         radio.setPALevel(RF24_PA_MAX); // set power amplifier (PA) level
16         radio.setDataRate(RF24_1MBPS); // set data rate through the air
17         radio.setRetries(0, 15);      // set the number and delay of retries
18         radio.openWritingPipe(addresses); // open a pipe for writing
19         radio.openReadingPipe(1, addresses); // open a pipe for reading
20         radio.startListening();        // start monitoring start listening on the pipes opened
21         Serial.println("Start listening remote data ... ");
22     }
23     else {
24         Serial.println("Not found the RF chip!");
25     }
26 }
27
28 void loop() {
29     if (radio.available()) {          // if receive the data
30         while (radio.available()) {    // read all the data
31             radio.read(RFDataRead, sizeof(RFDataRead)); // read data
32         }
33         Serial.print("P1/P2/X/Y/Z/S1/S2/S3 : ");
34         for (int i = 0; i < sizeof(RFDataRead) / 2; i++) {
35             Serial.print(RFDataRead[i]);
36             Serial.print('\t');
37         }
38         Serial.print('\n');
39     }
```

```

40 }
41

```

The communication is as follows:



`radio.write(dataWrite, sizeof(dataWrite))` Transfer the data.

Both of the codes for car and for remote have following contents:

```

5  const byte addresses[6] = "Free1";
    .....
18      radio.openWritingPipe(addresses);    // open a pipe for writing
19      radio.openReadingPipe(1, addresses); // open a pipe for reading

```

This is used to set writing (transmitting) and reading (receiving) addresses of the RF24L01 module. Module B can receive the data of module A, only when the writing address of module A is the same as the reading address of module B. The module's own writing and reading addresses can be the same or different. When there are many RF modules around, the writing and reading addresses can be changed to avoid interference from other devices.

After we receive data from remote, we will recognize which component is operated according to the data change.

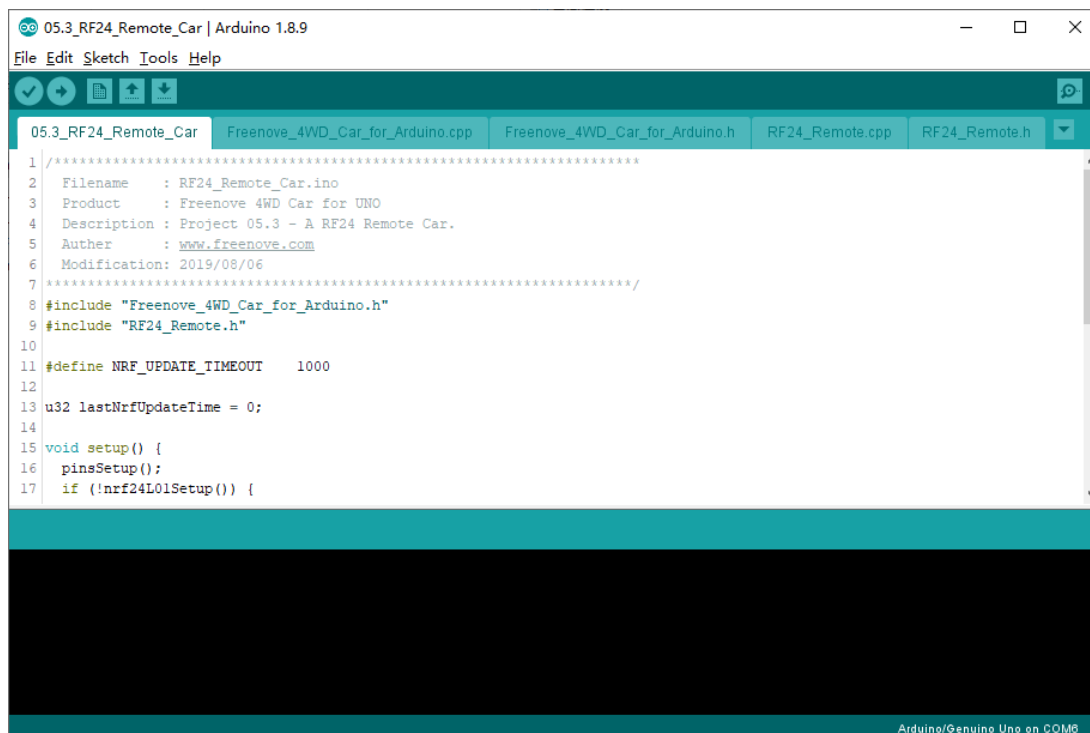
5.3 RF Remote Car

After successfully receiving the data sent by the RF remote control, we can use the data to control car movement, buzzer sounds, etc.

Upload Code and Running

Connect Car to computer with USB cable, and upload code in Sketches\05.3_RF24_Remote_Car.ino.

You need remove Bluetooth Module first when you upload code.

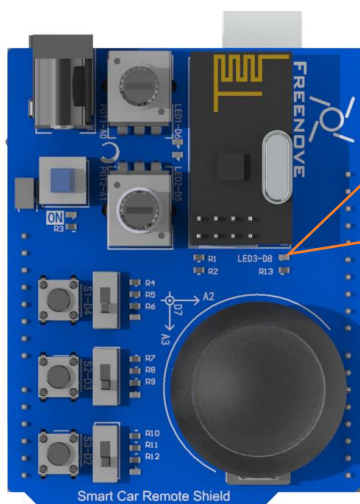


```
05.3_RF24_Remote_Car | Arduino 1.8.9
File Edit Sketch Tools Help

05.3_RF24_Remote_Car Freenove_4WD_Car_for_Arduino.cpp Freenove_4WD_Car_for_Arduino.h RF24_Remote.cpp RF24_Remote.h

1 /*****
2  Filename   : RF24_Remote_Car.ino
3  Product    : Freenove 4WD Car for UNO
4  Description : Project 05.3 - A RF24 Remote Car.
5  Author     : www.freenove.com
6  Modification: 2019/08/06
7 *****/
8 #include "Freenove_4WD_Car_for_Arduino.h"
9 #include "RF24_Remote.h"
10
11 #define NRF_UPDATE_TIMEOUT 1000
12
13 u32 lastNrfUpdateTime = 0;
14
15 void setup() {
16   pinsSetup();
17   if (!nrf24L01Setup()) {
```

Then disconnect the USB cable and turn on the RF Remote Control and Car power switches. Operate the joystick to control the movement of the car. Press the joystick to turn on the buzzer.



LED on state indicate the communication successes.
LED off state indicates the communication fails.

When it is off, you need restart power of remote controller and car.

Code

The project has five labels, of which "RF24_Remote.h" and "RF24_Remote.cpp" store contents about data reading and data processing for the RF24 remote control. These contents are called in "05.3_RF24_Remote_Car.ino".

RF24_Remote.h

```

1  #ifndef _RF_REMOTE_h
2  #define _RF_REMOTE_h
3
4  #include "Freenove_4WD_Car_for_Arduino.h"
5  #include "RF24.h"
6
7  extern RF24 radio;
8
9  enum RemoteData
10 {
11     POT1 = 0,
12     POT2 = 1,
13     JOYSTICK_X = 2,
14     JOYSTICK_Y = 3,
15     JOYSTICK_Z = 4,
16     S1 = 5,
17     S2 = 6,
18     S3 = 7
19 };
20 enum RemoteMode {
21     ON_ON_ON = 0,
22     ON_ON_OFF = 1,           //Servo calibration mode.
23     ON_OFF_ON = 2,
24     ON_OFF_OFF = 3,         //Ultrasonic obstacle avoidance mode.
25     OFF_ON_ON = 4,
26     OFF_ON_OFF = 5,         //Line tracking mode.
27     OFF_OFF_ON = 6,        //
28     OFF_OFF_OFF = 7         //Remote control mode.
29 };
30 enum RemoteModeSwitchState {
31     MODE_SWITCHING_IS_INITIALIZING = 0,
32     MODE_SWITCHING_IS_PROCESSING = 1,
33     MODE_SWITCHING_IS_CONFIRMING = 2,
34     MODE_SWITCHING_WAS_FINISHED = 3
35 };
36
37 #define MODE_REMOTE_CONTROL      OFF_OFF_OFF

```

```

38 #define MODE_LINE_TRACKING          OFF_ON_OFF
39 #define MODE_OBSTACLE_AVOIDANCE     ON_OFF_OFF
40
41 extern int RFDDataRead[8];
42
43 bool RF24L01Setup();
44 bool getRF24L01Data();
45 void updateCarActionByRFRemote();
46 void resetRFDDataBuf();
47 u8 updateRFCarMode();
48
49 #endif

```

Where enum RemoteData defines the order of the remote control data received, using more meaningful variable names instead of numbers. For example, if you want to use the X-axis data of the remote control, you can use RFDDataRead[JOYSTICK_X] instead of the no meaningless RFDDataRead[2]. It like define

```

enum RemoteData
{
    POT1 = 0, // similar to #define POT1 0
    POT2 = 1,
    JOYSTICK_X = 2,
    JOYSTICK_Y = 3,
    JOYSTICK_Z = 4,
    S1 = 5,
    S2 = 6,
    S3 = 7
};

```

The enum RemoteMode defines all states of the remote control switches S1, S2, S3. The enum RemoteModeSwitchState defines the state when the remote control switches modes. These contents will also be used in the next project.

```

enum RemoteMode {
    ON_ON_ON = 0,
    ON_ON_OFF = 1,          //Servo calibration mode.
    ON_OFF_ON = 2,
    ON_OFF_OFF = 3,         //Ultrasonic obstacle avoidance mode.
    OFF_ON_ON = 4,
    OFF_ON_OFF = 5,         //Line tracking mode.
    OFF_OFF_ON = 6,        //
    OFF_OFF_OFF = 7        //Remote control mode.
};

enum RemoteModeSwitchState {
    MODE_SWITCHING_IS_INITIALIZING = 0,
    MODE_SWITCHING_IS_PROCESSING = 1,

```

```

    MODE_SWITCHING_IS_CONFIRMING = 2,
    MODE_SWITCHING_WAS_FINISHED = 3
};

```

```
bool RF24L01Setup();
```

Initialize RF24L01. If success, return true, if not, return false.

```
bool getRF24L01Data();
```

Read data from remote control. If there is data read, return true; if not, return false.

```
void updateCarActionByRFRemote();
```

Control car actions (mainly for motor and buzzer) according to data received.

```
void resetRFDataBuf();
```

Set values of RFData array to default values.

```
u8 updateRFCarMode();
```

Combine data of S1 S2 S3 of remote control into one value and return it.

05.3_RF24_Remote_Car.ino

```

1  #include "Freenove_4WD_Car_for_Arduino.h"
2  #include "RF24_Remote.h"
3
4  #define RF_UPDATE_TIMEOUT    1000
5  u32 lastRFUpdateTime = 0;
6
7  void setup() {
8      pinsSetup();
9      if (!RF24L01Setup()) {
10         alarm(4, 2);
11     }
12 }
13
14 void loop() {
15     if (getRF24L01Data()) {
16         updateCarActionByRFRemote();
17         lastRFUpdateTime = millis();
18     }
19     if (millis() - lastRFUpdateTime > RF_UPDATE_TIMEOUT) {
20         lastRFUpdateTime = millis();
21         resetRFDataBuf();
22         updateCarActionByRFRemote();
23     }
24 }

```

In setup(), initialize pins and RF24L01. In loop(), receive data from the remote control and process the data. If receiving data is time out, the remote control signal is considered to be lost and the car will be placed in the initialization state.

5.4 Multifunctional RF24 Remote Car

This project combines almost all functions of the car, such as obstacle avoidance mode, line tracking mode, remote control mode, RGB LED display mode, servo position calibration and so on. The switching among different functions is realized by switching the states of S1, S2, and S3 on the remote controller.

The project code is more complicated, but the components and knowledge used have been introduced in the previous project. The only difference is that they are integrated into one project, which is quite large.

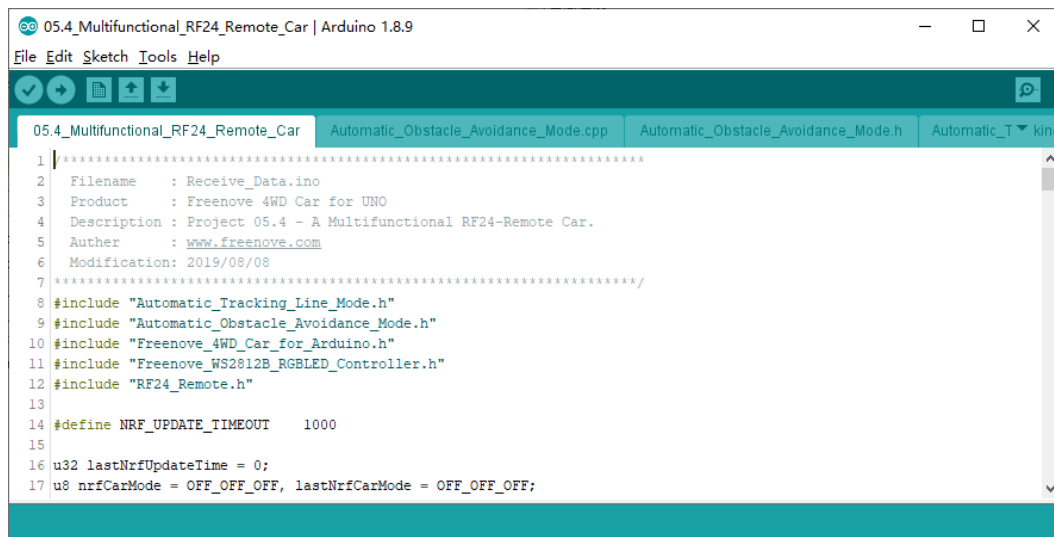
Upload Code and Running

Connect Car to computer with USB cable.

You need remove Bluetooth Module first when you upload code.

And upload code in **Sketches\05.5_One_Code_Multifunctional_RF24_Remote_Car**

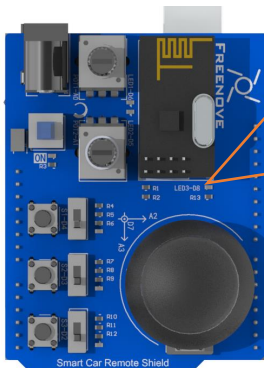
You can also choose to **Sketches\05.4_Multifunctional_RF24_Remote_Car.ino**.



```

1 //*****
2 Filename   : Receive_Data.ino
3 Product    : Freenove 4WD Car for UNO
4 Description : Project 05.4 - A Multifunctional RF24-Remote Car.
5 Author     : www.freenove.com
6 Modification: 2019/08/08
7 //*****
8 #include "Automatic_Tracking_Line_Mode.h"
9 #include "Automatic_Obstacle_Avoidance_Mode.h"
10 #include "Freenove_4WD_Car_for_Arduino.h"
11 #include "Freenove_WS2812B_RGBLED_Controller.h"
12 #include "RF24_Remote.h"
13
14 #define NRF_UPDATE_TIMEOUT    1000
15
16 u32 lastNrfUpdateTime = 0;
17 u8 nrfCarMode = OFF_OFF_OFF, lastNrfCarMode = OFF_OFF_OFF;
  
```

Disconnect the USB cable. Turn on the power of the remote control and the car. By default, the car is in manual remote control mode, and the S1, S2, and S3 switches on the remote control are off.



LED ON state indicates the communication successes.
LED OFF state indicates the communication fails.

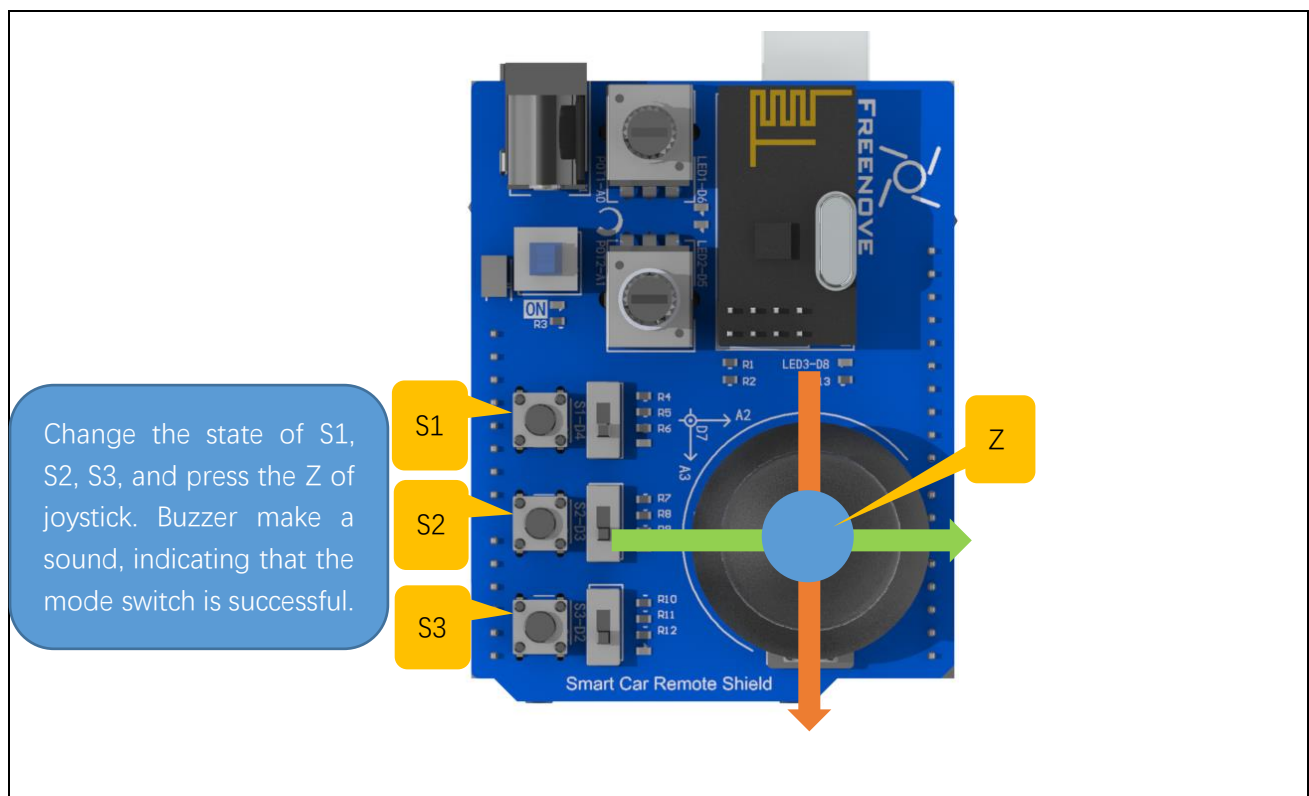
When it is OFF, you need to restart power of remote controller and car.

Switch different modes

- 1, Change the switch state of S1, S2, and S3, and the car will stop moving.
- 2, Press the Z axis of the joystick and the buzzer B sounds to indicate that the mode is successfully switched.

The following table shows the modes indicated by different states of the three switches S1, S2, and S3. The LED next to the switch illumination to indicate ON and OFF state of switches. The three switches can form $2 \times 2 \times 2 = 8$ modes.

S1	S2	S3	Mode No.	Mode
ON	ON	ON	0	None
ON	ON	OFF	1	Calibrate servo mode
ON	OFF	ON	2	None
ON	OFF	OFF	3	Obstacle avoidance mode
OFF	ON	ON	4	None
OFF	ON	OFF	5	Line tracking mode
OFF	OFF	ON	6	Switch LED mode
OFF	OFF	OFF	7	Manuel control mode / Default mode



Mode 0, 2, 4

Reserved. We did not assign functions for them.

Mode 1-Calibrate servo

If your servo is not accurately mounted at 90 degrees, you can use this mode for fine adjustment (+ -10 degrees).

In this mode, you can adjust potentiometer 2 (POT2) to fine tune the angle of the servo. When you adjust the servo to the correct angle, press the Z-axis of joystick to save calibration data to EEPROM. It will be saved permanently unless it is modified.

Mode 3-obstacle avoidance, Mode 5-line tracking mode

These two modes have been learned separately in the previous project, and their running logic and codes are consistent with the previous project.

The difference is that in this project, the car can respond to commands from the remote control at any time. Therefore, in this project, it is still necessary to communicate with the remote controller in these two modes. When the remote control signal is disconnected, the car will stop. Therefore, the normal communication between the remote control and the car should be maintained at any time. Poor communication conditions may cause these two modes to work abnormally.

Mode 6-switch LED display mode

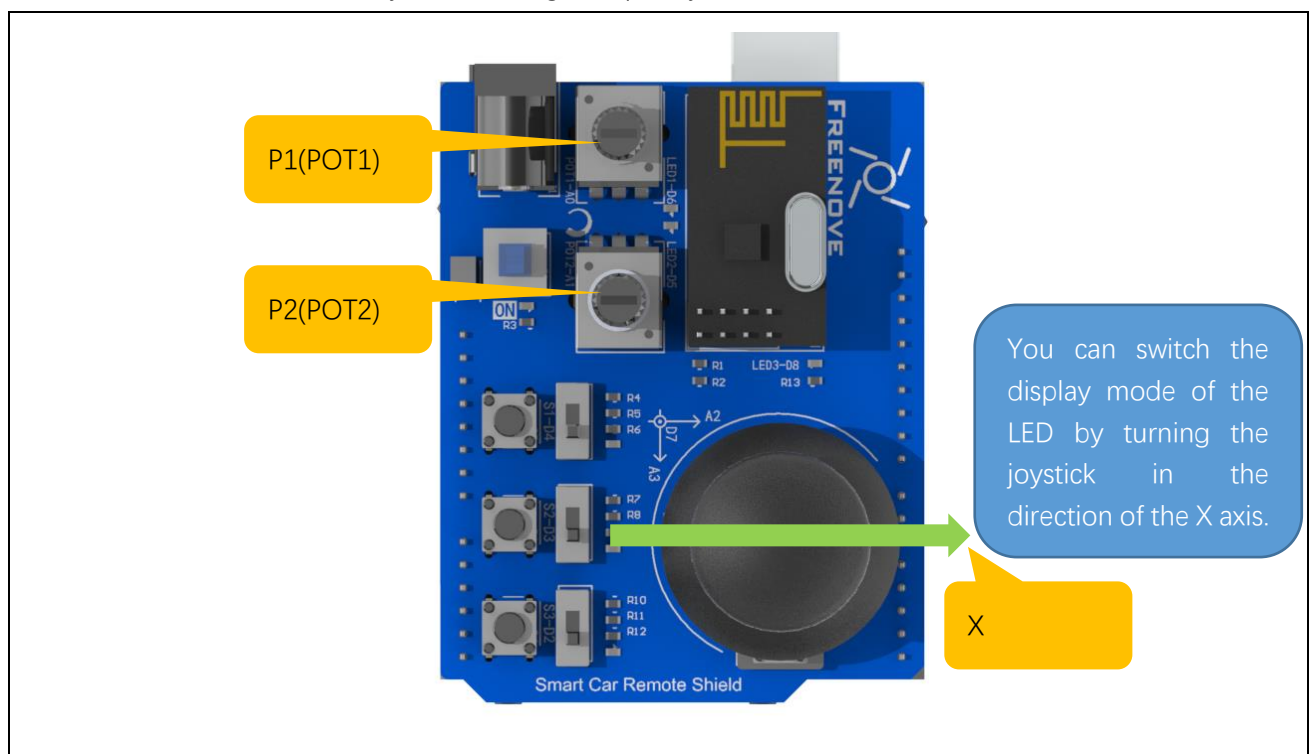
There are three display modes for the LEDs on the car, which are 0-flowing rainbow, 1-flowing water led, 2-Blink. In this mode, the display mode of the LED can be switched.

After entering this mode,

Move the joystick along the positive direction of its X-axis to switch the LED to the next mode.

Move the joystick in the negative direction of its X-axis to switch the LED to the previous mode.

In any mode, the LEDs can be adjusted with potentiometers P1 and P2. P1 is used to adjust the color change of the LED, and P2 is used to adjust the change frequency of LED.



Mode 7- manual remote mode

This mode is manual remote mode and is the default mode. This mode is consistent with the previous project "RF_Remote_Car". Use the joystick to control to move forward, move back and turn left, turn right.

Code

There are 9 (file) labels for this project, 8 of which have appeared in previous projects. In the file "05.4_Multifunctional_RF24_Remote_Car.ino", it is mainly the logical management of each module function, such as switching of the car mode, switching of the LED display mode.

05.4_Multifunctional_RF24_Remote_Car.ino

```

1  #include "Automatic_Tracking_Line_Mode.h"
2  #include "Automatic_Obstacle_Avoidance_Mode.h"
3  #include "Freenove_4WD_Car_for_Arduino.h"
4  #include "Freenove_WS2812B_RGBLED_Controller.h"
5  #include "RF24_Remote.h"
6
7  #define RF_UPDATE_TIMEOUT    1000
8
9  u32 lastRFUpdateTime = 0;
10 u8 RFCarMode = OFF_OFF_OFF, lastRFCarMode = OFF_OFF_OFF;
11 u8 switchModeState = MODE_SWITCHING_WAS_FINISHED;
12 u8 joystickSwitchState = MODE_SWITCHING_WAS_FINISHED;
13
14 #define STRIP_I2C_ADDRESS    0x20
15 #define STRIP_LEDS_COUNT    10
16
17 #define MAX_NUMBER_OF_DISP_MODES  3
18
19 u8 colorPos = 0;
20 u8 colorStep = 50;
21 u8 stripDisplayMode = 1;
22 u8 currentLedIndex = 0;
23 u16 stripDisplayDelay = 100;
24 u32 lastStripUpdateTime = 0;
25 Freenove_WS2812B_Controller strip(STRIP_I2C_ADDRESS, STRIP_LEDS_COUNT, TYPE_GRB);
26
27 void setup() {
28     pinsSetup();
29     if (!RF24L01Setup()) {
30         alarm(4, 2);
31     }
32     servoSetup();
33     while (!strip.begin());
34     strip.setAllLedsColor(0xFF0000);
35 }
36
37 void loop() {

```

```
38     if (getRF24L01Data()) {
39         RFCarMode = updateRFCarMode();
40         if (RFCarMode != lastRFCarMode) {
41             if (switchModeState == MODE_SWITCHING_WAS_FINISHED) {
42                 switchModeState = MODE_SWITCHING_IS_INITIALIZING;
43             }
44             switch (switchModeState)
45             {
46             case MODE_SWITCHING_IS_INITIALIZING:
47                 //Serial.println("Switching Mode Init...");
48                 resetCarAction();
49                 writeServo(90);
50                 switchModeState = MODE_SWITCHING_IS_PROCESSING;
51                 break;
52             case MODE_SWITCHING_IS_PROCESSING:
53                 if (RFDatRead[JOYSTICK_Z] == 0) {
54                     //Serial.println("Switching Mode ...");
55                     setBuzzer(true);
56                     switchModeState = MODE_SWITCHING_IS_CONFIRMING;
57                 }
58                 break;
59             case MODE_SWITCHING_IS_CONFIRMING:
60                 if (RFDatRead[JOYSTICK_Z] == 1) {
61                     //Serial.println("Comfirm Switched.");
62                     setBuzzer(false);
63
64                     switchModeState = MODE_SWITCHING_WAS_FINISHED;
65                     lastRFCarMode = RFCarMode;
66                     switch (RFCarMode)
67                     {
68                     case MODE_OBSTACLE_AVOIDANCE:
69                         oa_CalculateVoltageCompensation();
70                         break;
71                     case MODE_LINE_TRACKING:
72                         tk_CalculateVoltageCompensation();
73                         break;
74                     default:
75                         break;
76                     }
77                     //Serial.println("Switch mode finished !");
78                 }
79                 break;
80             case MODE_SWITCHING_WAS_FINISHED:
81                 break;
```

```

82         default:
83             break;
84     }
85 }
86 else {
87     if (switchModeState != MODE_SWITCHING_WAS_FINISHED) {
88         switchModeState = MODE_SWITCHING_WAS_FINISHED;
89     }
90     switch (RFCarMode)
91     {
92     case ON_ON_ON:
93         break;
94     case ON_ON_OFF:    //S1, S2 ON, S3 OFF
95         setServoOffset(map(nrfDataRead[1], 0, 1023, -20, 20));
96         if (RFDataRead[JOYSTICK_Z] == 0) {
97             setBuzzer(true);
98             writeServoOffsetToEEPROM();
99         }
100        else {
101            setBuzzer(false);
102        }
103        break;
104    case ON_OFF_ON:
105        break;
106    case ON_OFF_OFF:    ////Sonic Obstacle Avoidance Mode, S1 is ON and S2, S3 are
107 OFF
108
109        updateAutomaticObstacleAvoidance();
110        break;
111    case OFF_ON_ON:
112        break;
113    case OFF_ON_OFF:    //Tracking Mode, S2 is ON and S1, S3 are OFF.
114        updateAutomaticTrackingLine();
115        break;
116    case OFF_OFF_ON:    //S3 is ON and S1, S2 are OFF
117        //stripDisplayMode = RFDataRead[POT2] / 512;
118        switch (joystickSwitchState)
119        {
120            static u8 switchCounter = 0;
121            case MODE_SWITCHING_IS_INITIALIZING:
122                if (RFDataRead[JOYSTICK_X] > 900)
123                {
124                    setBuzzer(true);
125                    switchCounter++;
126                    joystickSwitchState = MODE_SWITCHING_IS_PROCESSING;

```

```
126         }
127         else if (RFDataRead[JOYSTICK_X] < 200)
128         {
129             setBuzzer(true);
130             switchCounter--;
131             joystickSwitchState = MODE_SWITCHING_IS_PROCESSING;
132         }
133         break;
134     case MODE_SWITCHING_IS_PROCESSING:
135         if ((RFDataRead[JOYSTICK_X] < 600) && (RFDataRead[JOYSTICK_X] > 400))
136         {
137             setBuzzer(false);
138             joystickSwitchState = MODE_SWITCHING_IS_CONFIRMING;
139         }
140         break;
141     case MODE_SWITCHING_IS_CONFIRMING:
142         stripDisplayMode += switchCounter;
143         if (stripDisplayMode == 0xff)
144         {
145             stripDisplayMode = MAX_NUMBER_OF_DISP_MODES - 1;
146         }
147         stripDisplayMode %= MAX_NUMBER_OF_DISP_MODES;
148         joystickSwitchState = MODE_SWITCHING_WAS_FINISHED;
149         break;
150     case MODE_SWITCHING_WAS_FINISHED:
151         switchCounter = 0;
152         joystickSwitchState = MODE_SWITCHING_IS_INITIALIZING;
153         break;
154     default:
155         break;
156     }
157     break;
158     case OFF_OFF_OFF: //Remote Mode, all switch leds are OFF
159         updateCarActionByRFRemote();
160         break;
161     default:
162         break;
163     }
164 }
165 colorStep = map(RFDataRead[POT1], 0, 1023, 0, 255);
166 stripDisplayDelay = RFDataRead[POT2];
167 lastRFUpdateTime = millis();
168 }
169 }
```

```
170     if (millis() - lastRFUpdateTime > RF_UPDATE_TIMEOUT) {
171         lastRFUpdateTime = millis();
172         resetRFDataBuf();
173         updateCarActionByRFRemote();
174         RFCarMode = lastRFCarMode = MODE_REMOTE_CONTROL;
175     }
176     switch (stripDisplayMode)
177     {
178     case 0:
179         if (millis() - lastStripUpdateTime > stripDisplayDelay)
180         {
181             for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
182                 strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
183             }
184             strip.show();
185             colorPos += colorStep;
186             lastStripUpdateTime = millis();
187         }
188         break;
189     case 1:
190         if (millis() - lastStripUpdateTime > stripDisplayDelay)
191         {
192             strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
193             currentLedIndex++;
194             if (currentLedIndex == STRIP_LEDS_COUNT)
195             {
196                 currentLedIndex = 0;
197                 colorPos += colorStep; //
198             }
199             lastStripUpdateTime = millis();
200         }
201         break;
202     case 2:
203         colorPos = colorStep;
204         if (millis() - lastStripUpdateTime > stripDisplayDelay)
205         {
206             static bool ledState = true;
207             if (ledState)
208             {
209                 strip.setAllLedsColor(strip.Wheel(colorPos));
210             }
211             else
212             {
213                 strip.setAllLedsColor(0x00);
```

```
214         }
215         ledState = !ledState;
216         lastStripUpdateTime = millis();
217     }
218     break;
219 default:
220     break;
221 }
222 }
```

In `setup()`, initialize pins and RF24L01 and LED module. In `loop()`, receive data from the remote control and process the data. If receiving data is time out, the remote control signal is considered to be lost. LED display code is added in this code, which is different from code in the previous project.

```
void setup() {
    pinsSetup();
    if (!RF24L01Setup()) {
        alarm(4, 2);
    }
    servoSetup();
    while (!strip.begin());
    strip.setAllLedsColor(0xFF0000);
}

void loop() {
    if (getRF24L01Data()) {
        .....
    }

    if (millis() - lastRFUpdateTime > RF_UPDATE_TIMEOUT) {
        lastRFUpdateTime = millis();
        resetRFDataBuf();
        updateCarActionByRFRemote();
        RFCarMode = lastRFCarMode = MODE_REMOTE_CONTROL;
    }
    switch (stripDisplayMode)
    {
        .....
    }
}
```

After reading the remote control data, judge whether the mode of the car has changed according to the state of S1, S2, S3. If the mode is changed, the code of the corresponding mode will be executed. Otherwise, the subsequent code will be executed.

```
if (getRF24L01Data()) {
    RFCarMode = updateRFCarMode();
    if (RFCarMode != lastRFCarMode) {
        if (switchModeState == MODE_SWITCHING_WAS_FINISHED) {
            switchModeState = MODE_SWITCHING_IS_INITIALIZING;
        }
        switch (switchModeState)
        {
            .....
        }
        .....
    }
    else {
        .....
    }
}
```

When the state of S1, S2, S3 changes, it means that the user wants to change the mode of the car, but in order to ensure that this change is not an erroneous operation, the user needs to press the Z axis of the joystick to confirm.

This series of actions can be implemented using the idea of a finite state machine. When the switch state changes, the state "MODE_SWITCHING_IS_INITIALIZING" is entered, the car's power module, motor and servo are initialized to prevent the car from running uncontrolledly.

```
case MODE_SWITCHING_IS_INITIALIZING:
    //Serial.println("Switching Mode Init...");
    resetCarAction();
    writeServo(90);
    switchModeState = MODE_SWITCHING_IS_PROCESSING;
    break;
```

Then enter next state "MODE_SWITCHING_IS_PROCESSING" and wait for the user to press the rocker Z axis to confirm. If the user presses the Z axis of the joystick, the buzzer sounds and enters the next state "MODE_SWITCHING_IS_CONFIRMING", waiting for the user to release the joystick. When the user releases the joystick, the buzzer stops sounding and changes the value of the variable switchModeState that holds the car mode. After the mode is switched, the completion status "MODE_SWITCHING_WAS_FINISHED" is entered.

```
case MODE_SWITCHING_IS_PROCESSING:
    if (RFDataRead[JOYSTICK_Z] == 0) {
        //Serial.println("Switching Mode ...");
        setBuzzer(true);
        switchModeState = MODE_SWITCHING_IS_CONFIRMING;
    }
    break;
```

```

case MODE_SWITCHING_IS_CONFIRMING:
    if (RFDataRead[JOYSTICK_Z] == 1) {
        //Serial.println("Confirm Switched.");
        setBuzzer(false);
        switchModeState = MODE_SWITCHING_WAS_FINISHED;
        lastRFCarMode = RFCarMode;
        switch (RFCarMode)
        {
            case MODE_OBSTACLE_AVOIDANCE:
                oa_CalculateVoltageCompensation();
                break;
            case MODE_LINE_TRACKING:
                tk_CalculateVoltageCompensation();
                break;
            default:
                break;
        }
        //Serial.println("Switch mode finished!");
    }
    break;
case MODE_SWITCHING_WAS_FINISHED:
    break;
default:
    break;
}

```

Next, use the switch-case statement to let the car perform the corresponding function or action according to the value of the car mode variable RFCarMode.

```

switch (RFCarMode)
{
case ON_ON_ON:
    break;
case ON_ON_OFF:  //S1,S2 ON, S3 OFF
    .....//Set Servo offset
case ON_OFF_ON:
    break;
case ON_OFF_OFF:  ///Sonic Obstacle Avoidance Mode, S1 is ON and S2, S3 are OFF
    updateAutomaticObstacleAvoidance();
    break;
case OFF_ON_ON:
    break;
case OFF_ON_OFF:  //Tracking Mode, S2 is ON and S1, S3 are OFF.
    updateAutomaticTrackingLine();
    break;
}

```

```

        case OFF_OFF_ON:    //S3 is ON and S1, S2 are OFF
            .....//Change LED Mode
            break;
        case OFF_OFF_OFF:    //Remote Mode, all switch leds are OFF
            updateCarActionByRFRemote();
            break;
        default:
            break;
    }

```

In the calibration servo mode, the angle of the servo is adjusted according to the value of the potentiometer POT2. When the rocker Z-axis is pressed, the adjusted data is written into the EEPROM, and the buzzer will make a sound to confirm the writing.

```

        case ON_ON_OFF:    //S1, S2 ON, S3 OFF
            setServoOffset(RFDataRead[1] / 100);
            if (RFDataRead[JOYSTICK_Z] == 0) {
                setBuzzer(true);
                writeServoOffsetToEEPROM();
            }
            else {
                setBuzzer(false);
            }
            break;

```

In the mode of switching the LED display, the idea of finite state machine is still used.

First, enter the initialization state, waiting for the joystick to move in the direction of the X axis. If there is, the variable switchCounter will be self-added or self-decremented according to the positive and negative directions. Then enter next state, wait for the joystick to return to the middle. When the joystick returns to the middle, enter the next state, process the data, after the data processing is completed, enter the completion state.

```

    switch (joystickSwitchState)
    {
        static u8 switchCounter = 0;
        case MODE_SWITCHING_IS_INITIALIZING:
            if (RFDataRead[JOYSTICK_X] > 900)
            {
                setBuzzer(true);
                switchCounter++;
                joystickSwitchState = MODE_SWITCHING_IS_PROCESSING;
            }
            else if (RFDataRead[JOYSTICK_X] < 200)
            {
                setBuzzer(true);
                switchCounter--;
            }

```

```

        joystickSwitchState = MODE_SWITCHING_IS_PROCESSING;
    }
    break;
case MODE_SWITCHING_IS_PROCESSING:
    if ((RFDataRead[JOYSTICK_X] < 600) && (RFDataRead[JOYSTICK_X] > 400))
    {
        setBuzzer(false);
        joystickSwitchState = MODE_SWITCHING_IS_CONFIRMING;
    }
    break;
case MODE_SWITCHING_IS_CONFIRMING:
    stripDisplayMode += switchCounter;
    if (stripDisplayMode == 0xff)
    {
        stripDisplayMode = MAX_NUMBER_OF_DISP_MODES - 1;
    }
    stripDisplayMode %= MAX_NUMBER_OF_DISP_MODES;
    joystickSwitchState = MODE_SWITCHING_WAS_FINISHED;
    break;
case MODE_SWITCHING_WAS_FINISHED:
    switchCounter = 0;
    joystickSwitchState = MODE_SWITCHING_IS_INITIALIZING;
    break;
default:
    break;
}
break;

```

The variable that controls the LED display color change and the variable of the display frequency are updated every time according to remote control data.

```

colorStep = map(RFDataRead[POT1], 0, 1023, 0, 255);
stripDisplayDelay = RFDataRead[POT2];

```

Finally, the LED display mode variable stripDisplayMode, the color change amplitude variable colorStep, the display period variable stripDisplayDelay.

```

switch (stripDisplayMode)
{
case 0:
    if (millis() - lastStripUpdateTime > stripDisplayDelay)
    {
        for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
            strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
        }
        strip.show();
    }
}

```

```
        colorPos += colorStep;
        lastStripUpdateTime = millis();
    }
    break;
case 1:
    if (millis() - lastStripUpdateTime > stripDisplayDelay)
    {
        strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
        currentLedIndex++;
        if (currentLedIndex == STRIP_LEDS_COUNT)
        {
            currentLedIndex = 0;
            colorPos += colorStep; //
        }
        lastStripUpdateTime = millis();
    }
    break;
case 2:
    colorPos = colorStep;
    if (millis() - lastStripUpdateTime > stripDisplayDelay)
    {
        static bool ledState = true;
        if (ledState)
        {
            strip.setAllLedsColor(strip.Wheel(colorPos));
        }
        else
        {
            strip.setAllLedsColor(0x00);
        }
        ledState = !ledState;
        lastStripUpdateTime = millis();
    }
    break;
default:
    break;
}
```

Chapter 6 Bluetooth control

This section requires an Android or iPhone device with Bluetooth to control the car.
If you have any concerns, please feel free to contact us via support@freenove.com

6.1 Set Bluetooth and Receive Data

Android and iPhone iOS App

Download and install app. You can use app to control the robot.

You can download Freenove App through below ways:

View or download on Google Play:

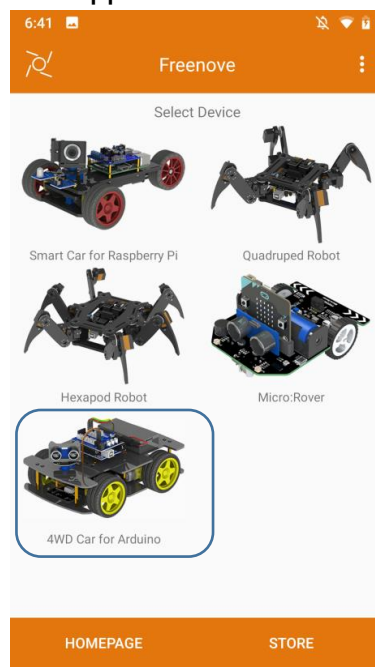
<https://play.google.com/store/apps/details?id=com.freenove.suhayl.Freenove>

Or download APK file directly

https://github.com/Freenove/Freenove_App_for_Android/raw/master/freenove.apk

Then install it on your Android phone.

For **iPhone app**, please search **freenove** in **App Store**.



Instructions for using the Bluetooth module

The Bluetooth module uses the serial port to communicate with the Arduino. The serial communication is also used to upload the program to Arduino. **Therefore, when uploading the program to Arduino, the Bluetooth module should be unplugged; otherwise, the program upload will fail!**

When the Bluetooth module is not connected by other device, the Bluetooth module can be configured using the AT command. Once connected, the Bluetooth module acts as a data pipe and cannot be configured.

By default, the Bluetooth module has a baud rate of 115200, no parity, 8 data bits, and 1 stop bit. Bluetooth name "BT05", role mode is slave mode.

Set name of bluetooth

If you have multiple Bluetooth modules with the same name around you, you will be confused when you connect. Which one is the Bluetooth module I want to connect to?

In the next project, we will introduce how to use the AT command to modify the name of the Bluetooth module and the master-slave role in the program.

For AT commands and more information about the Bluetooth module, refer to the documentation in the package for Datasheets/BT05-Instruction.pdf

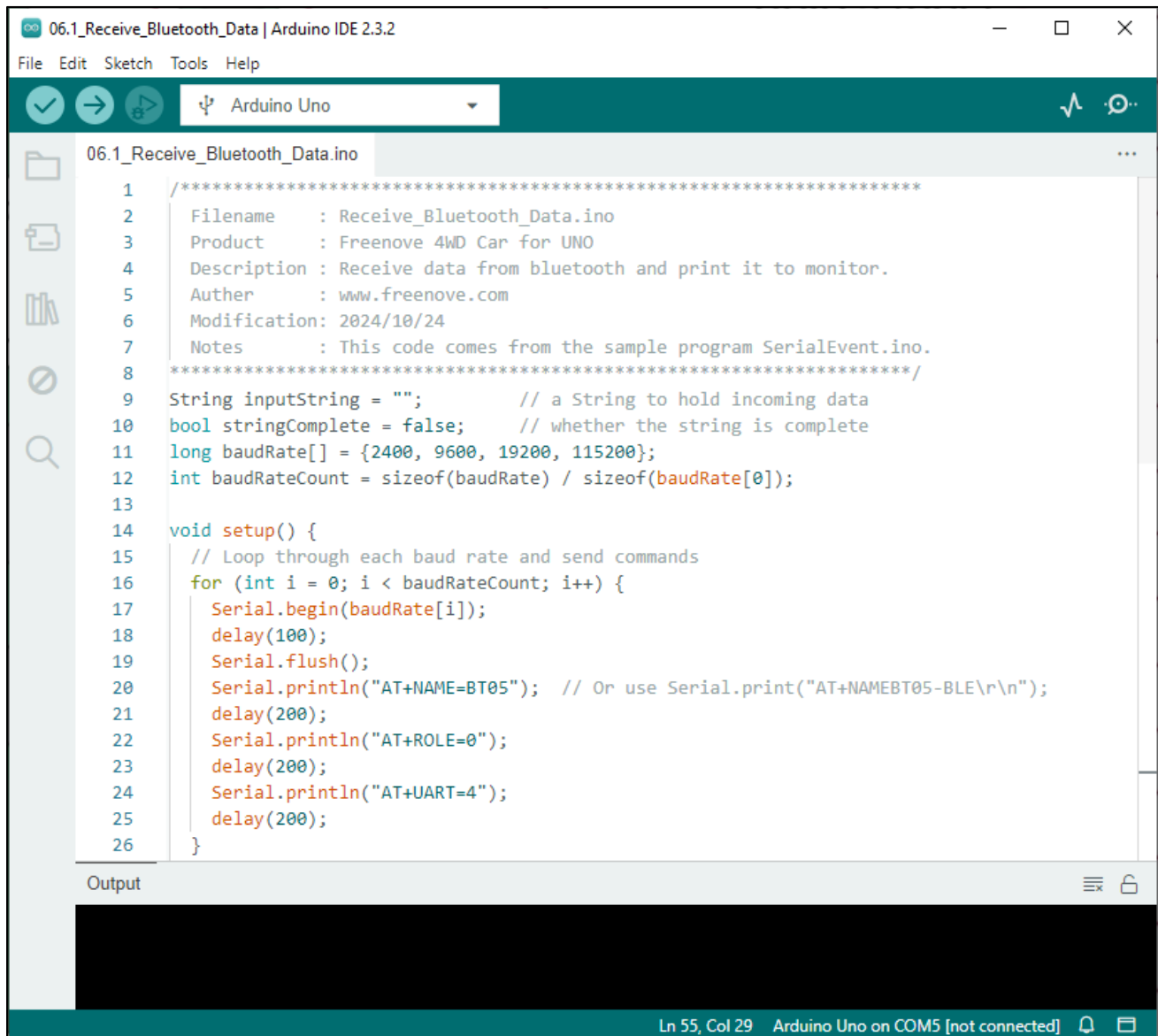
This project uses the AT command to make some settings for the Bluetooth module, then uses the Bluetooth module to receive data from the app and print the data on the serial monitor.

Upload Code and Running

You need remove Bluetooth Module first when you upload code.

Then upload code in Sketches/06.1_Receive_Bluetooth_Data.ino.

Note, when upload code, we need unplug Bluetooth first. Or the uploading will fail.



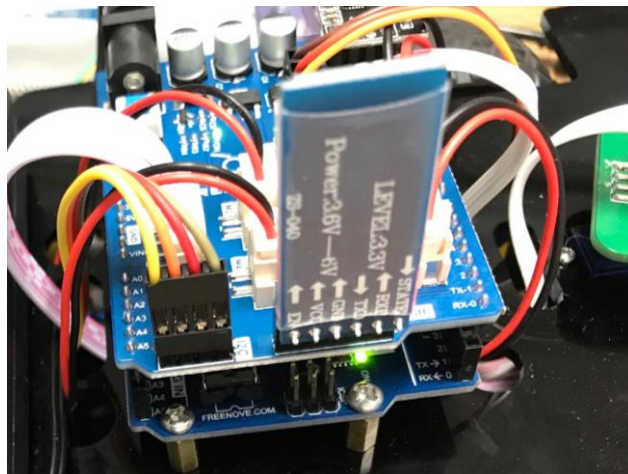
```
06.1_Receive_Bluetooth_Data | Arduino IDE 2.3.2
File Edit Sketch Tools Help
Arduino Uno

06.1_Receive_Bluetooth_Data.ino
1  /*****
2  Filename   : Receive_Bluetooth_Data.ino
3  Product    : Freenove 4WD Car for UNO
4  Description: Receive data from bluetooth and print it to monitor.
5  Author     : www.freenove.com
6  Modification: 2024/10/24
7  Notes      : This code comes from the sample program SerialEvent.ino.
8  *****/
9  String inputString = "";           // a String to hold incoming data
10 bool stringComplete = false;        // whether the string is complete
11 long baudRate[] = {2400, 9600, 19200, 115200};
12 int baudRateCount = sizeof(baudRate) / sizeof(baudRate[0]);
13
14 void setup() {
15     // Loop through each baud rate and send commands
16     for (int i = 0; i < baudRateCount; i++) {
17         Serial.begin(baudRate[i]);
18         delay(100);
19         Serial.flush();
20         Serial.println("AT+NAME=BT05"); // Or use Serial.print("AT+NAMEBT05-BLE\r\n");
21         delay(200);
22         Serial.println("AT+ROLE=0");
23         delay(200);
24         Serial.println("AT+UART=4");
25         delay(200);
26     }
}

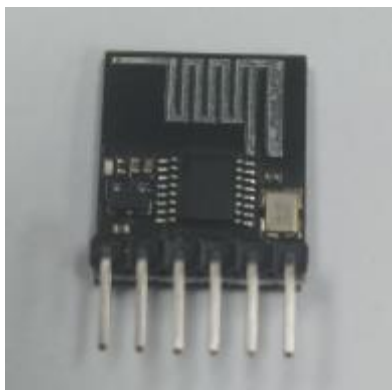
Output
Ln 55, Col 29  Arduino Uno on COM5 [not connected]
```

After the code is successfully uploaded, follow the steps below.

1. Plug the Bluetooth module on car as shown below. **Don't reverse it. The wrong connection may damage your hardware.**



Please note: your Bluetooth module may also be this way. They are universal.

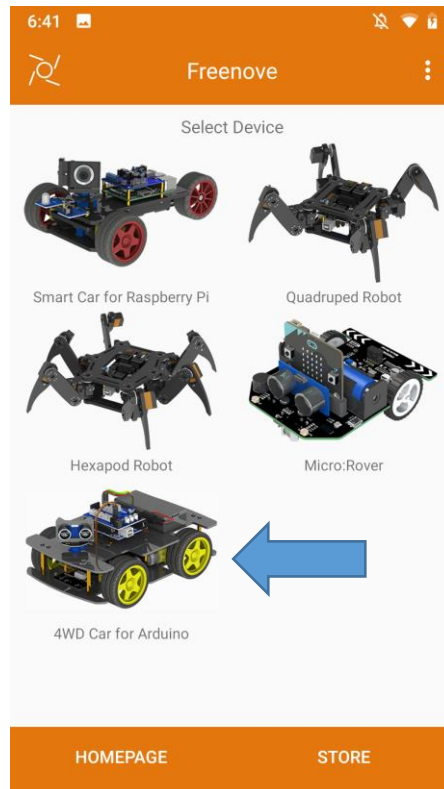


2. Open Arduino monitor. Reset uno board.

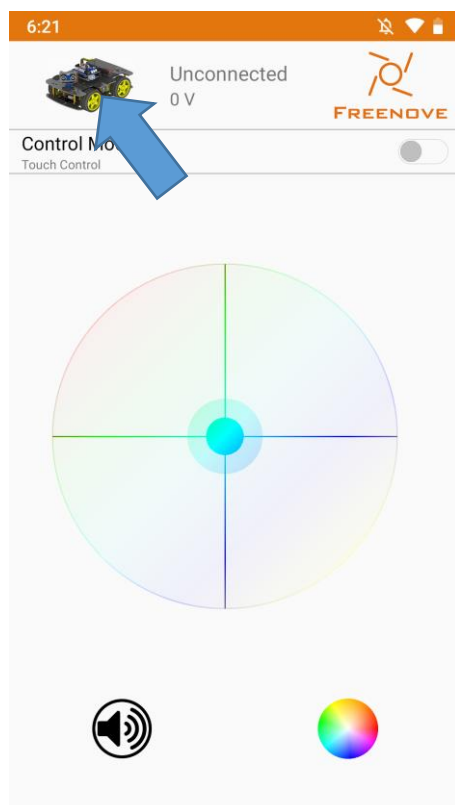
As shown in the figure below, the Bluetooth name is set to "BT05" and the Bluetooth mode is slave mode (ROLE=0).

```
COM4
11:15:10.099 -> AT+NAMEBT05-GEN
11:15:10.287 -> AT+NAMBBT05-BLE
11:15:10.473 -> AT+ROLE0
11:15:10.708 -> +OK
11:15:10.708 -> +OK
11:15:10.708 ->
```

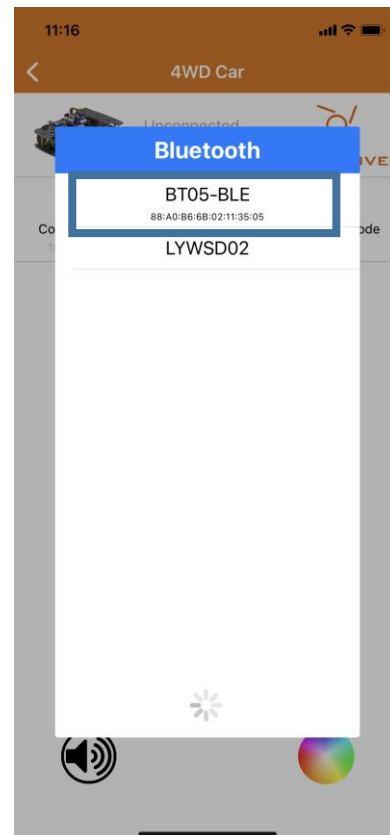
Next, open the app and click the 4WD Car for Arduino.



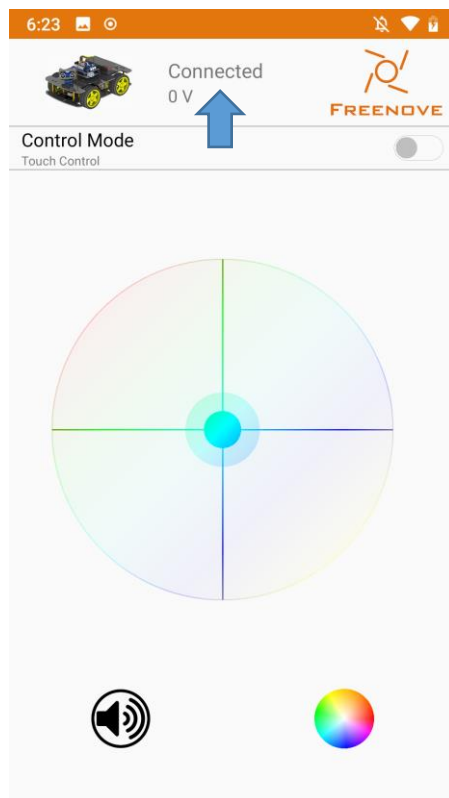
Click following icon.



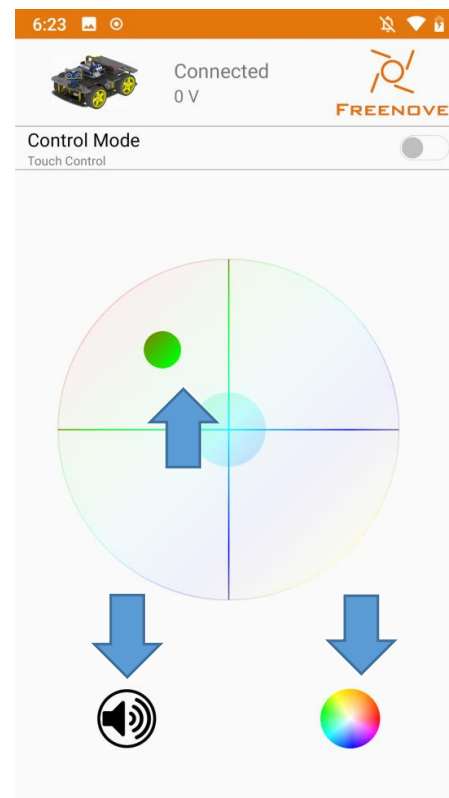
Click BT05.



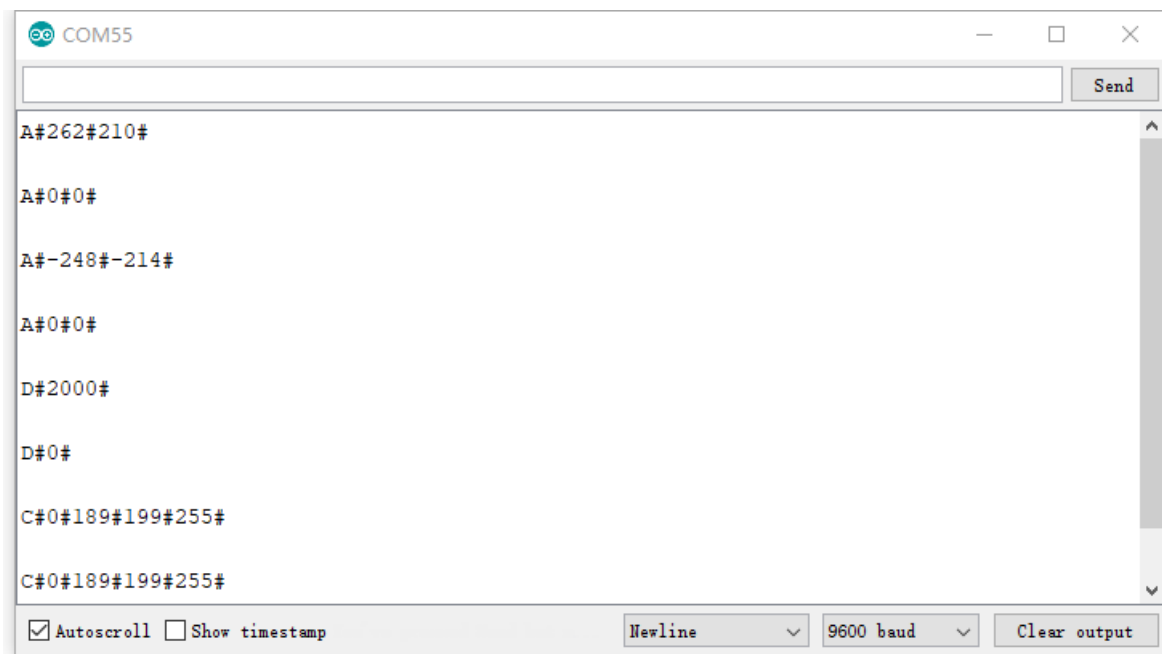
If the connection succeeds, it will be connected.



Operate the app. And observe the monitor.



The monitor will show content below:



Bluetooth data- car action

The command format for communication between app and car is A#xxx#xxx#...xxx#, where # is a separator, the first character A represents the action command, it can be other characters, such as B, C, D...The xxx represents the parameters of the action command. And different commands carry different parameters. The command list is as below:

Action command	Description	Command character	Number of parameters (app send/receive)	Format example (app send/receive)
MOVE	Move, parameters are speeds of the two motors	A	2	A#100#100#
STOP	Stop moving	B	0	B#
LED_RGB	Control RGBLED, parameters respectively are serial number of LED mode, red value, green value, blue value.	C	3	C#2#100#150#200#
BUZZER	Control buzzer, and parameter is the frequency.	D	1	D#2000#
VOLTAGE	Get the batteries voltage, parameter is the voltage value, unit mV	I	0/1	I# /I#4100#

Code

06.1_Receive_Bluetooth_Data

The data of the Bluetooth module is sent to the Arduino through the serial port. The project code comes from the Arduino example code "SerialEvent". Receive data from the serial port and print the data when '\n' is received.

The code is below:

```

1  String inputString = "";           // a String to hold incoming data
2  bool stringComplete = false;      // whether the string is complete
3  long baudRate[] = {2400, 9600, 19200, 115200};
4  int baudRateCount = sizeof(baudRate) / sizeof(baudRate[0]);
5
6  void setup() {
7      // Loop through each baud rate and send commands
8      for (int i = 0; i < baudRateCount; i++) {
9          Serial.begin(baudRate[i]);
10         delay(100);
11         Serial.flush();
12         Serial.println("AT+NAME=BT05"); // Or use Serial.print("AT+NAME=BT05-BLE\r\n");
13         delay(200);
14         Serial.println("AT+ROLE=0");

```

```
15     delay(200);
16     Serial.println("AT+UART=4");
17     delay(200);
18 }
19
20 // Finally, set the serial to the desired baud rate (e.g., 115200)
21 Serial.begin(115200);
22
23 // reserve 200 bytes for the inputString:
24 inputString.reserve(200);
25 }
26
27 void loop() {
28     // print the string when a newline arrives:
29     if (stringComplete) {
30         Serial.println(inputString);
31         // clear the string:
32         inputString = "";
33         stringComplete = false;
34     }
35 }
36
37 /*
38  SerialEvent occurs whenever a new data comes in the hardware serial RX. This
39  routine is run between each time loop() runs, so using delay inside loop can
40  delay response. Multiple bytes of data may be available.
41  */
42 void serialEvent() {
43     while (Serial.available()) {
44         // get the new byte:
45         char inChar = (char)Serial.read();
46         // add it to the inputString:
47         inputString += inChar;
48         // if the incoming character is a newline, set a flag so the main loop can
49         // do something about it:
50         if (inChar == '\n') {
51             stringComplete = true;
52         }
53     }
54 }
```

The default data transmission baud rate of the Bluetooth module is 115200, so set serial port baud rate to 115200.

```
Serial.begin(115200); //default data transmission baud rate of the Bluetooth module is 9600
```

Then use the AT command to set the name of the Bluetooth module to "BT05". The format of the command is "AT+NAME=xxx\r\n", where "AT+NAME" is a fixed format, and the following "xxx" is the set name. The maximum length of the name is 18 characters. The command must be followed by "\r\n" ends. Since the Serial.println() function adds "\r\n" to the end, there is no need to add it in the program. The delay is 200ms to ensure that the Bluetooth module has enough time to complete the setup.

```
Serial.println("AT+NAME=BT05"); // Or use Serial.print("AT+NAME=BT05-BLE\r\n");
delay(200);
```

Similarly, continue to use the AT command to set the Bluetooth module's role mode to slave mode. The Bluetooth module can be set to master mode or slave mode. In master mode, the Bluetooth module can actively search for and connect to other Bluetooth devices, but cannot be searched and connected by other devices. In order to be searched by the mobile app via Bluetooth, the Bluetooth role module needs to be set to the slave mode. According to the Bluetooth manufacturer's manual, 0 is the slave mode and 1 is the master mode.

```
Serial.println("AT+ROLE0");
delay(200);
```

In loop(), keep querying if stringComplete is true. If it is true, it indicates that the complete data has been received. The data will be printed on the serial monitor, and then the data is cleared. And then the array is ready to receive new data.

```
if (stringComplete) {
    Serial.println(inputString);
    // clear the string:
    inputString = "";
    stringComplete = false;
}
```

Function serialEvent() is used to receive the data transmitted by the serial port. When the newline character is received, the flag stringComplete is set to true, and the received data will be printed in loop().

```
void serialEvent() {
    while (Serial.available()) {
        // get the new byte:
        char inChar = (char)Serial.read();
        // add it to the inputString:
        inputString += inChar;
        // if the incoming character is a newline, set a flag so the main loop can
        // do something about it:
        if (inChar == '\n') {
            stringComplete = true;
        }
    }
}
```

Bluetooth AT Command

AT command mode when the module is not connected.

AT command, which belongs to the character line instruction, is parsed according to the line (That is, AT command must be returned by carriage return or `\r\n`, hexadecimal number is 0D0A). For more details about AT command, please refer to Datasheets/BT05-Instruction.pdf.

void serialEvent()

SerialEvent occurs whenever a new data comes in the hardware serial RX. This routine is run between each time `loop()` runs, so using `delay` inside `loop` can delay response. Multiple bytes of data may be available.

6.2 Bluetooth Remote Car

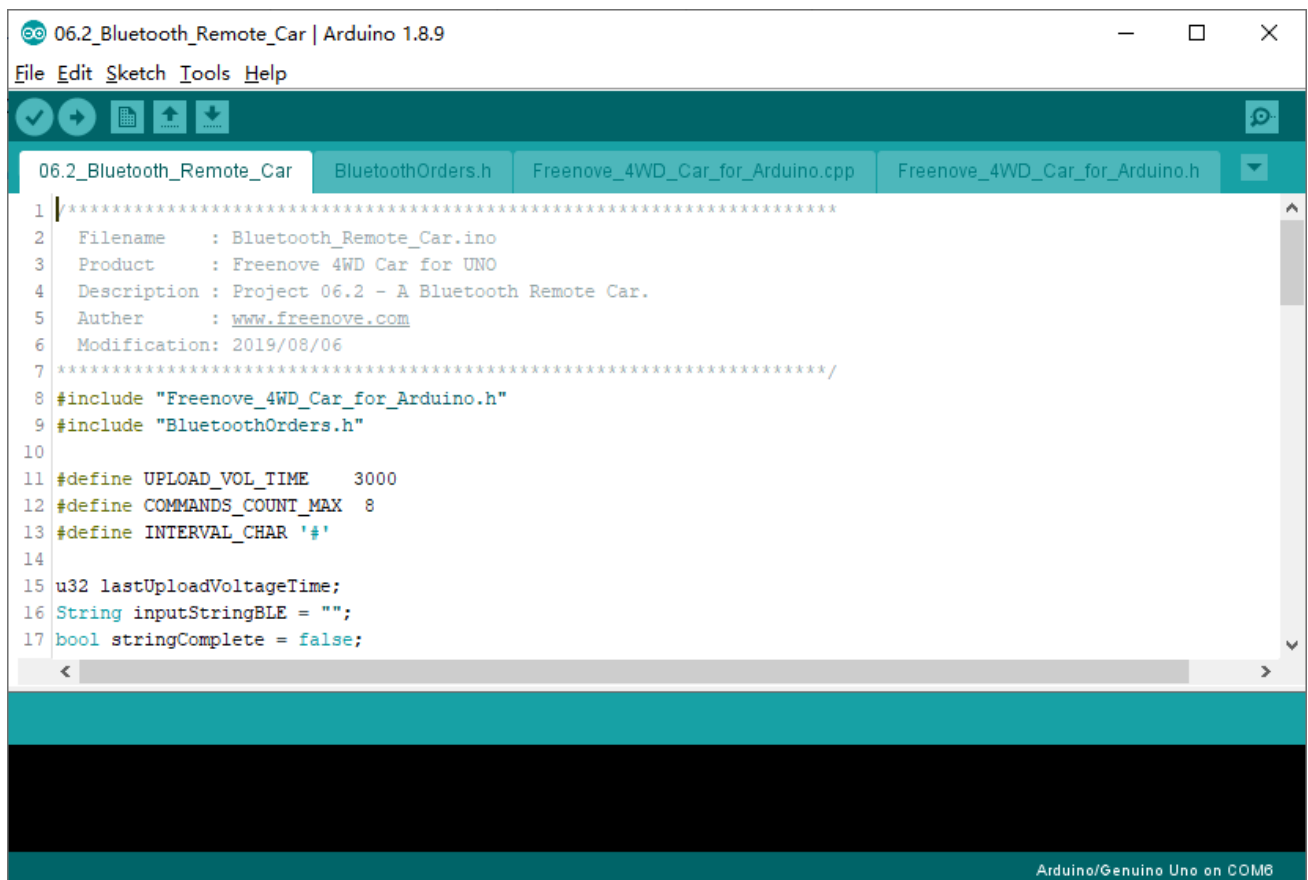
After learning how to receive the app's data via Bluetooth and the meaning of data formats, use the data to let the car do something. Control the car movement, the buzzer sounds, and the LED switches the display mode.

Upload Code and Running

You need remove Bluetooth Module first when you upload code.

And connect car to computer with USB cable.

Then upload code in Sketches/06.2_Bluetooth_Remote_Car.ino.



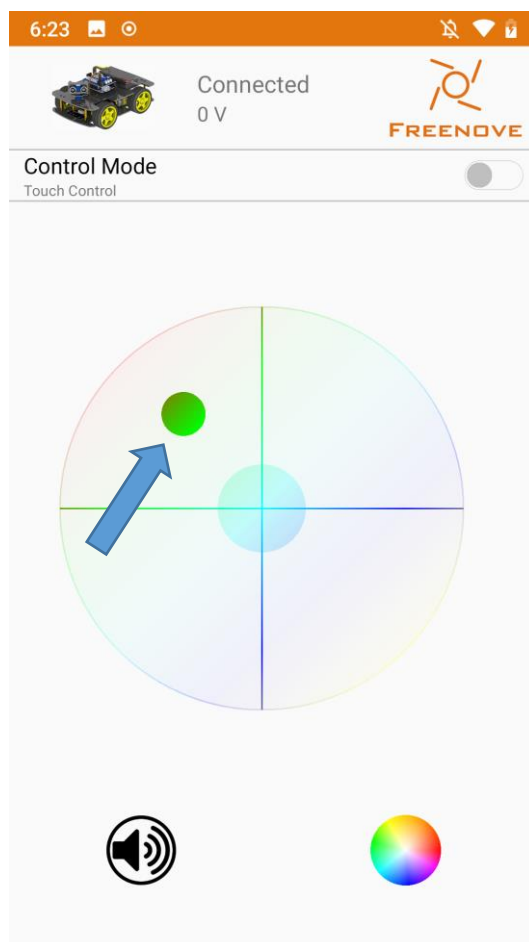
```
06.2_Bluetooth_Remote_Car | Arduino 1.8.9
File Edit Sketch Tools Help

06.2_Bluetooth_Remote_Car BluetoothOrders.h Freenove_4WD_Car_for_Arduino.cpp Freenove_4WD_Car_for_Arduino.h
1 *****
2 Filename : Bluetooth_Remote_Car.ino
3 Product : Freenove 4WD Car for UNO
4 Description : Project 06.2 - A Bluetooth Remote Car.
5 Author : www.freenove.com
6 Modification: 2019/08/06
7 *****/
8 #include "Freenove_4WD_Car_for_Arduino.h"
9 #include "BluetoothOrders.h"
10
11 #define UPLOAD_VOL_TIME 3000
12 #define COMMANDS_COUNT_MAX 8
13 #define INTERVAL_CHAR '#'
14
15 u32 lastUploadVoltageTime;
16 String inputStringBLE = "";
17 bool stringComplete = false;
```

Arduino/Genuino Uno on COM6

After the code is successfully uploaded, unplug the USB cable and plug the Bluetooth module on car. Turn on the car power switch.

According to the previous way, the Bluetooth module of the car is connected through app, then the car can be controlled by clicking or sliding the operation panel on the APP.



Code

The project has 4 labels, the label "BluetoothOrders.h" stores Bluetooth communication, and command characters are used to control the car, only a part of which is used. The main label "06.2_Bluetooth_Remote_Car.ino" is the main content of this project.

BluetoothOrders.h

```

1  #ifndef _BLUETOOTHORDERS_h
2  #define _BLUETOOTHORDERS_h
3
4  #define ACTION_MOVE    'A'
5  #define ACTION_STOP    'B'
6  #define ACTION_RGB     'C'
7  #define ACTION_BUZZER  'D'
8  #define ACTION_ULTRASONIC 'E'
9  #define ACTION_LIGHT_TRACING 'F'
10 #define ACTION_TRACKING 'G'
11 #define ACTION_CAR_MODE 'H'

```

```
12 #define ACTION_GET_VOLTAGE 'I'
13 #define ECHO_OK 'J'
14 #define ACTION_NONE 'K'
15
16 #endif
```

06.2_Bluetooth_Remote_Car.ino

```
1 #include "Freenove_4WD_Car_for_Arduino.h"
2 #include "BluetoothOrders.h"
3
4 #define UPLOAD_VOL_TIME 3000
5 #define COMMANDS_COUNT_MAX 8
6 #define INTERVAL_CHAR '#'
7
8 u32 lastUploadVoltageTime;
9 String inputStringBLE = "";
10 bool stringComplete = false;
11
12 void setup() {
13     pinsSetup();
14     Serial.begin(9600);
15 }
16
17
18 void loop() {
19     if (millis() - lastUploadVoltageTime > UPLOAD_VOL_TIME) {
20         upLoadVoltageToApp();
21         lastUploadVoltageTime = millis();
22     }
23     if (stringComplete) {
24         String inputCommandArray[COMMANDS_COUNT_MAX];
25         int paramters[COMMANDS_COUNT_MAX], paramterCount = 0;
26         String inputStringTemp = inputStringBLE;
27         for (u8 i = 0; i < COMMANDS_COUNT_MAX; i++) {
28             int index = inputStringTemp.indexOf(INTERVAL_CHAR);
29             if (index < 0) {
30                 break;
31             }
32             paramterCount = i; //
33             inputCommandArray[i] = inputStringTemp.substring(0, index);
34             inputStringTemp = inputStringTemp.substring(index + 1);
35             paramters[i] = inputCommandArray[i].toInt();
36         }
37         stringComplete = false;
```

```
38     inputStringBLE = "";
39
40     char commandChar = inputCommandArray[0].charAt(0);
41     switch (commandChar)
42     {
43     case ACTION_MOVE:
44         if (paramterCount == 2) {
45             motorRun(paramters[1], paramters[2]);
46         }
47         break;
48     case ACTION_BUZZER:
49         if (paramterCount == 1) {
50             setBuzzer(paramters[1]);
51         }
52         break;
53     default:
54         break;
55     }
56 }
57
58
59 void upLoadVoltageToApp() {
60     int voltage = 0;
61     if (getBatteryVoltage()) {
62         voltage = batteryVoltage * 1000;
63     }
64     String sendString = String(ACTION_GET_VOLTAGE) + String(INTERVAL_CHAR) +
65 String((voltage)) + String(INTERVAL_CHAR);
66     Serial.println(sendString);
67 }
68
69 void serialEvent() {
70     while (Serial.available()) {
71         char inChar = (char)Serial.read();
72         inputStringBLE += inChar;
73         if (inChar == '\n') {
74             stringComplete = true;
75         }
76     }
77 }
```

In loop (), the voltage value is uploaded to app at intervals.

```
if (millis() - lastUploadVoltageTime > UPLOAD_VOL_TIME) {
    upLoadVoltageToApp();
    lastUploadVoltageTime = millis();
}
```

In the sub-function upLoadVoltageToApp(), first read the battery voltage, then convert the voltage unit to mv, and send it in the format of "l#xxx#". Use Serial.println() to send a Newline character '\n', after sending the previous data,

```
void upLoadVoltageToApp() {
    int voltage = 0;
    if (getBatteryVoltage()) {
        voltage = batteryVoltage * 1000;
    }
    String sendString = String(ACTION_GET_VOLTAGE) + String(INTERVAL_CHAR) +
String((voltage)) + String(INTERVAL_CHAR);
    Serial.println(sendString);
}
```

Then in loop(), if the required data is received, the data will be parsed into commands and parameters and will be saved in an array.

Suppose you receive a command string of "A#100#200#". First use the String.indexOf() function to find the position of the separator "#". And then use the String.substring() function to split the command string into "A" and "100#200#", and save the first element "A" that is split in the array "inputCommandArray".

And then continue to repeat this operation on the split string "100#200#". You can split the element "A" into "100" and "200" respectively and save them in arrays. Finally, convert these elements to integer types and save them in the parameter array paramters.

```
if (stringComplete) {
    String inputCommandArray[COMMANDS_COUNT_MAX];
    int paramters[COMMANDS_COUNT_MAX], paramterCount = 0;
    String inputStringTemp = inputStringBLE;
    for (u8 i = 0; i < COMMANDS_COUNT_MAX; i++) {
        int index = inputStringTemp.indexOf(INTERVAL_CHAR);
        if (index < 0) {
            break;
        }
        paramterCount = i; //
        inputCommandArray[i] = inputStringTemp.substring(0, index);
        inputStringTemp = inputStringTemp.substring(index + 1);
        paramters[i] = inputCommandArray[i].toInt();
    }
    .....
}
```

Finally, let the car perform different actions according to the commands and parameters.

```
char commandChar = inputCommandArray[0].charAt(0);
switch (commandChar)
{
case ACTION_MOVE:
    if (paramterCount == 2) {
        motorRun(parameters[1], parameters[2]);
    }
    break;
case ACTION_BUZZER:
    if (paramterCount == 1) {
        setBuzzer(parameters[1]);
    }
    break;
default:
    break;
}
```

indexOf()

Description

Locates a character or String within another String. By default, it searches from the beginning of the String, but it can also start from a given index, allowing for the locating of all instances of the character or String.

Syntax

myString.indexOf(val)

myString.indexOf(val, from)

Parameters

myString: a variable of type String.

val: the value to search for. Allowed data types: char, String.

from: the index to start the search from.

Returns

The index of val within the String, or -1 if not found.

substring()

Description

Get a substring of a String. The starting index is inclusive (the corresponding character is included in the substring), but the optional ending index is exclusive (the corresponding character is not included in the substring). If the ending index is omitted, the substring continues to the end of the String.

Syntax

myString.substring(from)

myString.substring(from, to)

Parameters

myString: a variable of type String.

from: the index to start the substring at.

to (optional): the index to end the substring before.

Returns

The substring.

toInt()**Description**

Converts a valid String to an integer. The input String should start with an integer number. If the String contains non-integer numbers, the function will stop performing the conversion.

Syntax

```
myString.toInt()
```

Parameters

myString: a variable of type String.

Returns

If no valid conversion could be performed because the String doesn't start with an integer number, a zero is returned. Data type: long.

More information refer to: <https://www.arduino.cc/reference/en/language/variables/data-types/stringobject/>

6.3 Multifunctional Bluetooth Remote Car

This project adds the function of LED display and the function of LED display mode switching on the basis of the previous project.

Upload Code and Running

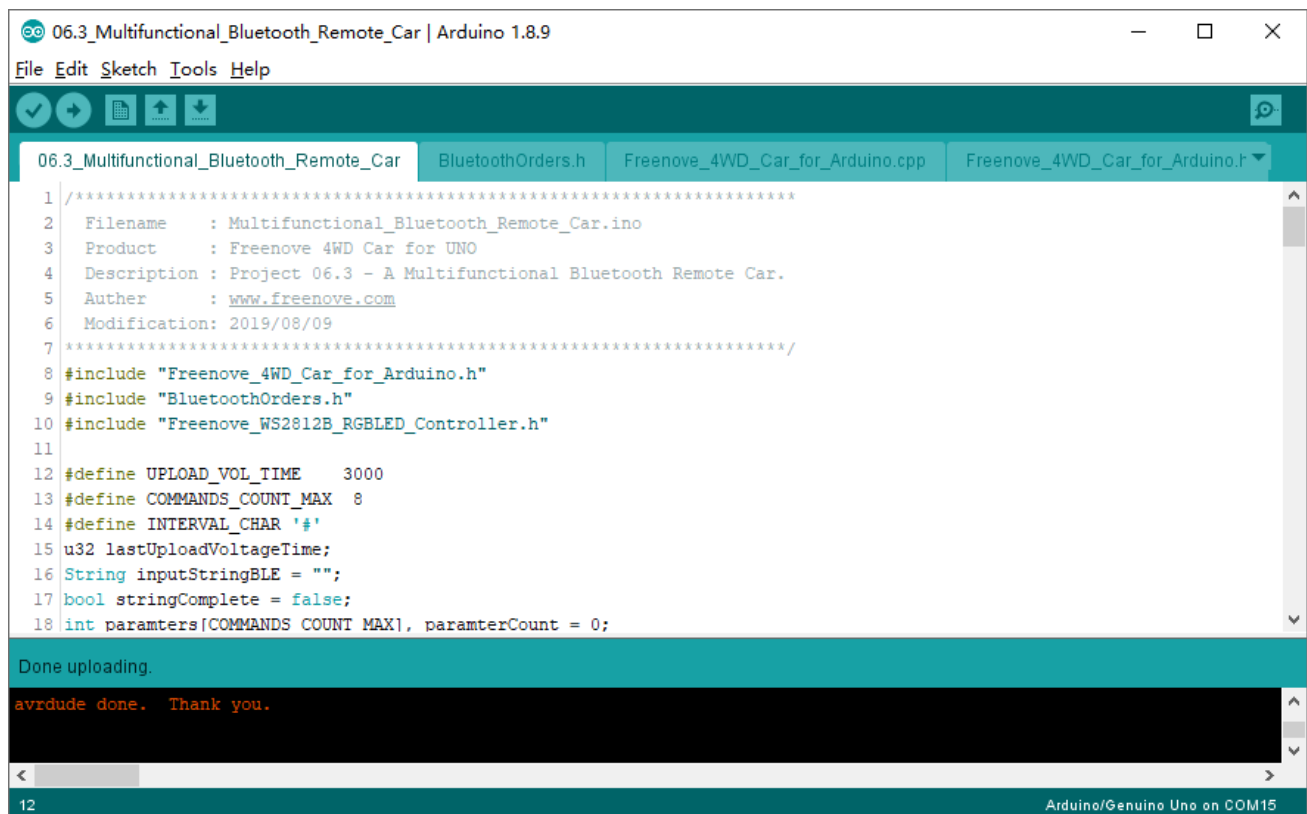
Connect car to computer with USB cable.

1 You need remove Bluetooth Module first when you upload code.

2 Then upload the code in **Sketches/06.4_One_Code_Multifunctional_Bluetooth_Remote_Car**.

3 After upload, reconnect the Bluetooth and reset the uno board.

And you can also upload code in Sketches/06.3_Multifunctional_Bluetooth_Remote_Car.ino. The code is same. Don't separate the files in the folder.



```

06.3_Multifunctional_Bluetooth_Remote_Car | Arduino 1.8.9
File Edit Sketch Tools Help
06.3_Multifunctional_Bluetooth_Remote_Car BluetoothOrders.h Freenove_4WD_Car_for_Arduino.cpp Freenove_4WD_Car_for_Arduino.h
1 /*****
2  Filename   : Multifunctional_Bluetooth_Remote_Car.ino
3  Product    : Freenove 4WD Car for UNO
4  Description : Project 06.3 - A Multifunctional Bluetooth Remote Car.
5  Author     : www.freenove.com
6  Modification: 2019/08/09
7 *****/
8 #include "Freenove_4WD_Car_for_Arduino.h"
9 #include "BluetoothOrders.h"
10 #include "Freenove_WS2812B_RGBLED_Controller.h"
11
12 #define UPLOAD_VOL_TIME    3000
13 #define COMMANDS_COUNT_MAX  8
14 #define INTERVAL_CHAR '#'
15 u32 lastUploadVoltageTime;
16 String inputStringBLE = "";
17 bool stringComplete = false;
18 int paramters[COMMANDS_COUNT_MAX], paramterCount = 0;

```

Done uploading.

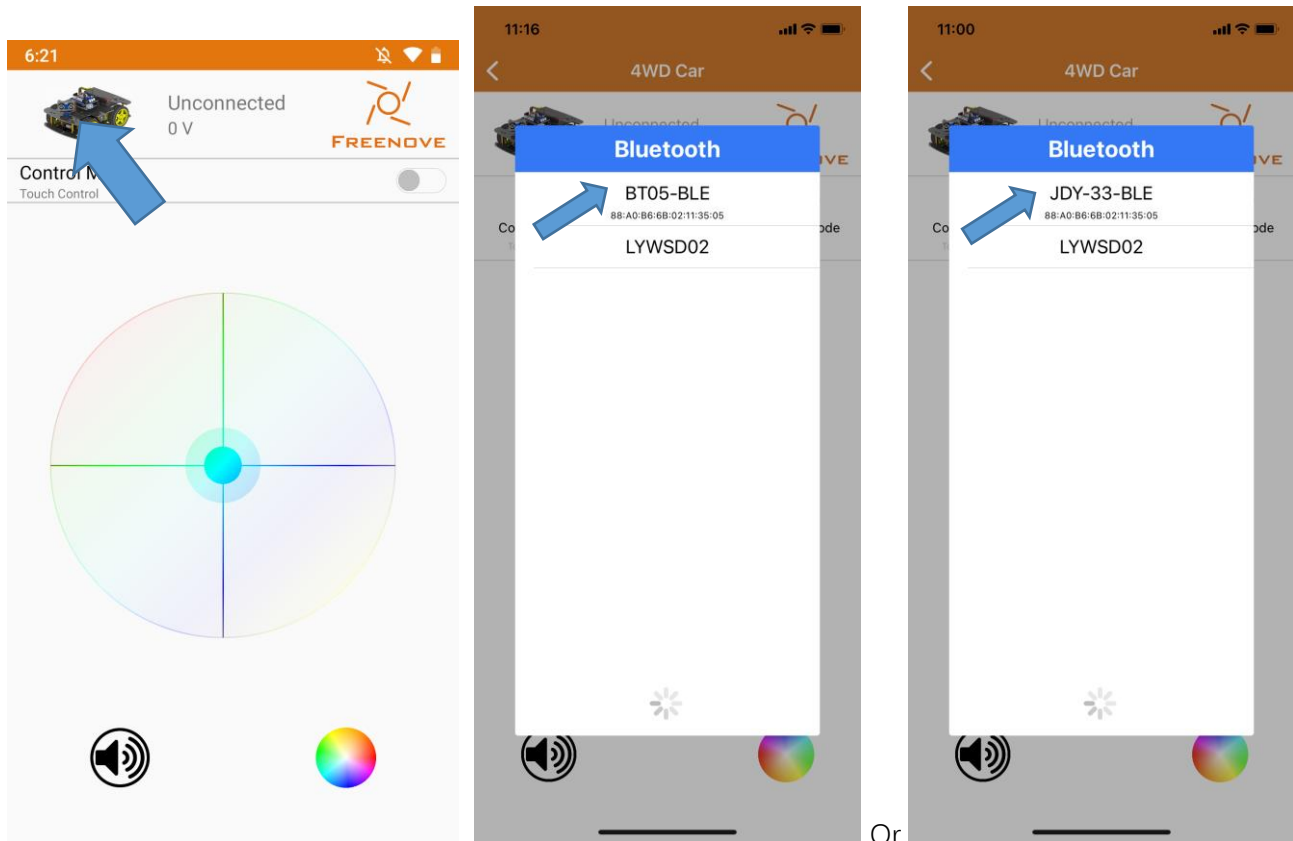
avrdude done. Thank you.

12 Arduino/Genuino Uno on COM15

After the code is successfully uploaded, unplug the USB cable and plug the Bluetooth module on car. Turn on the car power switch.

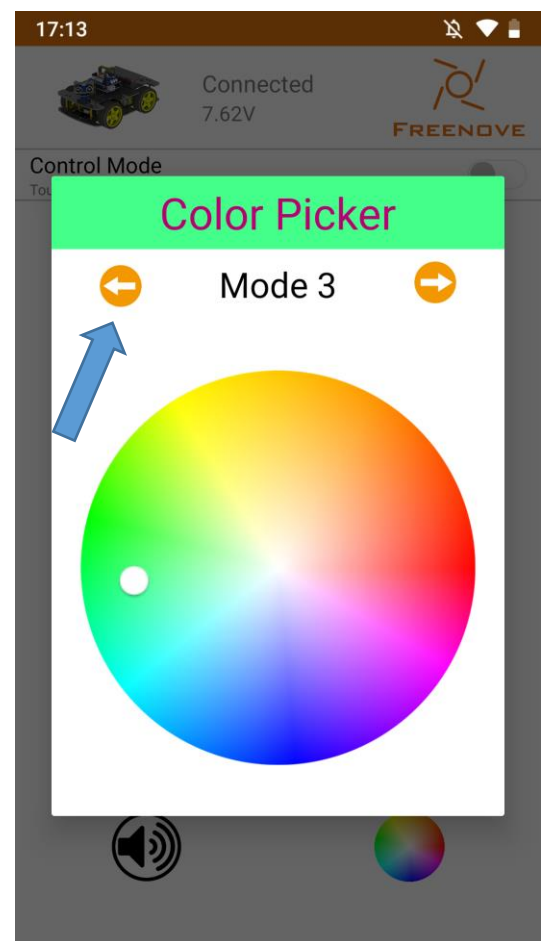
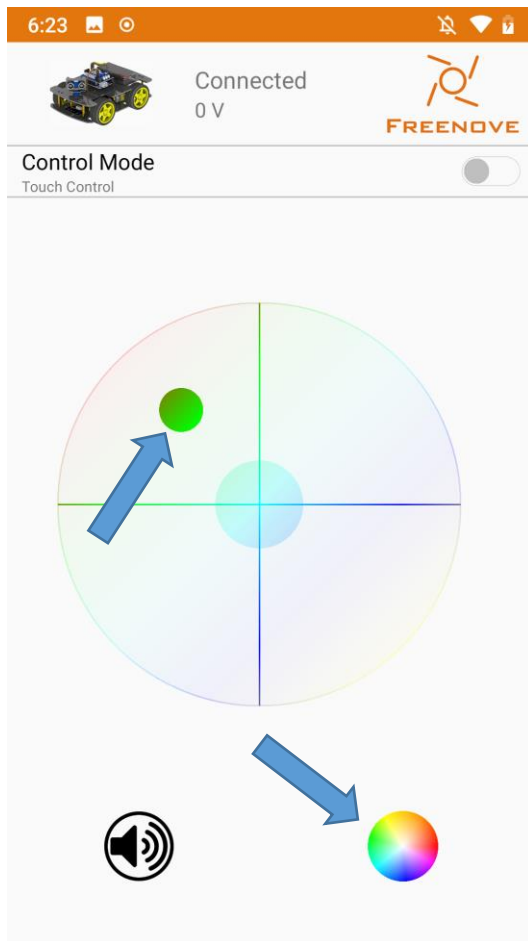
Click following icon.

Click BT05 or JDY-33-BLE (Default name).



According to the previous method, connect Bluetooth module of the car with the app, and the operation panel of the APP can be clicked or swiped to control the movement of the car. And the color palette of the lower right corner can be clicked to control the display mode and color of the LED. Among them,

- Mode 0 is a flowing rainbow,
- Mode 1 is a flowing water LED with color changing.
- Mode 2 is a color-adjustable Blink.
- Mode 3 displays the selected color of the current color picker for all LEDs.



Code

The code for this project is based on the previous engineering code, adding code related to LED control. Nothing else has changed.

06.3_Multifunctional_Bluetooth_Remote_Car

```

1  #include "Freenove_4WD_Car_for_Arduino.h"
2  #include "Automatic_Obstacle_Avoidance_Mode.h"
3  #include "Automatic_Tracking_Line_Mode.h"
4  #include "BluetoothOrders.h"
5  #include "Freenove_WS2812B_RGBLED_Controller.h"
6
7  #define UPLOAD_VOL_TIME    3000
8  #define COMMANDS_COUNT_MAX  8
9  #define INTERVAL_CHAR  '#'
10 u32 lastUploadVoltageTime;
11 String inputStringBLE = "";
12 bool stringComplete = false;
13 int paramters[COMMANDS_COUNT_MAX], paramterCount = 0;
14 int bleCarMode = MODE_NONE;

```

```
15
16 #define STRIP_I2C_ADDRESS 0x20
17 #define STRIP_LEDS_COUNT 10
18
19 u8 colorPos = 0;
20 u8 colorStep = 50;
21 u8 stripDisplayMode = 1;
22 u8 currentLedIndex = 0;
23 u16 stripDisplayDelay = 100;
24 u32 lastStripUpdateTime = 0;
25 Freenove_WS2812B_Controller strip(STRIP_I2C_ADDRESS, STRIP_LEDS_COUNT, TYPE_GRB);
26
27 void setup() {
28     pinsSetup();
29     Serial.begin(9600);
30     servoSetup();
31     while (!strip.begin());
32     strip.setAllLedsColor(0xFF0000);
33 }
34
35 void loop() {
36     if (millis() - lastUploadVoltageTime > UPLOAD_VOL_TIME) {
37         upLoadVoltageToApp();
38         lastUploadVoltageTime = millis();
39     }
40     if (stringComplete) {
41         String inputCommandArray[COMMANDS_COUNT_MAX];
42
43         String inputStringTemp = inputStringBLE;
44         for (u8 i = 0; i < COMMANDS_COUNT_MAX; i++) {
45             int index = inputStringTemp.indexOf(INTERVAL_CHAR);
46             if (index < 0) {
47                 break;
48             }
49             paramterCount = i; //
50             inputCommandArray[i] = inputStringTemp.substring(0, index);
51             inputStringTemp = inputStringTemp.substring(index + 1);
52             paramters[i] = inputCommandArray[i].toInt();
53         }
54         stringComplete = false;
55         inputStringBLE = "";
56
57         char commandChar = inputCommandArray[0].charAt(0);
58         switch (commandChar)
```

```
59 {
60     case ACTION_MOVE:
61         if (paramterCount == 2) {
62             motorRun(paramters[1], paramters[2]);
63         }
64         break;
65     case ACTION_CAR_MODE:
66         if (paramterCount == 1) {
67             bleCarMode = paramters[1];
68             switch (bleCarMode)
69             {
70                 case MODE_NONE: case MODE_GRAVITY:
71                     resetCarAction();
72                     writeServo(OA_SERVO_CENTER);
73                     break;
74                 case MODE_ULTRASONIC:
75                     oa_CalculateVoltageCompensation();
76                     break;
77                 case MODE_TRACKING:
78                     tk_CalculateVoltageCompensation();
79                     break;
80                 default:
81                     break;
82             }
83         }
84         break;
85     case ACTION_BUZZER:
86         if (paramterCount == 1) {
87             setBuzzer(paramters[1]);
88         }
89         break;
90     case ACTION_RGB:
91         if (paramterCount == 4) {
92             stripDisplayMode = paramters[1];
93             switch (stripDisplayMode)
94             {
95                 case 0:
96                     colorStep = 5;
97                     stripDisplayDelay = 100;
98                     break;
99                 case 1:
100                     colorStep = 50;
101                     stripDisplayDelay = 50;
102                     break;
```

```
103         case 2:
104             colorStep = 5;
105             stripDisplayDelay = 500;
106             break;
107         case 3:
108             break;
109         default:
110             break;
111     }
112 }
113 break;
114 default:
115     break;
116 }
117 }
118 switch (bleCarMode)
119 {
120     case MODE_NONE: case MODE_GRAVITY:
121         break;
122     case MODE_ULTRASONIC:
123         upLoadSonarValueToApp();
124         updateAutomaticObstacleAvoidance();
125         break;
126     case MODE_TRACKING:
127         updateAutomaticTrackingLine();
128         break;
129     default:
130         break;
131 }
132 static u8 lastColor[3];
133 switch (stripDisplayMode)
134 {
135     case 0:
136         if (millis() - lastStripUpdateTime > stripDisplayDelay)
137         {
138             for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
139                 strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
140             }
141             strip.show();
142             colorPos += colorStep;
143             lastStripUpdateTime = millis();
144         }
145         break;
146     case 1:
```

```
147     if (millis() - lastStripUpdateTime > stripDisplayDelay)
148     {
149         strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
150         currentLedIndex++;
151         if (currentLedIndex == STRIP_LEDS_COUNT)
152         {
153             currentLedIndex = 0;
154             colorPos += colorStep; //
155         }
156         lastStripUpdateTime = millis();
157     }
158     break;
159 case 2:
160     colorPos = colorStep;
161     if (millis() - lastStripUpdateTime > stripDisplayDelay)
162     {
163         static bool ledState = true;
164         if (ledState)
165         {
166             strip.setAllLedsColor(paramters[2], paramters[3], paramters[4]);
167         }
168         else
169         {
170             strip.setAllLedsColor(0x00);
171         }
172         ledState = !ledState;
173         lastStripUpdateTime = millis();
174     }
175     break;
176 case 3:
177     if (lastColor[0] != paramters[2] || lastColor[1] != paramters[3] || lastColor[2] !=
178 paramters[4])
179     {
180         strip.setAllLedsColor(paramters[2], paramters[3], paramters[4]);
181         lastColor[0] = paramters[2];
182         lastColor[1] = paramters[3];
183         lastColor[2] = paramters[4];
184     }
185     break;
186 default:
187     break;
188 }
189 }
```

```

191 void upLoadVoltageToApp() {
192     int voltage = 0;
193     if (getBatteryVoltage()) {
194         voltage = batteryVoltage * 1000;
195     }
196     String sendString = String(ACTION_GET_VOLTAGE) + String(INTERVAL_CHAR) + String((voltage)) +
197     String(INTERVAL_CHAR);
198     Serial.println(sendString);
199 }
200
201 extern int distance[3];
202 void upLoadSonarValueToApp() {
203     String sendString = String(ACTION_ULTRASONIC) + String(INTERVAL_CHAR) +
204     String((distance[1])) + String(INTERVAL_CHAR);
205     Serial.println(sendString);
206 }
207
208 void serialEvent() {
209     while (Serial.available()) {
210         char inChar = (char)Serial.read();
211         inputStringBLE += inChar;
212         if (inChar == '\n') {
213             stringComplete = true;
214         }
215     }
216 }

```

There are three action modes. MODE_GAVITY, MODE_ULTRASONIC and MODE_TRACKING.

```

case ACTION_CAR_MODE:
    if (paramterCount == 1) {
        bleCarMode = paramters[1];
        switch (bleCarMode)
        {
            case MODE_NONE: case MODE_GRAVITY:
                resetCarAction();
                writeServo(OA_SERVO_CENTER);
                break;
            case MODE_ULTRASONIC:
                oa_CalculateVoltageCompensation();
                break;
            case MODE_TRACKING:
                tk_CalculateVoltageCompensation();
                break;
            default:

```

```

        break;
    }
}
break;

```

In the app, four LED display modes can be switched and different color change ranges and color change periods are set according to different modes. That is, the values of the variables `colorStep` and `stripDisplayDelay`.

```

switch (commandChar)
{
    case ACTION_MOVE:
        if (paramterCount == 2) {
            motorRun(paramters[1], paramters[2]);
        }
        break;
    case ACTION_CAR_MODE:
        if (paramterCount == 1) {
            bleCarMode = paramters[1];
            switch (bleCarMode)
            {
                case MODE_NONE: case MODE_GRAVITY:
                    resetCarAction();
                    writeServo(OA_SERVO_CENTER);
                    break;
                case MODE_ULTRASONIC:
                    oa_CalculateVoltageCompensation();
                    break;
                case MODE_TRACKING:
                    tk_CalculateVoltageCompensation();
                    break;
                default:
                    break;
            }
        }
        break;
    case ACTION_BUZZER:
        if (paramterCount == 1) {
            setBuzzer(paramters[1]);
        }
        break;
    case ACTION_RGB:
        if (paramterCount == 4) {
            stripDisplayMode = paramters[1];
            switch (stripDisplayMode)

```

```
        {
            case 0:
                colorStep = 5;
                stripDisplayDelay = 100;
                break;
            case 1:
                colorStep = 50;
                stripDisplayDelay = 50;
                break;
            case 2:
                colorStep = 5;
                stripDisplayDelay = 500;
                break;
            case 3:
                break;
            default:
                break;
        }
    }
    break;
default:
    break;
}
}
```

In loop(), the LED is displayed based on the received mode and parameters.

```
static u8 lastColor[3];
switch (stripDisplayMode)
{
    case 0:
        if (millis() - lastStripUpdateTime > stripDisplayDelay)
        {
            for (int i = 0; i < STRIP_LEDS_COUNT; i++) {
                strip.setLedColorData(i, strip.Wheel(colorPos + i * 25));
            }
            strip.show();
            colorPos += colorStep;
            lastStripUpdateTime = millis();
        }
        break;
    case 1:
        if (millis() - lastStripUpdateTime > stripDisplayDelay)
        {
            strip.setLedColor(currentLedIndex, strip.Wheel(colorPos));
        }
    }
}
```

```
        currentLedIndex++;
        if (currentLedIndex == STRIP_LEDS_COUNT)
        {
            currentLedIndex = 0;
            colorPos += colorStep; //
        }
        lastStripUpdateTime = millis();
    }
    break;
case 2:
    colorPos = colorStep;
    if (millis() - lastStripUpdateTime > stripDisplayDelay)
    {
        static bool ledState = true;
        if (ledState)
        {
            strip.setAllLedsColor(paramters[2], paramters[3], paramters[4]);
        }
        else
        {
            strip.setAllLedsColor(0x00);
        }
        ledState = !ledState;
        lastStripUpdateTime = millis();
    }
    break;
case 3:
    if (lastColor[0] != paramters[2] || lastColor[1] != paramters[3] || lastColor[2] !=
paramters[4])
    {
        strip.setAllLedsColor(paramters[2], paramters[3], paramters[4]);
        lastColor[0] = paramters[2];
        lastColor[1] = paramters[3];
        lastColor[2] = paramters[4];
    }
    break;
default:
    break;
}
}
```

What is next?

THANK YOU for participating in this learning experience!

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us: support@freenove.com

We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.

<http://www.freenove.com/>

Thank you again for choosing Freenove products.